

A PRACTITIONER'S DEEP DIVE

SPARK

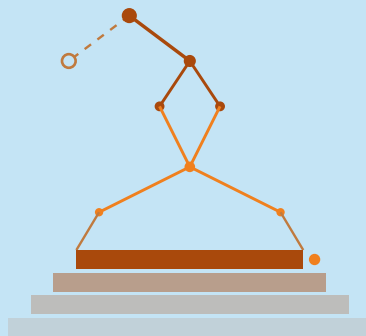
QUERY OPTIMIZATION

and the

ICEBERG

LAKEHOUSE

*Why your queries run the way they do — from Catalyst
to the shuffle to the metadata layer — and how to change it*



Alan Phan

2026

Preface

[Section to be written.]

Who This Book Is For

[Section to be written.]

How to Use This Book

[Section to be written.]

Conventions Used in This Book

[Section to be written.]

The four callouts look like this:

Note

Marks a version gotcha, a clarification, or a pointer to a later chapter.

Warning

Marks a default or a pattern that silently costs money, correctness, or data.

Tip

Marks a diagnostic shortcut — a UI panel to check, a one-line command.

Analogy

Marks a plain-language comparison used to build intuition for a mechanism — never a substitute for the mechanism itself, and always pushed only as far as it stays accurate. Skippable if it doesn't click for you; the surrounding prose never depends on it.

Contents

Preface	i
Who This Book Is For	ii
How to Use This Book	iii
Conventions Used in This Book	iv
I How Spark Executes a Query	1
1 The Shape of a Spark Application	2
1.1 A query, end to end	2
1.2 Driver, executors, and the cluster manager	3
1.3 Deploy modes and Spark on Kubernetes	7
1.4 Jobs, stages, and tasks	8
1.5 Narrow versus wide dependencies	10
1.6 Reading the UI for the first time	12
1.7 Summary	13
2 From SQL to Execution: the Catalyst Pipeline	14
2.1 The pipeline, overview	14
2.2 Unresolved to resolved: the catalog	16
2.3 Logical optimization: the rules that matter	17
2.4 Physical planning and strategies	18
2.5 Whole-stage code generation	19

2.6	Reading EXPLAIN	21
2.7	Summary	23
3	How an Executor Uses Memory and CPU	25
3.1	The JVM in one page: bytecode and just-in-time compilation	25
3.2	Heap layout and object representation	26
3.3	The executor memory model	27
3.4	Garbage collection: Parallel, G1, ZGC	31
3.5	Serialization and its cost	33
3.6	Summary	33
II	Making Queries Fast	35
4	The Shuffle	36
4.1	What a partition is: input partitions versus shuffle partitions	36
4.2	The shuffle write path	38
4.3	The shuffle read path	38
4.4	Sizing shuffle partitions	40
4.5	Shuffle compression	42
4.6	Push-based shuffle	43
4.7	Finding and removing unnecessary exchanges	43
4.8	Summary	44
5	Joins	46
5.1	Broadcast hash join	46
5.2	Shuffle hash join	48
5.3	Sort-merge join	48
5.4	Broadcast nested loop join	49
5.5	How the planner chooses	49
5.6	Forcing a strategy	50
5.7	Storage-partitioned joins for bucketed Iceberg tables	51
5.8	Summary	52

6	Adaptive Query Execution	53
6.1	Why runtime beats planning time	53
6.2	Coalescing shuffle partitions	54
6.3	Converting sort-merge to broadcast at runtime	54
6.4	The two broadcast thresholds	55
6.5	Skew-join handling: split and replicate	55
6.6	parallelismFirst and partition count	57
6.7	Reading an AQE plan	57
6.8	Summary	57
7	Partitioning, Bucketing, and Pruning	59
7.1	Partition pruning and the pushdown killers	59
7.2	Dynamic partition pruning	60
7.3	Bucketing to avoid a shuffle	61
7.4	repartition versus coalesce	62
7.5	Column pruning and ReadSchema	62
7.6	Summary	63
8	Caching and Reuse	64
8.1	When caching pays	64
8.2	Lazy cache() versus eager CACHE TABLE	65
8.3	Storage levels and the GC trade-off	65
8.4	How execution and storage memory borrow at runtime	66
8.5	Exchange and subquery reuse	68
8.6	Unpersist and cache thrashing	68
8.7	Summary	69
III	The Iceberg Lakehouse	70
9	Table Formats and the Iceberg Spec	71
9.1	From Hive tables to table formats	71
9.2	The Iceberg spec, format version 2	72

9.3	Snapshots and the metadata tree	73
9.3.1	The commit as an atomic pointer swap	74
9.4	Position deletes versus equality deletes	74
9.5	Iceberg, Delta Lake, and Hudi	75
9.6	Summary	76
10	Catalogs and Commits	78
10.1	What a catalog does	78
10.2	The Hive Metastore catalog	79
10.3	The REST catalog	79
10.4	The atomic pointer swap	80
10.5	Optimistic concurrency and retries	80
10.6	Catalog configuration in Spark	83
10.7	Summary	84
11	Iceberg Partitioning and Layout	85
11.1	Hidden partitioning and partition transforms	85
11.2	Partition evolution	87
11.3	Sort orders and Z-ordering	88
11.4	Target file size and the small-files problem	89
11.5	Write distribution modes	89
11.6	Summary	90
12	Reading Iceberg Efficiently	92
12.1	Manifest pruning and metadata filtering	92
12.2	Column statistics and predicate pushdown into Iceberg	93
12.3	Metrics modes: full, truncate, counts, none	93
12.4	Bloom filters	95
12.5	Vectorized reads	95
12.6	Split planning	95
12.7	The metadata tables as a diagnostic	96
12.8	Summary	97

13 Writing and Maintaining Iceberg	98
13.1 Copy-on-write versus merge-on-read	98
13.2 MERGE, UPDATE, and DELETE	99
13.3 Write distribution and skew on write	101
13.4 Compacting data files	101
13.5 Rewriting manifests	102
13.6 Snapshot expiration and orphan files	102
13.7 Metadata management for streaming tables	103
13.8 Summary	103
 IV Operating Spark	 105
14 Tuning Resources and Configuration	106
14.1 Executor sizing from the mechanism	106
14.2 Dynamic allocation	108
14.3 Parallelism defaults	109
14.4 The external and push-based shuffle service	109
14.5 Configuration precedence and profiles	110
14.6 Kyuubi and multi-tenancy	110
14.7 Summary	112
 15 Debugging and Profiling	 113
15.1 The Spark UI as a diagnostic	113
15.2 Event logs and the history server	114
15.3 Reading task, shuffle, spill, and GC metrics	115
15.4 Common failure signatures	116
15.5 async-profiler and flame graphs	118
15.6 Debugging query plans	119
15.7 Summary	119
 16 Streaming into the Lakehouse	 121
16.1 The micro-batch model and the unbounded table	121

16.2	Sources and sinks	122
16.3	Event time and watermarking	123
16.4	Stateful operations and the state store	124
16.5	Output modes and triggers	124
16.6	Exactly-once and the commit	125
16.7	Writing to Iceberg: cadence, small files, maintenance	126
16.8	Summary	127
V	Internals	129
17	Tungsten and Code Generation	130
17.1	The Tungsten binary row format	130
17.2	Off-heap memory management	131
17.3	Cache-aware computation	132
17.4	Whole-stage code generation internals	133
17.5	Expression code generation	135
17.6	Columnar processing and vectorization	135
17.7	Summary	136
18	The Shuffle Framework and the Scheduler	137
18.1	The shuffle manager and shuffle writers	137
18.2	The map output tracker	139
18.3	The DAG scheduler and the task scheduler	139
18.4	RPC and block transfer	140
18.5	Locality and speculation	140
18.6	Summary	142
19	DataSource V2 and Extensions	143
19.1	The DataSource V2 read and write API	143
19.2	Pushdown negotiation	144
19.3	How Iceberg reports splits and statistics	146
19.4	Custom Catalyst rules and SQL extensions	146

19.5	Plugins	147
19.6	Summary	148
A	The Reference Environment and the Running Example	149
A.1	The stack	149
A.2	The cluster shape	149
A.3	The running example: orders	150
A.4	A columnar-format primer	151
B	Configuration Reference	153
B.1	Memory (Chapter 3)	153
B.2	Shuffle (Chapter 4, Chapter 18)	154
B.3	Adaptive Query Execution (Chapter 6)	155
B.4	Joins (Chapter 5)	155
B.5	Partitioning and pruning (Chapter 7)	156
B.6	Iceberg catalog and commits (Chapter 10)	156
B.7	Iceberg table properties (Chapter 11, Chapter 12, Chapter 13)	157
B.8	Streaming (Chapter 16)	157
B.9	Dynamic allocation and scheduling (Chapter 14, Chapter 18)	158
C	Reading Query Plans	159
C.1	The six EXPLAIN modes	159
C.2	Reading a physical plan	160
C.3	Scan operators	161
C.4	Exchange operators	162
C.5	Sort operators	163
C.6	Join operators	163
C.7	Aggregation operators	164
C.8	Filter and Project	165
C.9	Whole-stage codegen markers	165
C.10	AQE nodes in a final plan	166
C.11	Predicate-pushdown killers	168

D Glossary	170
E Further Reading	174
References	177

List of Tables

1.1	The specimen at a glance, matching Figure 1.3.	10
1.2	Narrow versus wide, by operation. <code>coalesce</code> is the one surprise: it looks like <code>repartition</code> but avoids a shuffle by only ever merging partitions, never splitting or reshuffling keys (Section 7.4).	11
2.1	The analogy above, mapped exactly onto the pipeline in Figure 2.1. . .	16
3.1	The three collectors, at a glance.	32
4.1	Written once, read immediately (shuffle) wants decompression speed; written once, read many times (storage) can afford to spend more on ratio.	42
6.1	Both the relative check ($5\times$ median) and the absolute floor (256 MB) must pass before AQE calls a partition skewed. The 220 MB partition shows why the floor matters: it clears the relative bar but is still an ordinary amount of data to hold in one task, not a genuine straggler.	56
9.1	Where the three table formats put their defaults	76
10.1	Iceberg commit-retry table properties	82
11.1	Common Iceberg partition transforms	86
12.1	The pruning layers a scan passes through, in order	93
13.1	Choosing copy-on-write versus merge-on-read	99
14.1	Kyuubi’s share levels trade isolation for cold-start cost by changing what the registry key is derived from.	111

18.1	Which shuffle writer a given shuffle gets, in the order Spark checks . .	138
A.1	The reference stack	150
A.2	orders schema	150

List of Figures

1.1	One application: a single driver process that plans and coordinates, and a set of executor processes that run tasks and cache data. The cluster manager — here the Kubernetes API — hands out the containers the processes run in.	4
1.2	The specimen query from submission to result. The driver compiles and schedules but never touches the data; executors do all the reading, shuffling, and computing; the cluster manager only supplies the pods.	6
1.3	The specimen as one job of three stages. Stage 0 scans and filters orders; stage 1 scans <code>dim_product</code> ; both write shuffle files partitioned by <code>product_id</code> . Stage 2 reads from both, joins, and aggregates. Each stage runs one task per input partition. One task in stage 2 runs far longer than its peers — the first sighting of skew.	9
1.4	A narrow dependency: each output partition is built from one input partition, so the work pipelines in place. A wide dependency: each output partition draws from every input partition, so the data must be redistributed — a shuffle — and the stage ends.	10
2.1	The compilation pipeline. Parsing produces a tree that is pure syntax. Analysis resolves names against the catalog. Logical optimization rewrites the tree with rules. Physical planning picks concrete operators and shuffle points. Code generation emits Java that the JIT compiles. The side notes name what each stage decides.	15

3.1	An executor's memory: the JVM heap (reserved / unified / user), the optional off-heap pool, and the overhead the process needs outside both. The cluster manager's container limit must cover all three. Numbers are worked for an 8 GB heap with 2 GB overhead.	30
3.2	A stop-the-world pause freezes every task slot on the executor at once, not one at a time. If the pause outlasts the heartbeat interval, the driver concludes the executor is gone and re-runs its work elsewhere — so a GC problem shows up as lost executors and repeated stages, not just as slow ones.	32
4.1	The shuffle write and read paths. The map task never talks to the reduce task directly: it registers its file's location with the driver's map output tracker, and the reduce task asks the tracker where to fetch from. . . .	39
4.2	For a fixed shuffle size, size per task is exactly (shuffle bytes) / (partition count) — an exact relationship, not a rule of thumb. For a 100 GB shuffle the default 200 partitions puts 512 MB in every task, well past where it spills; 512–800 partitions lands each task in the 128–200 MB band. . .	40
5.1	The four join strategies as plan shapes. Broadcast hash join shuffles nothing on the large side; shuffled hash join shuffles both sides but sorts neither; sort-merge join shuffles and sorts both; broadcast nested loop join broadcasts one side to evaluate a condition that is not an equality. . . .	47
6.1	Without AQE, one oversized partition runs as a single task long after the rest of the stage has finished. AQE splits it into sub-partitions and replicates the small side across them, turning one straggler into several parallel, evenly-sized tasks.	56
7.1	Without DPP, pruning happens last: the whole fact table is scanned before the join discards most of it. With DPP, the small dimension side runs first and its matching keys are injected as a filter <i>before</i> the fact scan, so pruning happens first instead.	61

-
- 8.1 Borrowing is symmetric; reclaiming is not. Storage can spread into idle execution space, but the moment a join or sort needs that space back, execution evicts the borrower's cached blocks to reclaim it — storage has no equivalent power over a running task. 67
- 9.1 The metadata tree for one snapshot of orders. The catalog resolves to one table metadata file, which points to one manifest list, which points to the manifests that make up this snapshot, which list the actual data and delete files. Every level is immutable; a new commit writes new files at whichever levels changed and leaves the rest untouched. 72
- 10.1 Two writers racing for the same base version. The compaction job's swap lands first and succeeds; the INSERT session's swap against the same stale base is rejected, so it re-reads the new current version, rebuilds its manifest on top of it, and retries — succeeding one version later, with no error surfaced to either caller. 82
- 11.1 After evolving orders from `month(order_ts)` to `day(order_ts)`, both specs coexist under the same current snapshot. Old manifests keep describing month-partitioned files; new manifests describe day-partitioned files. One scan reads both, pruning each manifest against its own recorded spec. 88
- 12.1 The four pruning layers, narrowing top to bottom. Each layer only ever has to consider what the layer above it did not already exclude — a file skipped by the manifest list is never even checked against a bloom filter or row-group statistics. 94
- 13.1 The same UPDATE under copy-on-write and merge-on-read. CoW pays once, at write time, and every later read is plain data with no merge; MoR pays a small write-time cost and pushes the rest onto every read until the next compaction. 100

- 15.1 The UI read top to bottom: each tab narrows the search before the next one is worth opening. Starting at task metrics without first naming the operator and the stage is the most common way to waste time reading a plan that turns out not to be the bottleneck. 114
- 16.1 One iteration of the micro-batch loop. The checkpoint offset is only written after the Iceberg commit succeeds, which is what makes a restart safe: replaying from the last checkpointed offset re-runs exactly the range that either never committed or already did, never anything in between. . . . 122
- 17.1 One UnsafeRow: a null bitset, one 8-byte slot per field, and a variable-length region past the last fixed slot. A fixed field's slot holds its value directly; a variable field's slot holds an (offset, length) pointer into the variable region of the same buffer. 131
- 18.1 The scheduler's five components. The DAG scheduler hands the task scheduler a set of runnable tasks and is only reinvoked when the map output tracker reports a lost executor's output as gone; the task scheduler reads fetch locations from the tracker and issues its own small control messages over RPC, kept separate from the large shuffle bytes on the block-transfer path. 141
- 19.1 The same negotiation, two outcomes. A plain comparison resolves against partition values and column statistics, so Iceberg reports it fully handled and Spark drops the filter from the plan. A CAST on the same column cannot be resolved the same way, so Iceberg reports it unhandled and Spark keeps a Filter node to enforce it — still correct, just with no pruning benefit. 145

Part I

How Spark Executes a Query

Chapter 1

The Shape of a Spark Application

A query you would actually write — a week of orders joined to a product dimension, aggregated to an on-time delivery rate — runs in four minutes. This chapter does not make it faster. It uses it as a specimen: you run it, open the Spark UI, and learn to name every moving part — the driver, the executors, the one job, its stages, the shuffle between them, and the one task doing more than the rest. By the end you can read a running job instead of watching a progress bar.

1.1 A query, end to end

Here is the specimen. It asks, for each product category, what fraction of orders in the first week of March were delivered within their promised window. It runs against the orders table and the dim_product dimension introduced in Appendix A; orders is an Iceberg table partitioned by day(order_ts), holding a few hundred million rows for the month.

```
1 SELECT p.category ,
2         AVG(CASE WHEN o.delivery_days <= o.promised_days THEN
3             1.0 ELSE 0.0 END) AS on_time_rate
4 FROM orders o
5 JOIN dim_product p ON o.product_id = p.product_id
6 WHERE o.order_ts >= TIMESTAMP '2026-03-01 00:00:00'
```

```
6   AND o.order_ts <  TIMESTAMP '2026-03-08 00:00:00'
7 GROUP BY p.category
8 ORDER BY on_time_rate;
```

Listing 1.1: The specimen query

Submit it and Spark does not simply “run the SQL.” The text is parsed into a tree, resolved against the catalog, rewritten by the optimizer, turned into a plan of physical operators, and compiled to Java bytecode. That whole compilation is Chapter 2’s subject; for now, take it as given that by the time anything executes, Spark holds a *physical plan*: a tree of operators such as *scan*, *filter*, *exchange* (a shuffle), *sort-merge join*, and *hash aggregate*.

Executing that plan is a distributed affair. The plan is split into a *job*, the job into *stages*, and each stage into *tasks* that run in parallel on *executors*. The result flows back to the process that submitted the query, the *driver*. On the reference cluster this specimen produces one job, three stages, and a few hundred tasks, and it takes about four minutes.

Why four minutes and not one, or forty? That question is what the rest of the book answers. This chapter’s job is narrower: to give you the vocabulary and the tools to *see* the structure of the run, so that later chapters have something concrete to change. Everything that follows — joins, the shuffle, adaptive execution, the Iceberg metadata layer — is a way of moving less data, moving it less often, or moving it more evenly. You cannot decide to move data differently until you can see where it moves now.

1.2 Driver, executors, and the cluster manager

A Spark *application* is one driver plus a set of executors, all working on your queries until the application exits (Figure 1.1).

The **driver** is the process that runs your program. It holds the `SparkSession`, compiles each query into a plan, breaks the plan into stages and tasks, hands tasks to executors, tracks their progress, and gathers results. It is a single process and a single point of failure: if the driver dies, the application dies. It is also where a `collect()` or a `toPandas()` lands every row it pulls back, which is the most common way to run the driver out of memory (Chapter 3, Section 15.3).

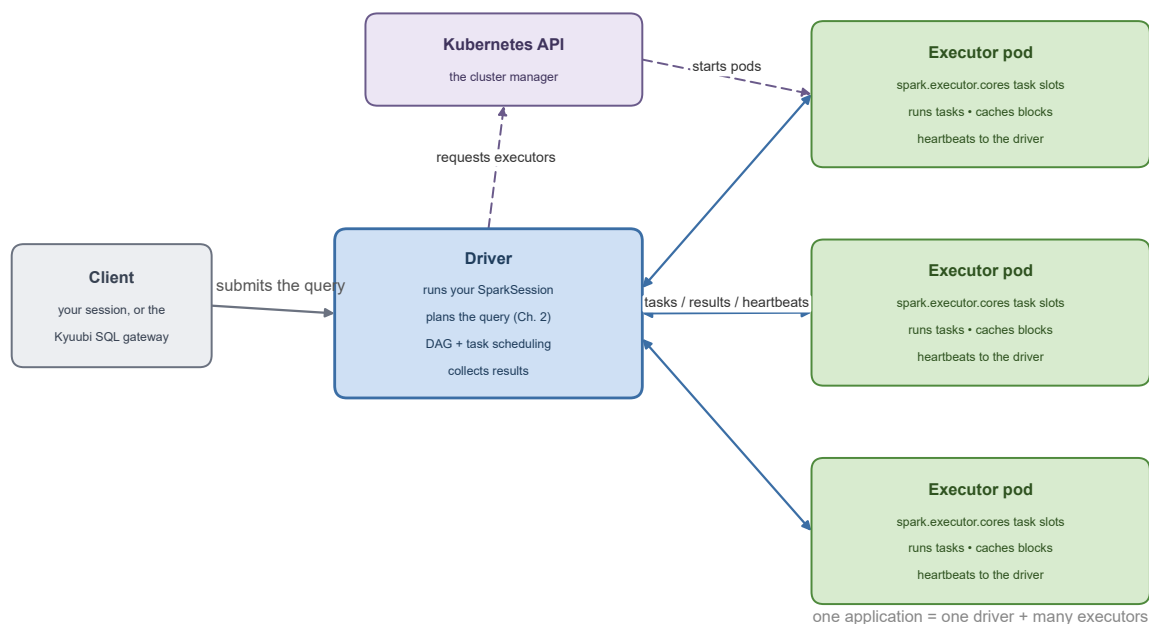


Figure 1.1: One application: a single driver process that plans and coordinates, and a set of executor processes that run tasks and cache data. The cluster manager — here the Kubernetes API — hands out the containers the processes run in.

An **executor** is a JVM process that does the actual work. Each executor has a fixed number of *task slots* — one per core, set by `spark.executor.cores` — and runs one task per slot at a time. Executors also hold cached blocks and the map-side output of shuffles, and they send regular heartbeats to the driver so it knows they are alive. An application typically has many executors and exactly one driver.

The **cluster manager** owns the machines and hands out the containers that the driver and executors run in. Spark supports several: its own standalone manager, Hadoop YARN, and Kubernetes (Apache Mesos support was removed in Spark 3.2). The reference stack uses Kubernetes, so “container” means “pod” and the cluster manager is the Kubernetes API server.

Note

The driver–executor split is why two very different failures look different. An executor lost to an out-of-memory kill takes its tasks down, and Spark simply re-runs them elsewhere; the job slows but survives. The driver running out of memory — usually from pulling too much data back — kills the whole application at once, with no retry.

Analogy

A busy restaurant kitchen has the same three roles. The **driver** is the head chef at the pass: reading each ticket, deciding how the dish comes together, calling out instructions — and never touching a pan. The **executors** are the line cooks, each at their own station with their own pans (task slots), doing the actual cutting and cooking and calling back to the pass when a plate is ready. The **cluster manager** is the staffing agency: when the chef needs more hands on a busy night, it sends cooks over, but it never runs a station itself. Push it further than that and it breaks — no kitchen survives a chef who cannot cook a single plate — but the division of labour is exactly right: one planner, many workers, and an outside party that only ever supplies the workers.

Following the specimen from submission to result

Putting the three roles together, here is what happens end to end when you run the query from Listing 1.1, step by step (Figure 1.2).

1. **Submit.** Your SQL client — here a connection to the Kyuubi gateway — sends the query *text* to the driver. Nothing is parsed on your side.
2. **Compile (driver).** The driver parses the text, resolves orders and `dim_product` against the catalog, runs the optimizer, and produces a physical plan (Chapter 2). Its DAG scheduler then cuts that plan into three stages at the shuffle and builds a set of tasks for each — one task per input partition.
3. **Acquire executors (cluster manager).** If the driver does not already have enough executors, it asks Kubernetes for more pods. Under Kyuubi the pool is usually already warm, so this step does nothing and the query starts at once.
4. **Run the map stages (executors).** The driver sends stage 0 and stage 1 tasks to free task slots, each task carrying its plan fragment and the id of the one partition it must process. Stage 0's roughly 180 tasks each read one file split of the March orders partitions from GCS, apply the `order_ts` filter, and write small files to the executor's *local* disk, bucketed by `product_id`. Stage 1 does the same for

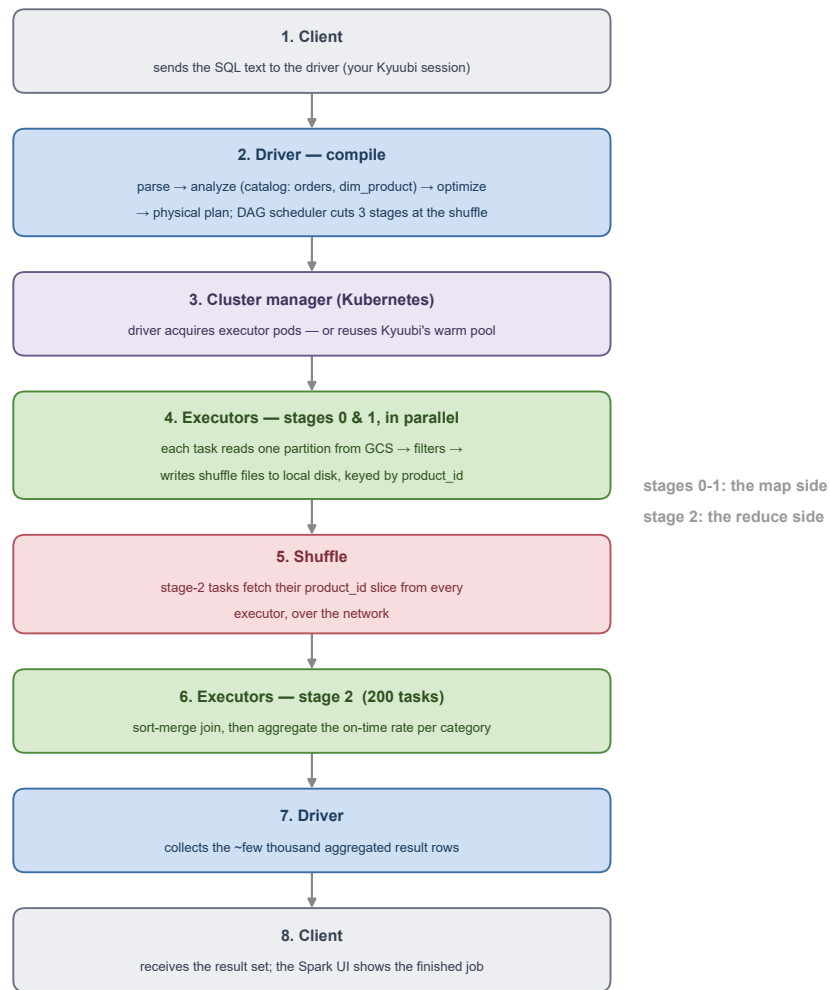


Figure 1.2: The specimen query from submission to result. The driver compiles and schedules but never touches the data; executors do all the reading, shuffling, and computing; the cluster manager only supplies the pods.

`dim_product`. The two stages run concurrently, and every task reports its metrics to the driver as it finishes.

5. **Shuffle.** Once both map stages are done the driver launches stage 2 — 200 tasks, one per value of `spark.sql.shuffle.partitions`. Each stage-2 task connects to *every* executor that ran a map task and fetches the shuffle files for its slice of `product_id`. This all-to-all transfer over the network is the shuffle, and it is why the map side (stages 0–1) and the reduce side (stage 2) are separate stages.
6. **Run the reduce stage (executors).** Each stage-2 task now holds every row for its products, from both tables. It sorts and merges them, performs the join, and aggregates the on-time rate per category.
7. **Collect (driver).** The result is one row per category — a handful of rows — so the driver gathers them from the stage-2 tasks. A query returning millions of rows would overwhelm the driver at exactly this step (Chapter 3).
8. **Return.** The driver sends the result set to the client and marks the job complete. The Spark UI now shows the finished job, its three stages, and every task’s metrics — the view you learn to read in Section 1.6.

Throughout, the driver is also doing bookkeeping the diagram leaves out: tracking which tasks have finished, re-sending any that failed to a different executor, and updating the UI. It never reads or writes the data itself — only the plan, the schedule, and the final small result.

1.3 Deploy modes and Spark on Kubernetes

Where the *driver* runs is the *deploy mode*. In **client mode** the driver runs in whatever process launched the application — your shell, a notebook kernel, or a long-lived gateway. In **cluster mode** the launcher asks the cluster manager to start the driver as just another container, then disconnects. Client mode is convenient for interactive work because the driver is local to you; cluster mode is standard for scheduled jobs because nothing on your side has to stay running.

On Kubernetes the sequence for a cluster-mode job is: `spark-submit` calls the Kubernetes API to create a *driver pod*; the driver pod starts, and its Spark code calls the same API to create *executor pods*; each executor pod starts, connects back to the driver, and registers its task slots. From then on the driver schedules tasks onto executors directly, and the Kubernetes API is only involved again when pods are added or removed — for example under dynamic allocation (Section 14.1).

The reference stack does not usually run `spark-submit` per query. **Kyuubi** runs a long-lived driver (a persistent `SparkSession`, one per user or group) and you send SQL to it over a connection, the way you would to any database. The engine, stages, and tasks are all still there; Kyuubi just keeps the driver warm so you are not paying startup cost on every query and so many users can share a pool of executors.

1.4 Jobs, stages, and tasks

Spark transformations are *lazy*: `filter`, `join`, `groupBy`, and the rest build up a plan but run nothing. Work starts only when an *action* asks for a result — a write, a count, a collect, or the implicit collect that returns rows to a SQL client. Each action triggers one **job**.

Within a job, the driver's *DAG scheduler* — named for the **directed acyclic graph** of stages it builds, since a plan's dependencies only ever point forward, never back into a cycle — cuts the plan into **stages**. The cut always falls at a shuffle: a stage is the longest run of work that can be pipelined without moving data between executors. Everything up to a shuffle goes in one stage; the shuffle ends it; what comes after starts a new one. A stage that reads a shuffle cannot begin until the stage that writes it has finished, which is why stage boundaries are also the points where a job can stall.

A stage is executed as a set of **tasks** — one per partition of the stage's input, each doing identical work on its own slice of the data. A task is the unit Spark schedules onto an executor's task slot; one task occupies one core for its duration. The number of tasks in a stage is therefore the number of partitions, and it sets how much parallelism the stage can use: 8 tasks on a cluster with 200 free slots leaves 192 idle.

Figure 1.3 shows the specimen's shape. Because `dim_product` has no statistics that

would let the planner broadcast it (an Iceberg table usually does not, until you `ANALYZE` it — Chapter 5), the planner chooses a sort-merge join: both inputs are shuffled so that rows with the same `product_id` land on the same executor. That gives:

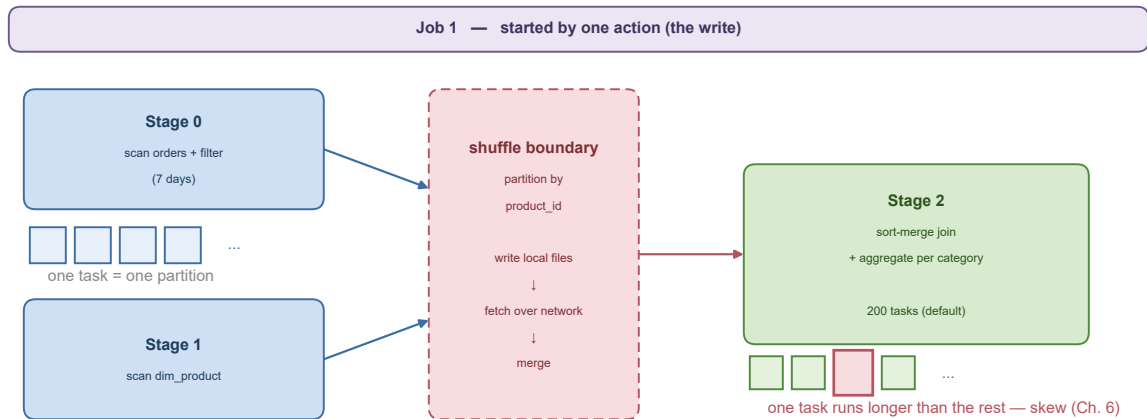


Figure 1.3: The specimen as one job of three stages. Stage 0 scans and filters orders; stage 1 scans `dim_product`; both write shuffle files partitioned by `product_id`. Stage 2 reads from both, joins, and aggregates. Each stage runs one task per input partition. One task in stage 2 runs far longer than its peers — the first sighting of skew.

- **Stage 0** — scan the March partitions of `orders`, apply the timestamp filter, and write shuffle files keyed by `product_id`. Roughly 180 tasks, one per input file split.
- **Stage 1** — scan `dim_product` and write shuffle files keyed by `product_id`. A handful of tasks; the table is tiny.
- **Stage 2** — fetch both sides' shuffle files, merge them on `product_id`, join, and aggregate per category. 200 tasks, because `spark.sql.shuffle.partitions` defaults to 200 (Chapter 4). The final `ORDER BY` is small enough to finish inside this stage.

Stage	Reads	Tasks	Writes / does
0	orders, March partitions, filtered	~180	shuffle files, keyed by product_id
1	dim_product	a handful	shuffle files, keyed by product_id
2	stage 0 + stage 1 shuffle output	200	sort-merge join, aggregate, sort

Table 1.1: The specimen at a glance, matching Figure 1.3.

The ORDER BY on the aggregated result is cheap — a few thousand rows — so it does not add a stage here. On a larger result it would.

1.5 Narrow versus wide dependencies

The reason a shuffle ends a stage is the kind of dependency it creates between partitions (Figure 1.4).

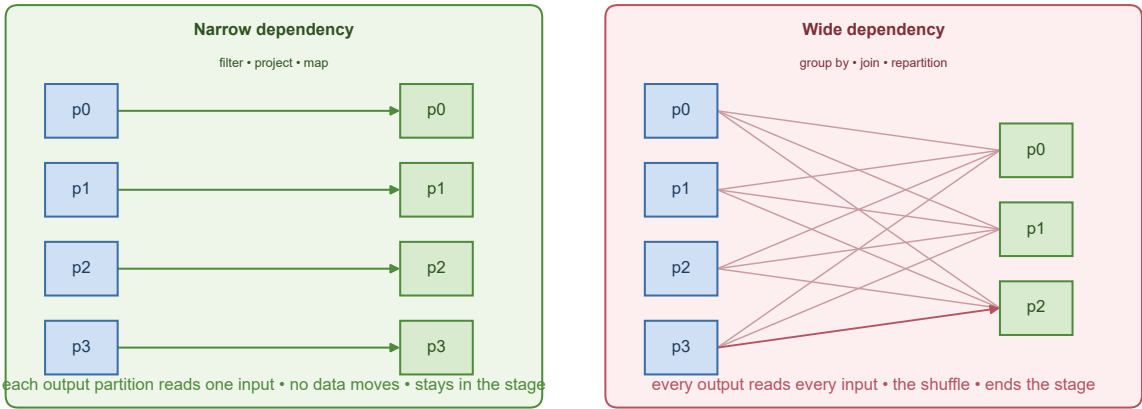


Figure 1.4: A narrow dependency: each output partition is built from one input partition, so the work pipelines in place. A wide dependency: each output partition draws from every input partition, so the data must be redistributed — a shuffle — and the stage ends.

A **narrow** dependency is one where each output partition depends on a bounded,

known set of input partitions — usually exactly one. `filter`, `select`, and `map` are narrow: partition 3 of the output is partition 3 of the input with a function applied. Narrow operations chain together with no data movement, which is why an entire run of them collapses into a single stage and executes as one pipelined pass per partition.

A **wide** dependency is one where an output partition can depend on *every* input partition. `groupBy`, `join` (unless one side is broadcast), `distinct`, `repartition`, and window functions are wide: to put all rows for `product_id = 47` in one place, Spark must look at every input partition, because any of them might hold such a row. That redistribution is the shuffle, and it is the wall between stages.

Operation	Dependency	Why
<code>filter</code> , <code>select</code> , <code>withColumn</code>	narrow	one input partition maps to one output partition
<code>union</code>	narrow	partitions are concatenated, no keys compared
<code>coalesce(n)</code> (fewer partitions)	narrow	merges adjacent partitions without moving keys
<code>groupBy</code> , <code>join</code> (no broadcast), window functions	wide	equal keys from anywhere must co-locate
<code>distinct</code>	wide	compiles to an aggregation
<code>repartition(n)</code> , <code>orderBy</code> (global)	wide	explicit or ordering-driven redistribution

Table 1.2: Narrow versus wide, by operation. `coalesce` is the one surprise: it looks like `repartition` but avoids a shuffle by only ever merging partitions, never splitting or reshuffling keys (Section 7.4).

This is the single most useful idea in the book: **cost lives at stage boundaries**. Narrow work is cheap and scales cleanly. Wide work writes data to disk, moves it over the network, and re-reads it, and it forces the whole cluster to wait at a barrier. Almost every technique in Chapter 4 through Chapter 8 is about having fewer wide dependencies, making the ones you keep move less data, or making them move it more evenly.

1.6 Reading the UI for the first time

The Spark UI — served by the driver, on port 4040 by default, and preserved after the fact by the history server (Section 15.3) — has a tab for each level of the model.

SQL / DataFrame The query as a plan tree, annotated with the number of rows and the time spent at each operator after execution. This is where you confirm which join strategy ran and count the *Exchange* nodes; each Exchange is one shuffle.

Jobs One row per action. The specimen shows a single job.

Stages The stages of each job, with a timeline and, for every stage, a *Summary Metrics* table: the minimum, 25th percentile, median, 75th percentile, and maximum across the stage’s tasks for duration, shuffle read size, records, spill, and GC time.

Executors One row per executor: task slots, memory used, disk spilled, and total GC time. A cluster where one executor shows far more GC than the others is usually holding skewed data.

Storage Cached datasets and how much of each is actually resident.

For the specimen, open the Stages tab and look at stage 2’s Summary Metrics. The median task duration is a few seconds; the maximum is close to a minute. One task is doing a large multiple of the work of the typical task. That is *data skew*: one `product_id` — a heavily-discounted item that went briefly viral — holds far more orders than the others, so the task that got that product’s partition runs alone while the rest of the stage has finished. You cannot fix it yet. Chapter 6 and Chapter 5 will. But you can now *see* it, and seeing it is the prerequisite for everything else.

Tip

Four numbers tell you most of what is wrong with a slow stage: max versus median *task duration* (skew), *shuffle read/write* size (how much data the wall is moving), *spill* (not enough execution memory, Chapter 3), and *GC time* (too much, Section 3.4). Check them in that order.

1.7 Summary

You can now describe what happens when a query runs. The *driver* plans it and coordinates; *executors* run *tasks*, one per core, on partitions of the data; the *cluster manager* (Kubernetes here) provides the containers. An action triggers a *job*; the job is cut into *stages* at every shuffle; a stage runs one task per partition. A *narrow* dependency keeps work in place; a *wide* dependency forces a shuffle and ends the stage, and that is where a query spends most of its time. In the Spark UI you can find the job, its stages, and their tasks, identify a stage boundary as a shuffle from the Exchange nodes in the SQL tab, and spot an uneven stage from the gap between its maximum and median task time.

The specimen still takes four minutes. Chapter 2 opens up how Spark decided on this plan; Chapter 4 onward is how you change it.

Exercises

1. Run two queries against orders: `SELECT count(*) FROM orders` and `SELECT product_id, count(*) FROM orders GROUP BY product_id`. Open the Stages tab for each. Which produced a second stage, and why?
2. In the SQL tab for the specimen query, count the *Exchange* nodes. Match each one to a stage boundary in Figure 1.3.
3. In stage 2's Summary Metrics, record the median and maximum task duration and the maximum shuffle-read size. Which of the four numbers from the tip is the outlier?

Chapter 2

From SQL to Execution: the Catalyst Pipeline

A SQL string does not run. It is parsed into a tree, resolved against the catalog, rewritten by rules, turned into a physical plan by choosing strategies, and compiled to JVM bytecode. Every change you can make to a query's performance is a change to one of those stages. This chapter traces the specimen query from Chapter 1 through all of them and teaches you to read `EXPLAIN FORMATTED` well enough to predict the physical plan before you run it.

2.1 The pipeline, overview

Catalyst is the optimizer inside Spark SQL. It is a library for building and rewriting trees, plus a fixed sequence of passes that turn a query into an executable plan (Figure 2.1). Every DataFrame operation and every SQL statement goes through the same pipeline; the DataFrame API is just a second front end that builds the same tree the SQL parser does.

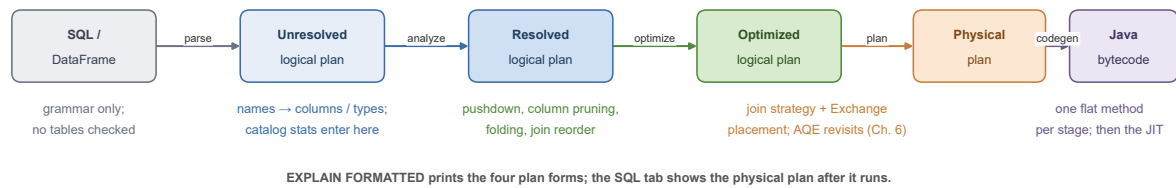


Figure 2.1: The compilation pipeline. Parsing produces a tree that is pure syntax. Analysis resolves names against the catalog. Logical optimization rewrites the tree with rules. Physical planning picks concrete operators and shuffle points. Code generation emits Java that the JIT compiles. The side notes name what each stage decides.

Analogy

Recall the kitchen from Section 1.2. The order goes through the same five stages there as a query does here. A waiter writes the spoken order onto a kitchen ticket in shorthand, without checking whether the kitchen actually has those ingredients tonight — that is *parsing*: pure syntax. The expediter then checks the ticket against the pantry and the menu, catching an order for a dish that is not served or an ingredient that is out — that is *analysis*, against the catalog. The head chef mentally simplifies the prep — “we’re already chopping onions for table 4, chop extra for this ticket too” — without changing what gets served: that is *logical optimization*. Stations and techniques are then assigned — grill or oven, this cook or that one — based on what is actually fastest tonight: that is *physical planning*. And a dish the kitchen has made a thousand times gets cooked by hands that no longer think through each step: that is *whole-stage code generation* (Section 2.5).

In the kitchen	In Catalyst	What changes
waiter writes the ticket	parse	text → syntax tree; nothing checked
expediter checks the pantry & menu	analyze	names resolved against the catalog
chef simplifies the prep	logical optimization	rule-based rewrites (Section 2.3)
stations & techniques assigned	physical planning	operators & shuffle points chosen
practiced hands, no thinking	whole-stage codegen	generated Java, then the JIT

Table 2.1: The analogy above, mapped exactly onto the pipeline in Figure 2.1.

Four representations matter, and you can print all of them (Section 2.6):

1. the **unresolved logical plan** — the parse tree; relations and columns are just names.
2. the **resolved (analyzed) logical plan** — names bound to real tables and typed columns.
3. the **optimized logical plan** — the same query, rewritten to do less work.
4. the **physical plan** — a tree of executable operators, with the shuffles made explicit as *Exchange* nodes.

The physical plan is what Chapter 1 called “the plan.” Reading it, and knowing which pass produced each part of it, is what lets you look at a query and say in advance how many stages it will have and where they will stall.

2.2 Unresolved to resolved: the catalog

The parser only checks grammar. `SELECT p.category FROM orders o JOIN dim_product p ...` parses whether or not those tables exist; the result is a tree whose relation and attribute nodes are still marked *unresolved* — `orders` and `p.category` are held as names, not yet bound to anything.

The *analyzer* resolves them. It asks the **catalog** what `orders` and `dim_product` are, gets back their schemas, and replaces each unresolved node with a concrete one:

category becomes a string column belonging to dim_product, delivery_days becomes an int on orders, the JOIN condition is type-checked, and AVG(...) is bound to the aggregate function. If a name does not resolve, this is where the query fails — before any work is scheduled.

In the reference stack the catalog is a Spark *catalog plugin* pointed at an Iceberg REST endpoint (backed by the Hive Metastore); Chapter 10 covers what it stores and how commits move through it. What matters here is that the catalog is also where *table statistics* come from — row counts, column value ranges, distinct counts. Spark’s cost-based optimizer uses them to order joins and to decide what is small enough to broadcast (Chapter 5). An Iceberg table exposes a total file size and row count from its metadata, but not per-column statistics unless you have run `ANALYZE TABLE ...COMPUTE STATISTICS`. This is why so many lakehouse joins pick the wrong strategy at planning time and have to be rescued by a hint or by AQE.

2.3 Logical optimization: the rules that matter

The optimizer applies *rules* — functions from a logical plan to a better logical plan — grouped into *batches* that run repeatedly until the plan stops changing. Spark ships around a hundred rules. A handful account for most of the speed-up, and all of them do the same thing: move work earlier, or delete it.

Predicate pushdown. A filter is pushed as close to the scan as the semantics allow, so fewer rows enter the rest of the plan. In the specimen the order_ts range moves into the orders scan, where it becomes a *partition filter* (Iceberg skips whole days) and a *pushed filter* (Parquet skips row groups). Chapter 7 is about keeping this rule working; a function or a CAST on the filtered column defeats it.

Column pruning. The scan reads only the columns the query references. The specimen touches product_id, delivery_days, promised_days, and order_ts from orders and product_id, category from dim_product — so a fifty-column orders table is read four columns wide. A `SELECT *` anywhere in a subquery turns this off.

Constant folding and simplification. Expressions with no runtime input are evaluated once at planning time; the `TIMESTAMP '2026-03-01 ...'` literals are computed here, not per row. Boolean and null simplifications (`x AND true`, `x = x`, `IS NOT NULL` on a non-nullable column) are folded away.

Join reordering. With reliable statistics and cost-based optimization enabled (`spark.sql.cbo.enabled` and `spark.sql.cbo.joinReorder.enabled`), a multi-way join is reordered to keep intermediate results small — join the pair that produces the fewest rows first. Without statistics Spark keeps the written order, which is one more reason Iceberg tables benefit from `ANALYZE`.

Note

Most optimizer rules are *heuristic*: they rewrite the plan into a form that is almost always better, using no cost model. Only join selection and join reordering are cost-based, and only when statistics exist. When people say “the optimizer is smart,” they usually mean the heuristic rules, which are reliable; the cost-based part is only as good as the catalog’s numbers.

2.4 Physical planning and strategies

The optimized *logical* plan still says only *what* to compute. The planner turns each logical operator into one or more *physical* operators by applying *strategies*:

- a logical `Join` becomes `BroadcastHashJoin`, `ShuffledHashJoin`, or `SortMergeJoin`, chosen from the estimated sizes of its inputs and `spark.sql.autoBroadcastJoinThreshold` (Chapter 5).
- a logical `Aggregate` becomes a partial `HashAggregate` on each partition, an `Exchange` to bring equal keys together, and a final `HashAggregate`.
- a logical `Sort` that needs a global order becomes a range-partitioning `Exchange` followed by a local `Sort`.

The two-phase aggregate is worth tracing through with actual row counts, because “partial then final” only pays off if the partial step actually shrinks the data. Say the

specimen's orders scan is split across 200 partitions, and category electronics has rows scattered across all of them. The *partial* HashAggregate runs once per partition, independently, before any data moves: partition 17 might hold 40,000 electronics rows and collapse them to a single (electronics, count=40000, sum=...) row; partition 84 does the same for its own share. Every partition now emits at most one row per category it happened to contain — typically a few dozen rows instead of tens of thousands. The Exchange then shuffles only *those* small partial rows by category, and the *final* HashAggregate combines the (now few hundred, not tens of millions) partial results into one row per category. The shuffle moves kilobytes of pre-aggregated rows instead of gigabytes of raw ones — that reduction is the entire reason the planner inserts a partial stage at all rather than shuffling raw rows straight to one final aggregate.

The planner also inserts Exchange (shuffle) and Sort nodes wherever an operator's required input distribution is not already satisfied — and, crucially, *removes* them where it is. If two joins share a key and the data is already partitioned by it, the second join reuses the first's exchange. This is why the *number of Exchange nodes in the physical plan* is the single best predictor of a query's cost, and why Chapter 1 told you to count them in the UI.

For the specimen, with no broadcastable statistics on dim_product, the planner produces: two scans, each followed by an Exchange hashpartitioning(product_id, 200), a SortMergeJoin, a partial and final HashAggregate (which need no new exchange, because the join already partitioned by product_id), and a final small sort. Three stages, exactly as Figure 1.3 showed.

2.5 Whole-stage code generation

A physical plan could be executed by walking the operator tree and calling `next()` on each node once per row — the “Volcano” iterator model. For a chain of `Filter` → `Project` → `HashAggregate`, every row passes through three virtual method calls, and the boxing and the short-lived row objects between operators usually cost more than the actual filtering and arithmetic.

Spark does not do this. For a chain of operators within a stage it *generates Java source*

for the whole chain — one method that scans, filters, projects, and feeds the aggregate in a tight loop, with no virtual call, no intermediate row object, and field access straight out of the packed binary row (Chapter 17). That method is compiled to bytecode and then by the JVM's JIT to native code.

In a printed plan you see this as a numbered marker. Operators fused into one generated function are wrapped: `*(2) HashAggregate,*(2) Project,*(2) Filter`. The `*(n)` means “part of generated stage n”; the number is just a sequential id, not an execution order. An operator *without* a star is a boundary where fusion stopped and Spark falls back to row-at-a-time.

A stage stops being generated for a handful of reasons, and a plan missing stars where you expect them is usually hitting one:

- an Exchange or BroadcastExchange — you cannot fuse operators across a network boundary, so codegen always restarts on the far side of a shuffle;
- a stage whose input or output schema has more columns than `spark.sql.codegen.maxFields` (100 by default);
- a generated method that would exceed the JVM's 64 KB bytecode limit for a single method (`spark.sql.codegen.hugeMethodLimit`); Spark logs `Found too long generated codes` and drops that stage to the interpreter;
- a compile failure in the generated Java, after which Spark silently falls back (`spark.sql.codegen.fallback`) — the only trace is a WARN in the executor log;
- an imperative (non-declarative) aggregate or UDAF. Spark's built-in aggregates (SUM, AVG, COUNT) are *declarative*: each is expressed as a small set of Catalyst expressions (an update expression, a merge expression) that the codegen framework can inline directly into the generated method, the same as any other expression. A UDAF written against the older imperative `Aggregator/UserDefinedAggregateFunction` API instead hands Spark opaque Scala/Java method bodies to call — there is no expression tree to inline, so Spark can only invoke those methods the ordinary virtual-call way, and fusion stops at that node.

A plan missing a star where you expected one confirms which of these fired. Compare a healthy plan, `*(2) HashAggregate(keys=[category], ...)`, against the same query with an imperative UDAF substituted in for the AVG: the node becomes plain `HashAggregate(keys=[category], ...)` with no leading `*`, and every operator that would otherwise have fused with it in that stage loses its star too, since fusion is all-or-nothing for the chain that touches the un-fusable node.

`EXPLAIN CODEGEN` prints the generated Java for every whole-stage stage, which is how you confirm what actually got fused.

2.6 Reading EXPLAIN

Now that you know what the `*(n)` markers and Exchange nodes mean, here is how to read the rest of a plan systematically. `EXPLAIN` prints the plans without running the query. There are six forms, selected by a keyword in SQL or a mode argument to `df.explain(...)`.

Mode	Shows
(default)	the physical plan only — the quick check for join strategy, shuffle count, codegen
EXTENDED	all four plans: parsed, analyzed, optimized logical, and physical — use to see what a rule did
FORMATTED	the physical plan as a compact tree <i>plus</i> a numbered list expanding each node’s detail — the one to learn
COST	the optimized logical plan annotated with the planner’s row-count and size <i>estimates</i> — use to see why it did not broadcast
CODEGEN	the physical plan plus the generated Java for each whole-stage stage

`EXPLAIN FORMATTED` has two parts: a compact tree, then a numbered list where each node’s details are spelled out. Read the tree **bottom-up** — the leaves are scans, the root is the final operator.

```
== Physical Plan ==
*(3) Sort (13)
+- *(3) HashAggregate (12)                -- final aggregate
```

```

+- *(2) HashAggregate (10)          -- partial aggregate
  +- *(2) SortMergeJoin Inner (9) -- both sides shuffled; not broadcast
    :- *(1) Sort (5)                -- local sort, one per SMJ input
    : +- Exchange hashpartitioning(product_id, 200) (4)  -- shuffle #1
    :   +- *(1) Filter (3)
    :     +- *(1) ColumnarToRow (2)
    :       +- BatchScan orders (1)  -- FileScan for a Hive table
+- *(1) Sort (8)
  +- Exchange hashpartitioning(product_id, 200) (7)  -- shuffle #2
    +- BatchScan dim_product (6)

(1) BatchScan orders
  PartitionFilters: [order_ts >= 2026-03-01, order_ts < 2026-03-08]
  PushedFilters:    [IsNotNull(product_id)]          -- row groups skipped
  ReadSchema:      struct<product_id,promised_days,delivery_days,order_ts>

```

Listing 2.1: EXPLAIN FORMATTED on the specimen (trimmed)

The nodes you will meet most — a full catalogue is Appendix C:

Scan, at every leaf. BatchScan for an Iceberg or other DataSource V2 table (Spark’s newer, pluggable table-source API, covered in Chapter 19 — Iceberg tables always go through it); FileScan parquet for a Hive-catalogued table; InMemoryTableScan for cached data. Its fields are where pruning is confirmed or denied: PartitionFilters (whole partitions skipped — directories on object storage), PushedFilters (row groups skipped inside the files that are read, using column min/max and bloom filters), DataFilters (everything, including the part evaluated row-by-row after reading), and ReadSchema (the columns actually read — should be only those the query needs). Batched: true means vectorized columnar reads are on.

Exchange, at every stage boundary. Each Exchange is one shuffle: serialize, write to local disk, transfer, re-read. Count them — it is the best single predictor of cost. hashpartitioning(key, n) co-locates equal keys (joins, GROUP BY); rangepartitioning produces globally ordered partitions (a global ORDER BY); SinglePartition funnels everything to one task (a global aggregate, coalesce(1)). The ENSURE_REQUIREMENTS

tag means the planner inserted this exchange because a downstream operator needs that distribution. `BroadcastExchange` is a different move: it ships one small side whole to every executor, and it appears only below the build side of a `BroadcastHashJoin`.

Sort. `Sort [key ASC NULLS FIRST], false, 0`—the boolean is true for a global sort, false for a local per-partition one. A `SortMergeJoin` always has two local sorts, one per input, between the exchanges and the join.

Tip

The fast scan of any plan, in order: (1) the join operator — `SortMergeJoin` on a small dimension means missing statistics (Chapter 5); (2) the number of `Exchange` nodes; (3) `PartitionFilters` non-empty on a partitioned table (Chapter 7); (4) `PushedFilters` non-empty; (5) `ReadSchema` listing only needed columns.

After the query runs, the SQL tab shows the same tree with *actual* row counts and, if AQE was active, the adapted plan rooted at `AdaptiveSparkPlan isFinalPlan=true` — which is the plan that really ran, not the one `EXPLAIN` printed beforehand (Chapter 6).

2.7 Summary

A query becomes an execution in fixed stages: parse to a syntax tree, resolve names and types against the catalog, rewrite the logical plan with mostly heuristic rules (pushdown, column pruning, folding; join reordering only with statistics), choose physical operators and shuffle points with strategies, and generate Java for each pipelined chain. The physical plan — readable with `EXPLAIN FORMATTED` — tells you the join strategy, the number of shuffles, and whether pruning and column selection are working, which together determine how many stages the query has and where it will spend its time. The recurring lakehouse failure is missing catalog statistics, which makes the cost-based decisions (join strategy and order) unreliable until you `ANALYZE` or override.

Exercises

1. Run `EXPLAIN FORMATTED` on the specimen query. Confirm the two `Exchange` nodes, the `SortMergeJoin`, and a non-empty `PartitionFilters` on the orders scan.

2. Write the same result three ways: with an explicit subquery, with a CTE, and as one flat statement. Compare the *optimized* logical plans. Which differ, and which are identical?
3. Add `WHERE CAST(o.product_id AS STRING) = '47'` to the query and re-run `EXPLAIN FORMATTED`. What happened to `PushedFilters`?

Chapter 3

How an Executor Uses Memory and CPU

Spark runs on the Java Virtual Machine: the driver and every executor is a JVM process, and the engine's memory behaviour, its failure modes, and much of its internal design follow directly from how the JVM manages memory and runs code. This chapter builds the memory map of a single executor — which regions exist, which a query can use, and which are consumed no matter what — then adds the two costs layered on top of it: garbage collection and serialization. By the end you can size a pod from its configuration, explain a container kill that left no Java error, and read GC time in the UI. How execution and storage memory borrow from each other while a job runs is picked up alongside caching in Chapter 8; how to size these regions for a workload is Chapter 14.

3.1 The JVM in one page: bytecode and just-in-time compilation

Spark is written in Scala and runs on the Java Virtual Machine, and two features of that platform shape how it performs.

The first is *just-in-time compilation*. Scala compiles to JVM *bytecode*, a portable instruction set the JVM does not run directly. Instead it interprets bytecode at first and

watches which methods run often; a *hot* method is then compiled to native machine code by the JIT, with optimizations — inlining, loop unrolling, dead-code elimination, escape analysis — that depend on what the method actually did the first few thousand times it ran. Two consequences follow. A Spark process is slowest in its first seconds, before the JIT has warmed up, which is one reason Kyuubi keeps a driver alive rather than starting one per query (Section 1.3) and why a micro-benchmark that runs a snippet once tells you almost nothing. And the JIT compiles *tight, predictable* code far better than code full of virtual calls and branches — which is exactly why Spark’s execution engine generates a single flat Java method for a chain of operators (Section 2.5, Chapter 17) instead of walking an operator tree.

The second is *automatic memory management*: the program allocates objects and never frees them, and a *garbage collector* reclaims what is no longer reachable. This is a convenience with a running cost, and for a data engine moving billions of rows the cost is large enough that Spark works hard to keep most of its data *out* of the managed heap entirely. The rest of this chapter is about where an executor’s memory goes, and Section 3.4 about what the garbage collector does to it.

3.2 Heap layout and object representation

The JVM manages a single contiguous region of memory, the *heap*, from which every object a program allocates is drawn. A Spark executor is one JVM process, and the value of `spark.executor.memory` is precisely the maximum size of that heap. Everything a task does — decoding a Parquet page into rows, building a hash table for a join, accumulating a group-by result — competes for space in it, and when it fills, the task must either spill to disk or fail.

Two properties of the heap drive most of Spark’s decisions about memory. The first is *per-object overhead*. Every object carries a header — a mark word and a pointer to its class — before any field is stored, and the allocator pads every object to an eight-byte boundary.

```
java.lang.Integer
  object header    12 bytes    (mark word + class pointer)
  int value        4 bytes
```

```
-----  
total          16 bytes    to carry 4 bytes of data
```

Listing 3.1: Cost of a boxed integer on a 64-bit JVM

A row represented as an array of boxed Java objects (an `Integer`, a `Double`, a `String` with its own header and backing character array) can be several times larger in the heap than the same row encoded as raw bytes. This is the amplification you see when a compressed Parquet file expands to many times its on-disk size once decoded into Java objects, and it is why Spark’s execution engine stores rows off the object heap in a packed binary format rather than as JVM objects (Chapter 17).

The second property is that the heap is *managed*: the program never frees memory explicitly, and reclamation is the job of the garbage collector (Section 3.4). Spark does not hand the whole heap to a query; it subdivides it into regions with different owners and different rules, described next.

3.3 The executor memory model

Configuring Spark’s memory means deciding how one executor process divides the memory it is allowed to use. That process holds three separately configured pools, and confusing them is the most common source of both wasted memory and unexplained container kills.

Analogy

Picture the heap as an apartment building with a fixed number of floors (`spark.executor.memory`). One utility closet is permanently locked for building systems (*reserved*) — no tenant ever gets a key. A shared common floor (*unified memory*) is reconfigured on demand: some nights it hosts a big event that needs the whole floor (*execution*), other nights it is rented out as storage lockers (*storage*) — and management can always cancel a storage booking to make room for a scheduled event, but never the reverse. The remaining floors are private apartments (*user memory*): whatever a tenant does inside their own apartment is invisible to the building manager, and if a tenant packs it past capacity, nobody bails them out — the apartment simply

collapses (`OutOfMemoryError`). A separate storage unit down the street, rented by some tenants but not others, is the *off-heap* pool; and the building's own infrastructure — elevators, wiring, hallway lighting (*overhead*) — draws power regardless of occupancy. The whole lot only fits inside the property line the landlord (the cluster manager) enforces: exceed it, and the building is condemned, tenants and all.

The heap. `spark.executor.memory` sets the maximum size of the JVM heap, the region every object the executor allocates comes from. Spark carves it into three parts:

- **Reserved memory** — a fixed 300 MB Spark holds back for its own internal objects. It is not configurable and never available to a query.
- **Unified memory** — by default 60% of (heap – 300 MB), set by `spark.memory.fraction`. This is the only pool Spark accounts for byte by byte and the only one that spills to disk gracefully when it runs out. It is shared between *execution memory* (the working memory of a running operator: hash tables for joins and aggregations, sort buffers, the shuffle write buffer) and *storage memory* (cached `DataFrame` and `RDD` blocks, and broadcast data held on the executor). `spark.memory.storageFraction` (default 0.5) is a soft boundary between the two: either side may borrow the other's idle space, but execution can always evict borrowed cached blocks to reclaim its half, whereas storage cannot push execution below the border. How this plays out while a job runs is covered in Chapter 8.
- **User memory** — the remaining 40%. Everything Spark does not manage lives here: the objects your UDFs and `RDD` closures create, the structures a `mapPartitions` builds up. Spark does not track this pool and cannot spill it; overflowing it throws `OutOfMemoryError` and kills the executor.

Off-heap memory. When `spark.memory.offHeap.enabled` is set, `spark.memory.offHeap.size` adds a second unified-memory pool that lives outside the heap, addressed directly by the engine and invisible to the garbage collector (Section 3.4). Execution and storage draw on it in addition to their on-heap pool. It is not taken out of `spark.executor.memory`; it is additional memory the process reserves, and it must be counted into the container's limit.

Concretely: an executor already running `spark.executor.memory=8g` that adds `spark.memory.offHeap.enabled=true` and `spark.memory.offHeap.size=4g` does not shrink its 8 GB heap down to make room for those 4 GB — the 4 GB is layered on top, for a combined 12 GB of heap plus off-heap unified memory before overhead is even added. The container or pod limit has to be resized to cover all 12 GB (plus overhead) or the extra off-heap pool simply causes the kind of container kill Section 15.4 describes, with nothing in the JVM’s own heap metrics hinting at why.

Overhead. `spark.executor.memoryOverhead` (default the larger of 384 MB and 10% of the heap) is memory the process needs that is neither heap nor off-heap pool: JVM thread stacks, class metadata (Metaspace), buffers for network transfer and compression codecs, and — for PySpark — the Python worker processes, which can each take hundreds of megabytes. It is not a safety margin; it is space that is actually consumed, and its default is frequently too small for PySpark or codec-heavy workloads, in which case the container is killed with no Spark-level error.

What the cluster manager enforces. The pod or container is sized as the sum of all three pools (Figure 3.1). If the process’s real usage crosses that limit — most often because overhead was underestimated or user memory grew unbounded — the kernel kills it (Section 15.4). The heap merely filling up, by contrast, makes unified memory spill to disk and only throws when user memory or a single un-spillable allocation is the cause.

Per task, not per executor. Execution memory is shared equally across the tasks an executor runs at once, which is `spark.executor.cores`. A larger heap with proportionally more cores gives no single task more room: memory per task is the execution pool divided by the core count. A 4 GB unified pool on a 4-core executor gives each of its 4 concurrently-running tasks about 1 GB to build a hash table or sort buffer in; keep the same 4 GB pool but raise the executor to 8 cores, and each task’s share drops to about 512 MB — more parallelism, but less room per task, on the very same heap. The number of cores per executor is therefore a memory decision as much as a parallelism one, and an executor with too many cores spills on work a narrower one finishes in memory



Figure 3.1: An executor’s memory: the JVM heap (reserved / unified / user), the optional off-heap pool, and the overhead the process needs outside both. The cluster manager’s container limit must cover all three. Numbers are worked for an 8 GB heap with 2 GB overhead.

(Section 14.1).

3.4 Garbage collection: Parallel, G1, ZGC

Because the heap is managed, the JVM must periodically find objects that are no longer reachable and reclaim their space. This work is the *garbage collector* (GC), and for a data-processing workload it is never free: the collector competes with the application for CPU, and some of its phases are *stop-the-world*, meaning every application thread in the JVM is frozen until the phase completes. On an executor a long stop-the-world pause stalls every running task at once, and if it exceeds the heartbeat timeout the driver declares the executor lost and re-runs its work elsewhere (Section 15.4).

Collectors differ mainly in how they trade pause length against throughput and CPU cost.

Analogy

Picture three ways to clean a large office building overnight. Parallel GC is an all-lights-off deep clean: every employee is sent home, the whole crew cleans the whole floor at once, and the doors reopen only when they are finished — the fastest total clean, but nobody works while it happens. G1 cleans one wing at a time while the rest of the building stays open, closing only the wing being cleaned right now, briefly and often. ZGC has cleaners who tidy around the working employees continuously and almost invisibly, closing nothing at all — at the cost of moving more carefully and needing a bigger crew.

- **Parallel GC** uses multiple threads but collects the whole heap in one stop-the-world pause. It gives the highest raw throughput when pauses do not matter, but pause time grows with heap size, so a large executor can freeze for seconds at a time.
- **G1** (Garbage-First) divides the heap into equal-sized regions and collects a subset of them per cycle, doing most of its marking concurrently with the application. Pause times are shorter and roughly bounded regardless of heap size, at the cost of more bookkeeping and CPU. It is the default at the heap sizes a Spark executor typically runs and the collector most Spark tuning assumes.

- **ZGC** does essentially all of its work concurrently and keeps pauses under a millisecond even on very large heaps, at a higher constant CPU and memory overhead. It is worth choosing only for executors with tens of gigabytes of heap where G1 pauses are still a problem.

Collector	Pause	Throughput / CPU	Best for
Parallel	long, grows with heap	highest throughput, lowest overhead	batch jobs where pauses don't matter
G1 (default)	short, roughly bounded	moderate bookkeeping cost	most Spark executors
ZGC	sub-millisecond	higher constant CPU & memory	very large heaps where G1 still pauses

Table 3.1: The three collectors, at a glance.

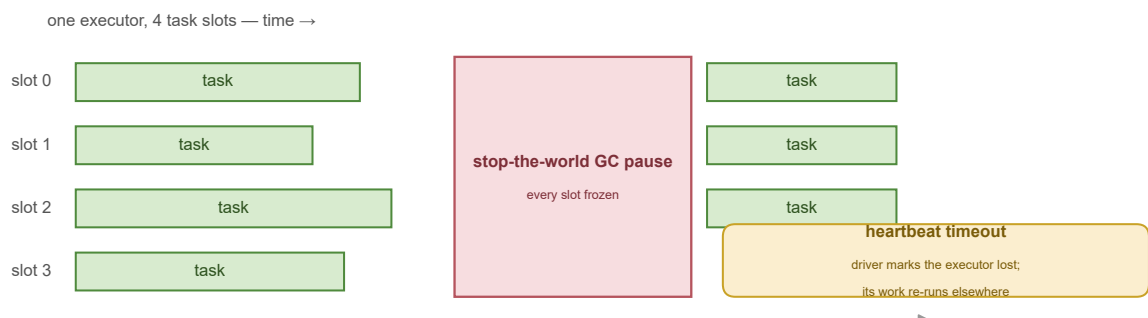


Figure 3.2: A stop-the-world pause freezes every task slot on the executor at once, not one at a time. If the pause outlasts the heartbeat interval, the driver concludes the executor is gone and re-runs its work elsewhere — so a GC problem shows up as lost executors and repeated stages, not just as slow ones.

The generational hypothesis — that most objects die young — underlies all three: they collect a small “young” area cheaply and often, and the full-heap collection rarely. A workload that holds a large amount of long-lived data in cache violates that assumption

and pushes the collector toward the expensive path, which is one more reason to cache deliberately (Chapter 8). Monitoring GC time is covered in Section 15.3; the executor JVM options that select and tune a collector are in Chapter 14.

3.5 Serialization and its cost

The engine keeps a join’s hash table or a block of cached rows as one large off-heap byte buffer rather than millions of small Java objects, which is why that data is invisible to the garbage collector and costs nothing to collect (Section 3.3, Chapter 17). Getting data into and out of that flat form is serialization.

Serialization is the conversion of an in-memory object graph into a flat sequence of bytes, and *deserialization* is its inverse. Spark serializes whenever data leaves a JVM: across the network during a shuffle, to disk when unified memory spills or a block is cached in serialized form, and when a broadcast variable is shipped to executors. The cost is both CPU and size.

The two options for arbitrary Java objects differ sharply. The built-in Java serializer writes the full class name and every field name and type alongside the values, repeated for every object, so most of the output is self-description rather than data. *Kryo* writes a compact encoding — a short numeric class identifier and the field values — and produces output several times smaller, which directly reduces shuffle volume, spill size, and broadcast footprint (Chapter 14).

DataFrame and SQL workloads sidestep both serializers for most operations: rows already live in the engine’s binary format (Chapter 17), so moving them is a memory copy with no object graph to walk. The Java-versus-Kryo choice still governs RDD code, user-defined functions, and anything that puts arbitrary Java objects into a shuffle or a broadcast — the same user-memory objects that Section 3.3 noted Spark does not manage.

3.6 Summary

An executor is one JVM process, and its memory is three separately configured pools: the heap (`spark.executor.memory`), an optional off-heap pool, and overhead. The heap

is carved into a fixed 300 MB reserve, *unified memory* (the only pool Spark accounts for and the only one that spills), and *user memory* (untracked, un-spillable, throws on overflow). The container limit the cluster manager enforces is the sum of all three; cross it and the process is killed with no Java error, whereas merely filling the heap makes unified memory spill. Execution memory is shared across an executor's cores, so cores-per-executor is a memory decision. On top of the map sit two costs: garbage collection, whose stop-the-world pauses freeze every task at once and can cost you an executor; and serialization, which Kryo makes several times cheaper than the Java default for RDD and UDF code, though DataFrame rows bypass it entirely. You can now size an executor from its workload and explain the two kinds of memory failure by which pool overflowed.

Exercises

1. Given a pod memory limit and a workload description, propose `spark.executor.memory`, `spark.executor.cores`, and `spark.executor.memoryOverhead`, and justify each number from the memory map in Figure 3.1.
2. An executor is killed by Kubernetes with `OOMKilled` and no Spark exception in the log. A second run fails with a `Java OutOfMemoryError` in a task. Which pool overflowed in each case, and which knob addresses each?
3. In the Executors tab, one executor shows 40% of task time in GC while the others show 5%. What is the most likely cause, and which chapter covers the fix?

Part II

Making Queries Fast

Chapter 4

The Shuffle

Every JOIN, GROUP BY, DISTINCT, window function, and repartition puts a wall between two stages: the shuffle. It writes every partition to disk, moves it across the network, and re-reads it. It is where most of a query's time goes, and it is the mechanism behind the wide dependency from Section 1.5. This chapter opens the wall: the write path, the read path, how to size shuffle partitions from data volume instead of the default 200, and how to find and remove exchanges the query does not need.

4.1 What a partition is: input partitions versus shuffle partitions

Spark never processes a table as one unit; it splits it into **partitions**, and one task handles exactly one partition (Section 1.4). Two different mechanisms create them, and mixing them up is a common source of confusion when reading a plan.

An **input partition** is created when a stage reads from storage. Spark decides how many by dividing the bytes to read by a target size: roughly `spark.sql.files.maxPartitionBytes` (128 MB) for Hive-catalogued files, or the split size Iceberg's planner reports from its manifests (`read.split.target-size`, also 128 MB by default, Chapter 12). One data file is not one partition either way — small files are combined and large ones split so that each input partition is close to the target size.

Note

Both configs measure the file's *on-disk* (compressed) length, never the decompressed size — neither Spark nor Iceberg opens a file just to plan splits. For a compressed columnar file this is the same amplification as Section 3.2, one step earlier: a “128 MB” partition can decode to several times that in a task's execution memory, a quiet cause of spill unrelated to shuffle-partition sizing.

Splittability is a separate question from ratio: Parquet and ORC stay splittable at any codec, since Spark cuts at independently-decodable row-group or stripe boundaries. A .gz-compressed text/CSV/JSON file cannot — gzip's stream has no resume point, so the whole file is one partition regardless of size.

A **shuffle partition** is created when a wide operator redistributes data. Its count is set directly: by `spark.sql.shuffle.partitions` (default 200) for a join or aggregation, or by an explicit `repartition(n)`. Spark decides which shuffle partition a row belongs to with a hash:

```
partition_id = hash(key) mod num_shuffle_partitions
```

The same key always hashes to the same partition, which is exactly what a join or a `GROUP BY` needs — it is how rows that must be compared end up on the same executor. Note what this does *not* guarantee: a shuffle partition holds every row for potentially many different keys, and nothing about the hash keeps that mixture even. Two keys can collide into the same partition, and one key alone can dominate a partition if enough rows share it. That is the seed of data skew (Chapter 6); for now, the point is that shuffle-partition count is a number *you* choose, not one Spark infers from the data the way it infers input-partition count.

Analogy

A shuffle is a citywide moving day: many offices, each organized by building, have to be reorganized by department into new, single-department buildings. Every current building (a map task) has its people first check which new building they've been assigned to (compute `partition_id`), sort themselves into that order, and box up their belongings by destination — each building keeping one manifest of what it packed and where each box is going (the one data file plus its index). The mov-

ing company's dispatcher (the map output tracker) is the only party who ever holds the complete picture of which old building currently has which department's boxes. When a new department building is ready (a reduce task), it does not go door to door itself — it asks the dispatcher, then sends trucks to every old building that packed something for it (the all-to-all fetch), and once everything arrives it unpacks and arranges the furniture in the new layout (the merge). If a truck can only carry so much in one trip, the extra boxes wait on the sidewalk until the next one — a spill to disk.

4.2 The shuffle write path

Figure 4.1 follows one shuffle from a map task in stage 0 of the specimen to a reduce task in stage 2. On the write side, every map task:

1. computes `partition_id = hash(product_id) mod 200` for each of its output rows;
2. sorts its rows by that partition id (Spark has used only this *sort-based* shuffle writer since 1.6 — there is no separate hash-shuffle path to choose between any more);
3. spills the sort to local disk if the shuffle write buffer (bounded by Section 3.3's execution memory) fills before the task's data does;
4. writes the result as *one* data file plus *one* index file for the whole task, with the index recording where each partition's byte range begins.

One file per map task, regardless of how many shuffle partitions there are, is why raising `spark.sql.shuffle.partitions` does not multiply the number of files on disk — it only changes how the one file's byte ranges are cut up. The driver's *map output tracker* records, for every map task, which executor holds that file and the offsets for each partition inside it.

4.3 The shuffle read path

A reduce task starts by asking the map output tracker for the location of its partition's data across *every* map task in the previous stage — for stage 2 of the specimen, that is

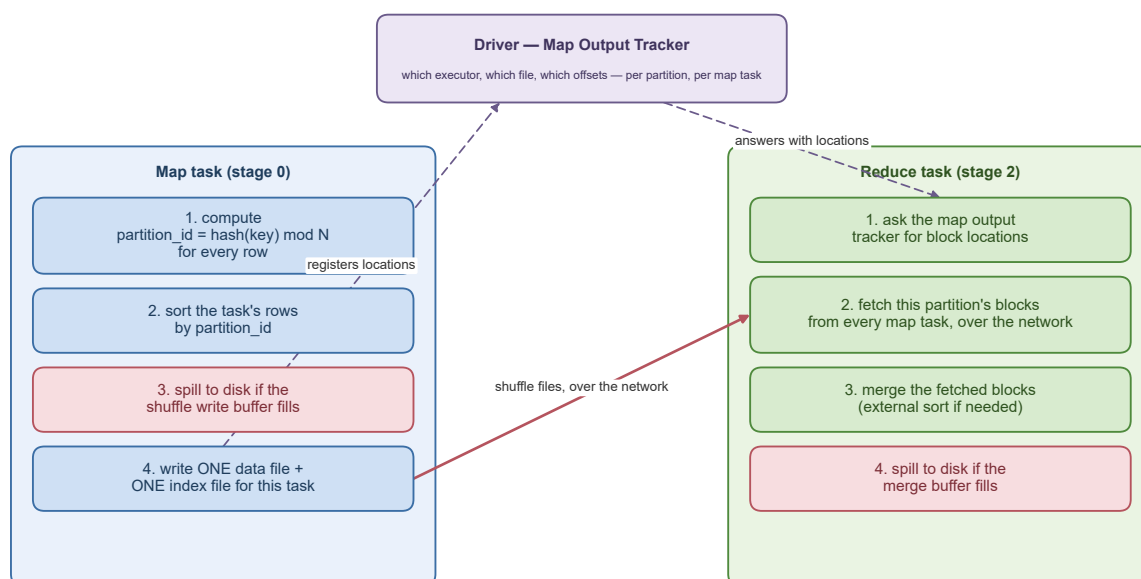


Figure 4.1: The shuffle write and read paths. The map task never talks to the reduce task directly: it registers its file’s location with the driver’s map output tracker, and the reduce task asks the tracker where to fetch from.

every one of stage 0’s and stage 1’s tasks. It then fetches those blocks over the network via the block transfer service, several fetches in flight at once up to `spark.reducer.maxSizeInFlight`, and merges them — an external (disk-backed) merge if the combined size does not fit in the task’s share of execution memory.

This is the step that makes a shuffle expensive relative to a narrow operation: it is genuinely all-to-all. With M map tasks and R reduce tasks, up to $M \times R$ blocks are produced and each of the R reduce tasks may fetch from each of the M executors that produced them. For the specimen’s stage 2, that is the roughly 180 map tasks from stage 0’s `orders scan` (Figure 1.3) against 200 reduce tasks: up to $180 \times 200 = 36,000$ distinct map-output blocks, and in the worst case every one of the 200 reduce tasks opens a connection to every one of the executors that ran a stage-0 task — not 200 fetches, but up to 36,000 individual block transfers, which is why this step, not the CPU work in either stage, is usually the wall-clock bottleneck. Section 4.6 covers the mechanism Spark added specifically to make this fetch pattern cheaper at scale; the scheduler machinery behind both sides is Chapter 18.

4.4 Sizing shuffle partitions

Because the write path always produces the same one file per task, the only real lever on shuffle cost is *how big each shuffle partition ends up*, which is set by `spark.sql.shuffle.partitions` against the total volume being shuffled:

```
target partitions = total shuffle bytes / target task size
(128-200 MB)
```

The default of 200 is not tuned to your data; it is just a constant. For a 100 GB shuffle it is badly wrong in one direction, as Figure 4.2 makes exact: 200 partitions puts 512 MB in every task, several times past the point where the task’s share of execution memory holds it and it spills to disk. A query that shuffles only a few gigabytes has the opposite problem — 200 partitions of a few megabytes each, so the fixed per-task scheduling overhead dominates the actual work.

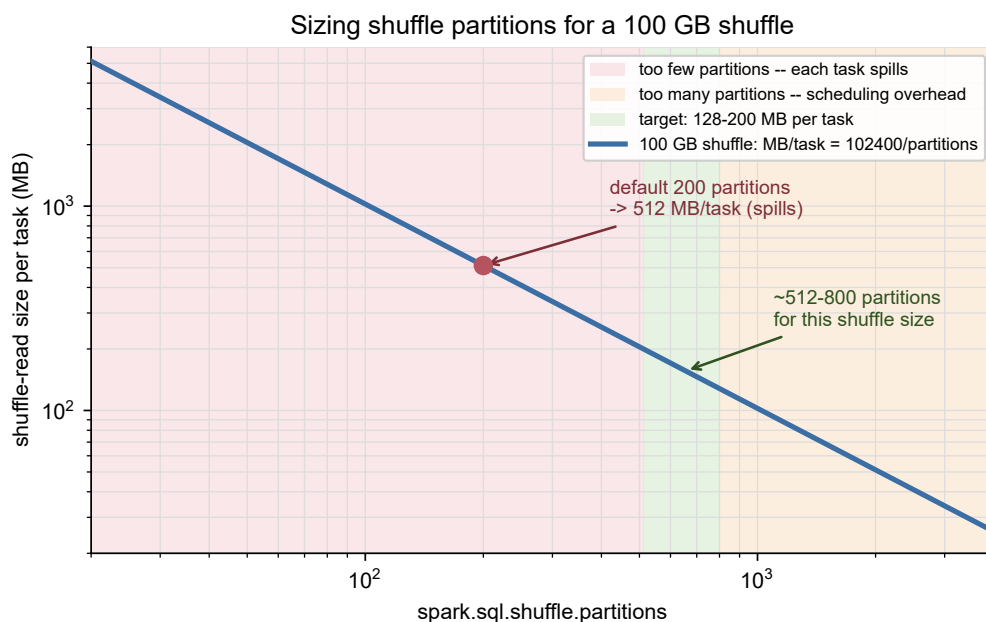


Figure 4.2: For a fixed shuffle size, size per task is exactly (shuffle bytes) / (partition count) — an exact relationship, not a rule of thumb. For a 100 GB shuffle the default 200 partitions puts 512 MB in every task, well past where it spills; 512–800 partitions lands each task in the 128–200 MB band.

Warning

If you set `spark.sql.shuffle.partitions` too low, Spark does *not* raise it for you. Partitions simply grow past the point where they fit in memory and spill, silently, until the job either finishes slowly or runs out of disk. There is no auto-correction on the low side.

The practical answer is to set the count generously as a ceiling and let Adaptive Query Execution shrink it back down once it has seen the real data size, rather than trying to predict the exact right number up front:

```
spark.sql.shuffle.partitions           = 2000
spark.sql.adaptive.enabled             = true
spark.sql.adaptive.coalescePartitions.enabled = true
spark.sql.adaptive.advisoryPartitionSizeInBytes = 134217728
-- 128 MB
```

Over-partitioning is cheap to fix at runtime — AQE merges small adjacent partitions after the shuffle completes and it knows their true size (Chapter 6); under-partitioning, which spills, is not something a later stage can undo. Set high and let AQE come down, not the other way around.

Partition count versus the cluster’s task slots. The formula above sizes a partition in bytes; it says nothing about how many can run at once, which is set separately by the cluster’s total task slots:

```
total task slots = executor count x spark.executor.cores
```

Partition count against *that* number, not against the shuffle size, is what Section 1.4 meant by “8 tasks on a cluster with 200 free slots leaves 192 idle” — too few partitions wastes capacity you are already paying for. The reverse failure is subtler: if partition count is only a little more than a multiple of the slot count — 41 partitions on 40 slots, say — the first wave uses every slot, and the second wave runs on a single core while the other 39 sit idle, costing nearly as much wall-clock time as the first wave for a fraction of the work. Prefer a partition count that divides the slot count cleanly over one that merely clears it.

Executor count is not a fixed input here either. Under dynamic allocation (Section 14.2), Spark requests more executors when it sees more pending tasks than running ones, so a higher partition count can grow the cluster rather than the other way around — the causal arrow runs from partitions to executors, not from a fixed executor count down to how many partitions to choose. What executor *shape* does constrain is *partition size*: execution memory per task is the execution pool divided by `spark.executor.cores` (Section 3.3), so more cores packed into the same executor leaves less room per task and calls for smaller partitions to avoid spill.

4.5 Shuffle compression

Shuffle data is compressed by default (`spark.shuffle.compress=true`) using `spark.io.compression.codec`, which defaults to `lz4`. That default is deliberate: shuffle bytes are written once and read back almost immediately by the very next stage, so *decompression speed* dominates the cost, not compression ratio. `lz4` and `snappy` both compress and decompress in a small fraction of the time `zstd` or `gzip` take, at a somewhat worse ratio. That trade-off is the opposite of Parquet storage on Iceberg, where a file is written once but may be read by hundreds of later queries, so a slower, denser codec such as `zstd` pays for itself (Chapter 11). Using the same codec for both because “it worked for storage” is a common way to leave shuffle performance on the table.

Codec	Ratio	Compress	Decompress	Best for
lz4	low–medium	very fast	very fast	shuffle (the default)
snappy	low–medium	very fast	very fast	shuffle, an alternative to lz4
zstd	high	medium	fast	Parquet storage on Iceberg
gzip	highest	slow	slow	archival only — avoid for shuffle

Table 4.1: Written once, read immediately (shuffle) wants decompression speed; written once, read many times (storage) can afford to spend more on ratio.

4.6 Push-based shuffle

The read path in Section 4.3 has a structural cost at scale: many small, random fetches, one per (map task, reduce partition) pair, which stresses the block transfer service and, if a map executor is lost, forces its tasks to be recomputed before the shuffle can finish. *Push-based shuffle* (stable since Spark 3.2) restructures this: after writing its file, each mapper also *pushes* its blocks to shuffle services, which merge blocks from many mappers that belong to the same reduce partition into a small number of larger merged files. A reduce task then does a handful of large sequential fetches instead of many small ones.

Note

Push-based shuffle needs an external, out-of-executor shuffle service capable of receiving and merging pushed blocks, which is a natural fit for YARN’s long-lived Node Managers. Its support on Kubernetes is narrower and depends on running a compatible external shuffle service alongside your executors — confirm it is available on your cluster (Appendix B) before turning on `spark.shuffle.push.enabled`; without it, the setting has nothing to push to.

4.7 Finding and removing unnecessary exchanges

Every Exchange in a physical plan is a shuffle you are paying for (Section C.4), and not every one is necessary. Four patterns account for most avoidable ones.

A manual `repartition()` right before an Iceberg write. If the target table has `write.distribution-mode=hash` (Chapter 13), Iceberg already shuffles the write by partition key. Calling `repartition()` yourself first adds a *second*, usually redundant, shuffle. Let the table’s distribution mode do it, or set the mode to `none` only when you are certain the data is already correctly distributed.

A `DISTINCT` on data that is already unique. `DISTINCT` compiles to an aggregation, which shuffles. If the upstream source already guarantees uniqueness — a primary key, or a prior `GROUP BY` on the same columns — the `DISTINCT` is pure cost.

An ORDER BY inside a subquery. Only the outermost ORDER BY in a query is guaranteed to survive to the result; SQL does not promise row order through an intermediate view or CTE. An inner sort still costs a range-partitioning exchange even though nothing downstream can rely on its effect.

Repeated aggregation on the same key. Two separate GROUP BY product_id computations in the same query — one for the on-time rate, one for total orders — shuffle twice on the same key when a single GROUP BY with two aggregate functions would shuffle once. Watch for ReuseExchange in the plan (Section C.4): the planner catches this only when the two exchanges are byte-for-byte the same partitioning, so a small difference (a different partition count, an added column) silently defeats it.

Tip

Run EXPLAIN FORMATTED and count the Exchange nodes before touching anything else. Every one you remove is a shuffle the job no longer pays for; every one you cannot explain is worth a second look.

4.8 Summary

A shuffle partition is not the same thing as an input partition: input partitions come from splitting storage by size, shuffle partitions come from a count you set and a hash that routes rows into it. The write path always produces one file per map task, indexed by partition; the read path is genuinely all-to-all, which is what makes a shuffle costly. Size `spark.sql.shuffle.partitions` from the actual volume being shuffled — 128–200 MB per task is the target — set it generously and let AQE coalesce down rather than risk under-partitioning, which spills and does not self-correct. lz4 trades ratio for decompression speed, which is the right trade for data read once, immediately. Push-based shuffle turns many small fetches into few large ones where an external shuffle service supports it. And the cheapest shuffle is the one you remove: count the Exchange nodes and look for a redundant repartition, DISTINCT, inner sort, or duplicated aggregation before reaching for any other fix.

Exercises

1. From a stage's shuffle-read size in the UI, compute a target partition count for a 128 MB task size, then verify against the observed task sizes.
2. Run the specimen query with `spark.sql.shuffle.partitions` set to 4000 instead of 200. Confirm in the UI that stage 2 now has 4000 tasks, and that Figure 4.2's formula predicts their size correctly.
3. Find a query in your own workload with two Exchange nodes that partition on the same column. Check whether `ReuseExchange` fired; if not, find the difference between the two exchanges that prevented it.

Chapter 5

Joins

Spark has four ways to run a join and picks one from a size estimate. When the estimate is right, the choice is right. When the table has no statistics — the common case for Iceberg, as the specimen query already showed (Section 2.2) — the planner guesses, and a join that should have broadcast a 30 MB dimension instead shuffles a billion rows. This chapter covers the four strategies, how the planner chooses between them, and how to override it when you know better than the statistics do.

Figure 5.1 shows the shape each strategy leaves in a plan. The rest of this chapter explains why each shape exists and what it costs; the identifiers to spot them by are catalogued in Section C.6.

5.1 Broadcast hash join

If one side of a join is small enough, Spark skips shuffling the large side entirely: it collects the small side to the driver, packages it once, and ships a full copy to every executor (a `BroadcastExchange`), where it is built into an in-memory hash table keyed on the join column. Each task on the large side then probes that local hash table row by row — no shuffle, no sort, and the large side is read exactly once, in place.

This is the cheapest strategy whenever it applies, which is exactly why the planner tries it first. Two costs are easy to underestimate. First, the *driver* holds the whole broadcast table in memory while assembling it, at roughly twice the table’s serialized size; a

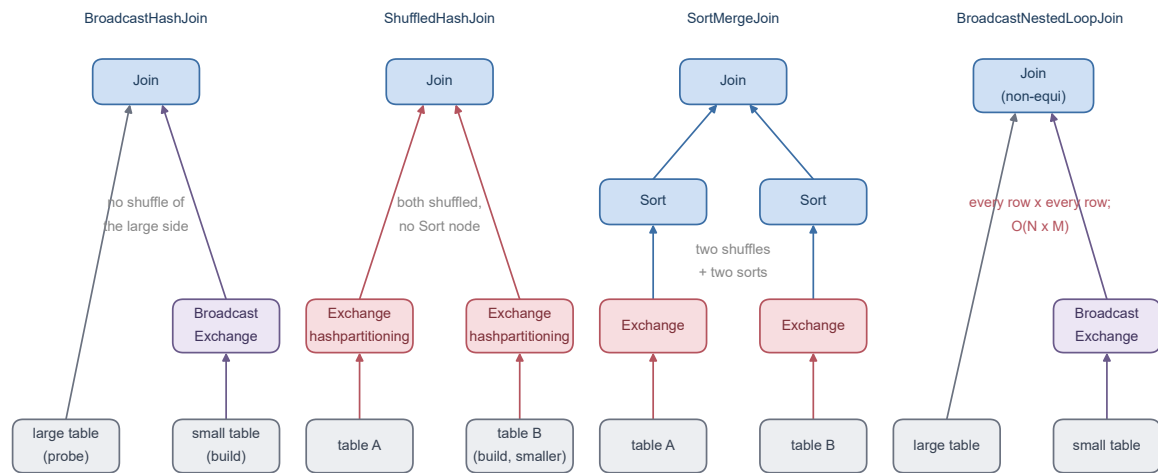


Figure 5.1: The four join strategies as plan shapes. Broadcast hash join shuffles nothing on the large side; shuffled hash join shuffles both sides but sorts neither; sort-merge join shuffles and sorts both; broadcast nested loop join broadcasts one side to evaluate a condition that is not an equality.

broadcast large enough to hurt the driver is a self-inflicted wound distinct from anything happening on the executors. Second, the size used to decide broadcastability is the table’s *on-disk* size, and a compressed columnar file routinely decodes to several times that once it is rows in memory (Section 3.2) — so a “100 MB” table can be genuinely 500 MB–1 GB once broadcast and decoded, which is why the planning-time threshold is conservative by design. Spark also enforces a hard ceiling (roughly 8 GB serialized, and a few hundred million rows) past which a table cannot be broadcast at all, regardless of the threshold.

Put numbers on it: `dim_product` (Section A.3) is a 6 MB Parquet file, comfortably under the default `10 MB spark.sql.autoBroadcastJoinThreshold` on its real, on-disk size. Here is what “twice the serialized size” means once that file is actually broadcast: Parquet’s columnar compression packs those 6 MB down from perhaps 20 MB of raw column values; decoded back into JVM row objects on the driver (Section 3.2’s per-object overhead applies here too), that 6 MB file can easily inflate to 25–35 MB in memory, and the driver holds roughly two copies of that while it collects the file’s rows and then builds the broadcast payload to ship out — 50–70 MB of driver heap spent broadcasting a file whose on-disk size looked negligible. A dimension table ten

times larger inflates the same way, which is exactly why the comfortable-looking 10 MB threshold on disk still leaves real headroom before it threatens a driver's heap.

5.2 Shuffle hash join

When neither side is small enough to broadcast but one side is still meaningfully smaller than the other, Spark can shuffle both sides on the join key and build an in-memory hash table from the smaller (build) side on each executor, then probe it with the larger (probe) side — without sorting either. This is a `ShuffledHashJoin`: two shuffles, like sort-merge, but no `Sort` nodes, which makes it cheaper than sort-merge whenever the build side reliably fits in memory per partition.

That condition is also its weakness: unlike sort-merge, a shuffled hash join has no graceful way to spill an oversized build-side partition to disk. If a single partition's build side is larger than expected — most often because of skew — the task fails outright rather than slowing down. Spark's planner is correspondingly conservative about choosing it: by default it prefers sort-merge whenever both sides have sortable keys (`spark.sql.join.preferSortMergeJoin=true`), and only considers a shuffled hash join when that preference is turned off and the build side is estimated at at least three times smaller than the probe side. Concretely: with the preference off, an estimated 500 MB `dim_customer` against a 2 GB `orders` slice qualifies (2 GB is $4\times$ larger), and the plan shows `ShuffledHashJoin` with no `Sort` node on either input; the same 500 MB against only 1 GB of `orders` does not qualify ($2\times$, short of the $3\times$ bar), and the planner falls back to sort-merge instead, adding a `Sort` node to each side of the plan that was absent in the qualifying case.

5.3 Sort-merge join

Sort-merge join is the default for two large, roughly comparable inputs, and it is what the specimen query uses in Chapter 2: both sides are shuffled on the join key, each partition is then sorted locally, and the join is produced by walking both sorted streams together, advancing whichever key is behind. It is the most expensive of the equi-join strategies — two shuffles and two sorts — but it is also the most robust: a sort spills to disk cleanly

under memory pressure the way a hash table cannot, so it scales to inputs of any size on either side.

5.4 Broadcast nested loop join

An equi-join needs a hash or a sort; a join whose condition is not an equality — `ON o.order_total > c.credit_limit`, for instance — needs neither, and Spark falls back to comparing every row of one side against every row of the other. If one side is small enough to broadcast, this is a `BroadcastNestedLoopJoin`: no shuffle, but $O(N \times M)$ comparisons inside each task. If neither side is small enough to broadcast, Spark falls all the way back to a `CartesianProduct` (Section C.6), which is rarely what you want on two large tables and is usually a sign that a join condition is missing.

Note

A full outer join cannot use broadcast hash join, even with an explicit hint: there is no way to detect an unmatched row on the broadcast side without scanning it, which the strategy is built to avoid. A `FULL OUTER` join falls back to sort-merge regardless of size.

5.5 How the planner chooses

For an equi-join, the planner tries strategies in a fixed order, stopping at the first one that applies:

1. an explicit join hint, if the query has one (Section 5.6);
2. **broadcast hash join**, if either side's estimated size is at or below `spark.sql.autobroadcastjointhreshold` (10 MB by default);
3. **sort-merge join**, if `spark.sql.join.preferSortMergeJoin` is true (the default) and the keys are sortable;
4. **shuffled hash join**, if that preference is off and the build side is estimated at least $3\times$ smaller than the probe side and fits per-partition memory;
5. **sort-merge join**, as the fallback for any remaining equi-join with sortable keys.

Every one of these decisions, except the explicit hint, is made from *estimated* sizes — from the catalog if statistics exist, or from on-disk file size if they do not (Section 2.2). This is the single most common way real joins go wrong: not a bug, but a plan built on a wrong number.

Trace one query through the ordered list to see how the choice actually falls out. For `orders JOIN dim_customer`, with `dim_customer` estimated at 500 MB and no hint present: step 1 does not apply (no hint); step 2 does not apply (500 MB is far past the 10 MB threshold); step 3 *does* apply — `preferSortMergeJoin` is true by default and both join keys are plain strings, sortable — so the planner stops right there and emits a sort-merge join. Step 4 (the shuffled-hash-join check) is never even evaluated, because the search stops at the first rule that matches; a shuffled hash join only becomes reachable at all if someone first turns `preferSortMergeJoin` off, which is why Section 5.2 described it as the strategy the planner reaches for last, not by default.

5.6 Forcing a strategy

When the estimate is missing or wrong, a hint overrides it directly:

```
1 SELECT /*+ BROADCAST(p) */ o.product_id, p.category
2 FROM orders o JOIN dim_product p ON o.product_id = p.product_id
   ;
```

Listing 5.1: Forcing a broadcast when statistics are missing

The hints, in priority order, are `BROADCAST` (or `BROADCASTJOIN/MAPJOIN`), `SHUFFLE_MERGE` (or `MERGE`) for sort-merge, and `SHUFFLE_HASH` for a shuffled hash join. A hint is an instruction, not a suggestion: Spark will honor it even when its own estimate disagrees, so a stale hint on a table that has since grown past a sane broadcast size is a real production hazard, not a theoretical one.

The more durable fix is to give the planner real numbers instead of overriding its guess:

```
1 ANALYZE TABLE db.dim_product COMPUTE STATISTICS FOR ALL COLUMNS
   ;
```

Listing 5.2: Giving the CBO real statistics

Iceberg tables carry file-level size and row counts in their metadata for free, but not the column-level statistics the cost-based optimizer uses to size a join — which is exactly why the specimen query’s small `dim_product` table was shuffled instead of broadcast in Chapter 2. Running `ANALYZE` once after a table’s data changes materially is usually cheaper, and safer over time, than hard-coding a hint.

Tip

Adaptive Query Execution catches many of these mistakes without any of the above: it re-evaluates a join’s strategy *after* its inputs have shuffled, against their real post-shuffle size, and promotes a sort-merge join to a broadcast if the data turns out small enough (Chapter 6). Hints and `ANALYZE` matter most for the joins AQE cannot rescue — most often because the small side is broadcastable only before a shuffle changes its size, not after.

5.7 Storage-partitioned joins for bucketed Iceberg tables

Every strategy above pays the same structural cost sort-merge join pays most visibly: two shuffles, one per side, every single time the query runs, even though the join key never changes between runs. The next strategy exists to avoid paying that cost at query time at all, by doing the equivalent work once, up front, when the tables are written. Every strategy above assumes the two sides arrive unaligned and have to be shuffled into agreement. If both tables are already *bucketed* on the join key with the same bucket count (Chapter 11), that assumption is false: rows that must be compared are already co-located file by file, and Spark can join corresponding buckets directly with no exchange on either side — a *storage-partitioned join*. This turns the most expensive part of a sort-merge join, the two shuffles, into work done once at write time instead of on every query that joins the tables.

The condition is strict: both tables need the identical bucket transform and bucket count on the join column, and Spark needs `spark.sql.sources.v2.bucketing.enabled` set to consider it at all. It is a design decision made when the tables are created, not a query-time tuning knob — worth it specifically for a join pattern repeated often enough that paying the shuffle cost once, at write time, beats paying it on every read.

5.8 Summary

Four strategies, in the order the planner tries them: broadcast hash join (cheapest, but bounded by the small side's real decoded size, not its file size), sort-merge join (the robust default for two large sides), shuffled hash join (cheaper than sort-merge when it applies, but unable to spill), and broadcast nested loop or Cartesian for a non-equi condition. Every automatic choice rests on a size estimate, and Iceberg tables commonly lack the column statistics that estimate needs — `ANALYZE TABLE` and join hints exist specifically to correct that, and AQE corrects a real subset of it for free at runtime. Bucketing both sides of a frequent join on the same key removes the shuffle from the decision entirely.

Exercises

1. Given four table sizes and the broadcast threshold, predict each join's strategy. Then change one config value and predict which join flips.
2. Run the specimen query, then run `ANALYZE TABLE dim_product COMPUTE STATISTICS FOR ALL COLUMNS` and run it again. Did the join strategy change? Explain from Section 5.5.
3. Write a join with a non-equi condition on two tables, neither of which is small. Confirm in `EXPLAIN FORMATTED` that it becomes a `CartesianProduct`, and rewrite it to avoid one.

Chapter 6

Adaptive Query Execution

The Catalyst optimizer plans before it has seen any data. Adaptive Query Execution re-plans after each shuffle, when the real partition sizes are known. It coalesces hundreds of tiny partitions into a few good ones, promotes a sort-merge join to a broadcast when the shuffled side turns out small, and splits a skewed partition so one task stops holding up the stage. This chapter covers what AQE re-decides, from what evidence, and which planning-time decisions it can still override.

6.1 Why runtime beats planning time

Every decision in Chapter 2 and Chapter 5 — how many shuffle partitions, which join strategy, whether to broadcast — is made from an *estimate*: a file size, a row count, a number the catalog may or may not actually have. Adaptive Query Execution is Spark inserting a checkpoint after every shuffle to replace a subset of those estimates with measurements. Once a shuffle has run, Spark knows the exact byte size of every resulting partition — no estimation error, no missing statistics, no compression-ratio surprise (Section 3.2) — and it uses that certainty to revise the plan for the stages that have not run yet. AQE has been on by default since Spark 3.2 (`spark.sql.adaptive.enabled`), and there is little reason to turn it off.

The trade AQE cannot make is the one that would help most: it cannot change a stage that has *already* run, and it cannot avoid a shuffle that planning-time decided was

necessary. It optimizes what happens after the measurement, not before it.

6.2 Coalescing shuffle partitions

Section 4.4 recommended setting `spark.sql.shuffle.partitions` generously and letting AQE bring it back down; this is the mechanism that does it. After a shuffle, AQE looks at the actual size of every partition and merges adjacent ones until each is close to `spark.sql.adaptive.advisoryPartitionSizeInBytes` (128 MB by default), stopping at `spark.sql.adaptive.coalescePartitions.minPartitionSize` (1 MB) so it does not over-merge. Put numbers on it: 2000 shuffle partitions averaging 3 MB each is 6 GB of real, measured data; merging adjacent partitions toward the 128 MB target collapses those 2000 tiny partitions down to roughly $6000\text{MB}/128\text{MB} \approx 47$ output partitions — the same 6 GB of data, the same query, just 47 tasks doing meaningful work in the next stage instead of 2000 tasks each moving a few megabytes, with no query change and no re-run.

Tip

Merging is one-directional. AQE only ever combines *adjacent* small partitions into fewer, larger ones — it never splits an oversized one this way. An oversized partition is handled by a different mechanism entirely (Section 6.5), and coalescing does nothing for it.

6.3 Converting sort-merge to broadcast at runtime

Section 5.5 chooses broadcast hash join from an estimate; AQE gets a second chance after the shuffle. If a sort-merge join’s shuffled input turns out, once measured, to be small enough, AQE replaces the sort-merge join with a broadcast hash join before the join actually runs — shown in the final plan as a `BroadcastQueryStage` in place of the exchange-and-sort pair (Section C.10). This rescues exactly the failure mode Chapter 5 described: an Iceberg table with no column statistics, planned as sort-merge because the planner had no better estimate, turns out small after its own shuffle, and AQE promotes it without you doing anything.

6.4 The two broadcast thresholds

`spark.sql.autoBroadcastJoinThreshold` and `spark.sql.adaptive.autoBroadcastJoinThreshold` are two separate configuration keys, and confusing them is easy because they default to the same value. They are not measuring the same thing. The planning-time threshold is compared against a size estimate — on-disk bytes, before decompression (Section 3.2) — so it has to stay conservative to avoid broadcasting something that turns out several times larger once decoded. The adaptive threshold is compared against the *actual* size of already-shuffled, already-decoded Tungsten rows (Chapter 17): there is no amplification left to guard against, because the number is real. That is why it is reasonable to tune the two independently rather than assume one governs the other.

Side by side: a `dim_customer` shuffle output that measures 60 MB of actual, already-decoded Tungsten rows clears the adaptive threshold (default 10 MB is too conservative here — a shop raising it to, say, 100 MB for adaptive promotions only would broadcast this) with no risk, because 60 MB *is* what will be shipped, not an estimate that might inflate. Compare that to a `dim_product` file quoted at 8 MB *on disk* at planning time, comfortably under the default 10 MB planning-time threshold — but if that Parquet file decodes to 40 MB once read into row objects, the planner never sees the 40 MB number at all, only the 8 MB one; the conservative planning-time threshold exists precisely because that gap between on-disk and decoded size is invisible before the file is actually read, while the adaptive threshold never has to guess, because the shuffle has already paid the cost of decoding by the time AQE checks it.

6.5 Skew-join handling: split and replicate

A shuffle partition can hold many keys unevenly (Section 4.1), and no amount of repartitioning fixes a single key that is simply larger than the others. AQE detects this after the shuffle: a partition is skewed if it is both larger than `spark.sql.adaptive.skewJoin.skewedPartitionFactor` ($5\times$ the median partition size) *and* larger than `spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes` (256 MB).

Requiring both avoids flagging a partition as skewed just because the median happened to be tiny. Table 6.1 makes the two-condition check concrete for a stage with a 40 MB median partition size (the specimen’s join stage on an ordinary week, before the viral product’s discount hits):

Partition	Size	vs. $5 \times$ median (200 MB)	Flagged skewed?
ordinary product key	55 MB	below	no
mildly popular key	220 MB	above, but below 256 MB floor	no
viral product key	3.2 GB	above, and above 256 MB floor	yes

Table 6.1: Both the relative check ($5 \times$ median) and the absolute floor (256 MB) must pass before AQE calls a partition skewed. The 220 MB partition shows why the floor matters: it clears the relative bar but is still an ordinary amount of data to hold in one task, not a genuine straggler.

Figure 6.1 shows the fix. AQE splits the oversized partition into several sub-partitions and *replicates* the corresponding partition from the other side of the join across all of them — cheap, because the other side is small by comparison. Each sub-partition then runs as its own task, in parallel, instead of one task carrying the whole skewed key alone while every other task in the stage sits idle.

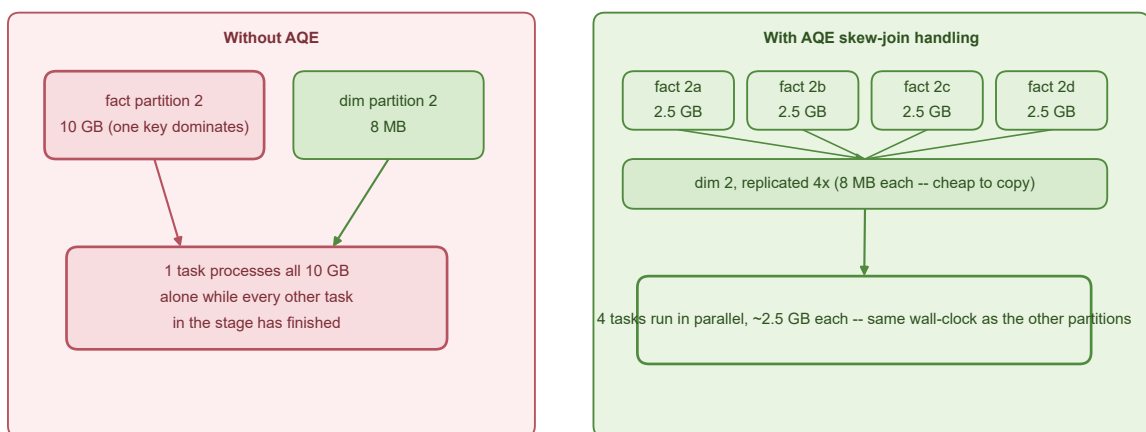


Figure 6.1: Without AQE, one oversized partition runs as a single task long after the rest of the stage has finished. AQE splits it into sub-partitions and replicates the small side across them, turning one straggler into several parallel, evenly-sized tasks.

For skew severe enough that the defaults do not catch it, lowering both thresholds — `skewedPartitionFactor` to 3.0 and `skewedPartitionThresholdInBytes` to 128 MB — catches it earlier, at the cost of treating more ordinary variance as skew.

6.6 `parallelismFirst` and partition count

Coalescing toward a target size can go too far on a small cluster. If a shuffle produces 20 small partitions on a cluster with 16 cores, merging purely by size might collapse them to 2 partitions — leaving 14 cores idle for that stage even though there was enough data to keep all 16 busy. `spark.sql.adaptive.coalescePartitions.parallelismFirst` (default `true`) protects against this: AQE first checks how many partitions are needed to use every available core (Section 4.4’s task-slot arithmetic), and only merges down to the size target if that does not fall below it. Parallelism is kept even when it means some partitions stay smaller than the 128 MB advisory size.

6.7 Reading an AQE plan

The plan EXPLAIN prints before an action runs is not necessarily the plan that ran. The one that matters is the SQL tab’s plan after execution, rooted at `AdaptiveSparkPlan` `isFinalPlan=true`; the node types that appear only in that final plan — `AQEShuffleRead` `coalesced`, `BroadcastQueryStage`, `isSkew=true` — are catalogued with their exact plan snippets in Section C.10. Reading them is how you confirm which of this chapter’s optimizations actually fired, rather than assuming from the config alone.

6.8 Summary

Planning happens before any data moves; AQE runs after every shuffle, when partition sizes are exact rather than estimated, and revises three things: it coalesces small partitions toward a target size (never splitting large ones by this mechanism), it promotes a sort-merge join to broadcast when the shuffled side is small enough by its own, more permissive threshold, and it detects a partition that is both far above the median and above an absolute size floor, splitting it and replicating the other side rather than leaving one

task to carry it alone. None of this helps a stage that has already run or a shuffle that planning already committed to — which is why the earlier chapters’ advice (size shuffle partitions generously, give the planner real statistics, bucket a repeated join) still matters; AQE is the safety net under those decisions, not a replacement for them.

Exercises

1. Disable AQE and run a skewed join; record the straggler task. Re-enable it and compare the plan and the task-time distribution.
2. Force a sort-merge join on a table small enough to broadcast after its shuffle (Section 5.6’s `SHUFFLE_MERGE` hint still lets AQE act). Confirm in the final plan that AQE converted it to a broadcast anyway.
3. Given a stage with 20 shuffle partitions on a 16-core cluster, predict how far `coalescePartitions` will merge them with `parallelismFirst` on versus off.

Chapter 7

Partitioning, Bucketing, and Pruning

*The cheapest data to move is data you never read. Partition pruning skips whole directories; dynamic partition pruning does it using values discovered from the other side of a join; bucketing lays data out so a join needs no shuffle at all. Each of these is easy to defeat — a function on the partition column, a CAST, a stray SELECT * — and the plan tells you when you have. This chapter covers each mechanism and how to confirm it is working.*

7.1 Partition pruning and the pushdown killers

A table physically partitioned by a column — for a Hive-catalogued table, one subdirectory per distinct value; for Iceberg, the hidden partitioning covered in Chapter 11 — lets the planner skip whole partitions before any file inside them is opened, whenever a filter can be resolved against the partition value alone. This shows up in EXPLAIN as a non-empty `PartitionFilters` on the scan (Section C.3); an empty one on a partitioned table means the filter did not resolve that way.

The most common reason it does not resolve is the filter itself. A function applied to the partition column, a CAST, a non-deterministic expression, or an OR with a non-

pushable branch all defeat this the same way they defeat row-group pushdown — the full catalogue of these pushdown killers, with the exact fix for each, is Section C.11. The specimen query’s own `order_ts >= TIMESTAMP '...'` filter works precisely because it is a plain comparison against a literal, resolved once at plan time, with nothing standing between the column and the value.

7.2 Dynamic partition pruning

Static pruning needs a literal known before the query runs. A star-schema join — fact table filtered indirectly, through a small dimension — has no literal for the planner to use:

```
1 SELECT * FROM orders o
2 JOIN dim_date d ON o.order_date = d.date
3 WHERE d.year = 2026;
```

Listing 7.1: A filter on the dimension, not the fact table

Without help, Spark scans every partition of `orders` across all years, then joins, then discards everything outside 2026 — all the pruning happens *after* the expensive read. *Dynamic partition pruning* (DPP, on by default via `spark.sql.optimizer.dynamicPartitionPruning.enabled`) rewrites this: it runs the small side first, collects the matching date values, and injects them into the fact scan as if the query had been written

```
WHERE o.order_date IN (SELECT date FROM dim_date WHERE year =
    2026)
```

so the same partition-level skipping from Section 7.1 applies to a filter that, in the query as written, lived on the other table entirely (Figure 7.1).

DPP has two prerequisites, and missing either one silently disables it: the fact table must be partitioned by the join column — there is nothing to skip by directory otherwise — and the dimension side must be cheap enough to execute first (broadcastable, in practice). The reason broadcastability specifically is what unlocks DPP, not just “small,” is the reuse mechanism in Figure 7.1: Spark runs the dimension side once, builds the broadcast hash table for the join out of it as it would anyway, and reuses that *same* in-memory result a second time as the list of matching keys for the fact-table filter — one

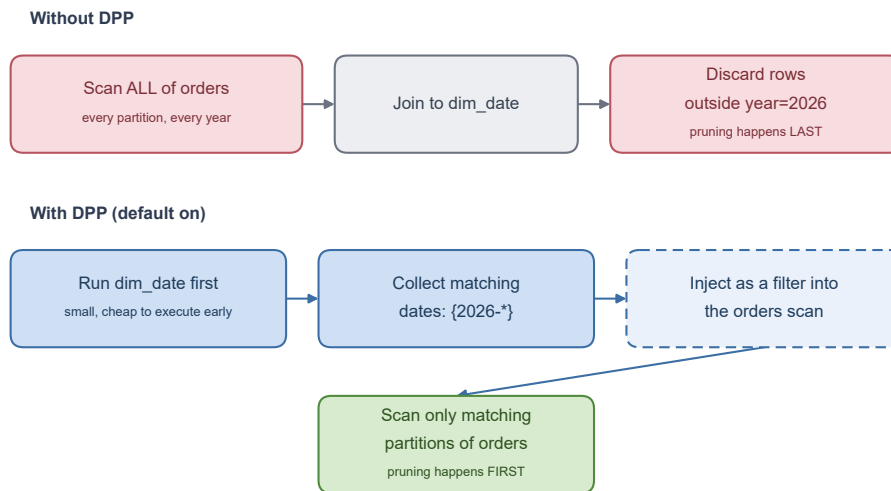


Figure 7.1: Without DPP, pruning happens last: the whole fact table is scanned before the join discards most of it. With DPP, the small dimension side runs first and its matching keys are injected as a filter *before* the fact scan, so pruning happens first instead.

execution, two consumers. If the dimension side is not broadcastable, there is no cheap, already-materialized copy of it sitting in memory to reuse this way; Spark would have to run the dimension side as its own separate query just to get the pruning values, which can cost more than the scan it was supposed to save, so the optimizer does not attempt it.

7.3 Bucketing to avoid a shuffle

Partitioning divides a table by directory; *bucketing* divides it further, within a partition, by hashing a column into a fixed number of files. Concretely, writing with `bucket(16, customer_id)` computes `hash(customer_id) mod 16` for every row and sends it to the file for that remainder — the same `hash(key) mod N` idea Section 4.2 used to assign shuffle output partitions, just applied at write time to decide which *file* a row lands in instead of which in-flight partition. Two tables bucketed on the same column with the same bucket count assign every row the same way from the same `customer_id`, so a row and its join partner always land in the file pair with the same bucket number — which is exactly why they end up co-located file-to-file on that column, and exactly the precondition Section 5.7 used for a storage-partitioned join: the shuffle that a normal join

would pay for is instead paid once, at write time, and every join on that key afterward pays nothing.

Classic Spark bucketing (`df.write.bucketBy(n, col)`, for a Hive-catalogued table) established the idea but is brittle in practice: the two tables must match bucket count exactly, and the bucketing is a Spark-only convention invisible to any other engine reading the same files. Iceberg’s `bucket(n, col)` partition transform (Section 11.1) is the modern equivalent on this stack — recorded in the table’s own metadata rather than inferred from file naming, so any engine reading the table sees and can exploit the same layout.

7.4 repartition versus coalesce

Both change how many partitions a DataFrame has; they get there by opposite mechanisms, and Table 1.2 already told you which is which. `repartition(n)` is wide: it shuffles, so it can raise *or* lower the partition count and it redistributes rows evenly across the new count regardless of how they were distributed before. `coalesce(n)` is narrow: it only lowers the count, and it does so by merging existing partitions together with no shuffle — which also means it cannot fix uneven input. Coalescing four huge partitions and four empty ones down to four still leaves you with four huge partitions; only a shuffle can redistribute the rows themselves.

`coalesce(n, shuffle=true)` exists specifically to opt back into a shuffle when a plain `coalesce` would preserve skew that already exists in the input — at that point it costs what `repartition` costs, and the narrow-versus-wide distinction has been given up on purpose.

7.5 Column pruning and ReadSchema

Column pruning — reading only the columns a query touches — is one of the optimizer’s heuristic rules (Section 2.3) and shows up as the scan’s `ReadSchema` (Section C.3); a `ReadSchema` wider than the query needs almost always traces back to a `SELECT *` in a view, a CTE, or a UDF’s input that the optimizer cannot see through.

It goes one level deeper for a nested (struct) column than the top-level case suggests. Parquet and Iceberg both store nested fields independently enough that a query touching only `event.location.city` out of a wide `event STRUCT<...>` column can prune to that one leaf field, without reading its siblings — visible in `ReadSchema` as the struct type narrowed to just the accessed fields, not dropped as a whole column. Whether this fires depends on the query actually projecting the leaf field explicitly; passing the whole struct into a UDF or a later transformation reintroduces the full column, the same way a `SELECT *` does.

7.6 Summary

Static partition pruning skips whole directories from a literal filter; dynamic partition pruning gets the same skipping for a filter that only exists on the other side of a join, by running that side first and reusing its result — provided the fact table is partitioned by the join key and the other side is cheap to run early. Bucketing extends pruning-by-layout into join cost: two tables bucketed identically need no shuffle to join, which Iceberg’s bucket transform makes a durable, engine-visible property of the table rather than a Spark-only convention. `coalesce` merges partitions without a shuffle and therefore cannot fix skew that already exists; `repartition` can, at the cost of paying for one. And column pruning reaches inside a struct, not just across the top-level schema, as long as the query actually projects the leaf field it needs.

Exercises

1. Write the same filter three ways — a function on the partition column, a `CAST`, and a plain literal — and check which keeps `PartitionFilters` populated in the scan node.
2. Run the star-schema query from Section 7.2 with `spark.sql.optimizer.dynamicPartitionPruning.enabled` set to `false`, then `true`. Compare the partitions read on the fact side in each plan.
3. Take a `DataFrame` with visibly uneven partition sizes and run `coalesce(4)` versus `repartition(4)`. Compare the resulting task-size distribution in the UI.

Chapter 8

Caching and Reuse

Caching keeps a result Spark would otherwise recompute — but the wrong cache wastes storage memory and starves execution, and a cache used once is pure overhead. Spark also reuses work you did not ask it to: an identical exchange or subquery computed once and read twice. This chapter covers when caching pays, which storage level to pick against the GC budget from Chapter 3, and how to see reuse in a plan. It is also where execution and storage memory finally meet at runtime.

8.1 When caching pays

A cached DataFrame pays for itself only when two things are both true: it is read more than once, and it fits comfortably inside the executor’s storage memory (Section 3.3) without pushing out other useful cached data. Caching something read exactly once adds the cost of writing it into storage memory (and, if it does not fit, spilling it to disk) for a saving that never arrives — the classic mistake is caching immediately before a single `write()`, which reads the cached data exactly once and gets nothing back for the cost of caching it.

```
1  -- good: filtered_orders is read by two different aggregations
2  filtered_orders.cache()
3  filtered_orders.count()                -- materializes the
    cache
```

```
4 result1 = filtered_orders.join(dim_product, "product_id")
5 result2 = filtered_orders.groupBy("product_id").sum("
    order_total")
```

Listing 8.1: Reused twice: worth caching

8.2 Lazy cache() versus eager CACHE TABLE

`df.cache()` does not cache anything by itself — it only marks the `DataFrame` for caching the next time an action runs it, consistent with transformations being lazy everywhere else (Section 1.4). Without a materializing action, the first real use of the `DataFrame` both computes it *and* populates the cache as a side effect, which works but hands you no control over exactly when the cost is paid; calling a cheap action like `count()` right after `cache()` is the common way to materialize it deliberately, on your own terms.

Note

SQL's `CACHE TABLE` is the one place this default flips: it is *eager*, executing immediately rather than waiting for an action. Use `CACHE LAZY TABLE` for the same lazy behavior `df.cache()` gives you.

8.3 Storage levels and the GC trade-off

The default storage level, `MEMORY_AND_DISK`, keeps cached blocks as deserialized JVM objects in storage memory and spills to disk only when it does not fit. Those objects carry the same per-object overhead Section 3.2 described for any row, and a large, long-lived cache is exactly the workload that pushes a garbage collector toward its expensive, full-heap path (Section 3.4). Two other levels exist specifically to trade that away:

- `MEMORY_AND_DISK_SER` keeps the cache serialized — Kryo-encoded bytes (Section 3.5) in one contiguous block rather than millions of small objects. Concretely: one million cached orders rows as deserialized JVM `Row` objects carry roughly 50 bytes of per-object overhead each (Section 3.2) on top of their actual field data, plus the garbage collector has a million separate objects to track and trace on every collection; the same million rows serialized into one contiguous `_SER` byte array

pay that overhead exactly once for the whole block, and the collector sees a single large byte array rather than a million objects to walk. That is the $2\text{--}3\times$ memory reduction and the “nearly invisible to the garbage collector” claim made concrete, at the cost of deserializing rows back out of that byte array on every read.

- `OFF_HEAP` (with `spark.memory.offHeap.enabled`) puts the cache entirely outside the JVM heap, the same off-heap pool from Section 3.3 — zero GC overhead for that data, at the cost of the off-heap capacity it consumes.

`DISK_ONLY` is rarely the right answer: reading a cache from disk is close enough to re-reading the source that the cache buys little.

Tip

Switch away from the deserialized default when GC time is 5–10% or more of task time for a job that caches heavily (Section 15.3). Below that, the deserialization cost of a `_SER` level is not worth paying.

8.4 How execution and storage memory borrow at run-time

Section 3.3 described unified memory’s split between execution and storage as a soft boundary set by `spark.memory.storageFraction`; here is what that means while a job actually runs. Either side may use the other’s *idle* space: a job with nothing cached lets execution use the whole unified pool for shuffles and joins, and a job running no wide operators lets storage use all of it for cached blocks.

Here is a concrete sequence of events showing why that boundary only matters once both sides actually want space at the same time. Storage’s own guaranteed half of a 4 GB unified pool is 2 GB. A pipeline caches `filtered_orders`, and that cache grows to 3 GB — 2 GB in storage’s own half, plus 1 GB borrowed from execution’s currently-idle half, because nothing else is running yet. Now a join starts in the same job and its sort buffers need 1.5 GB of execution memory. Execution is entitled to reclaim its own space, so Spark evicts blocks from the borrowed 1 GB (LRU order) to free it for the join, shrinking the cache back down toward storage’s guaranteed 2 GB; if the join later

needs more than its own guaranteed half too, it simply spills to disk rather than blocking, the same as it would with no cache present at all. Run the same scenario in reverse — the join starts first and grows into storage’s idle space, then the cache tries to grow — and storage cannot evict a word of the join’s live hash table to make room; the cache is capped at whatever storage’s guaranteed half still has free until the join finishes and releases its borrowed space back. The two directions are not symmetric:

- if execution needs memory that storage is currently borrowing, Spark evicts cached blocks (LRU) to reclaim it, down to storage’s own guaranteed half;
- if storage needs memory that execution is currently borrowing, Spark cannot evict a running task’s hash table or sort buffer to make room — storage simply does not get that space back until the task finishes.

Execution always wins a live contest, because failing to complete a running operator is worse than evicting a cache entry and recomputing it later. This is also why a cache that is evicted mid-job is not a bug: it is unified memory doing exactly what it is for (Figure 8.1).

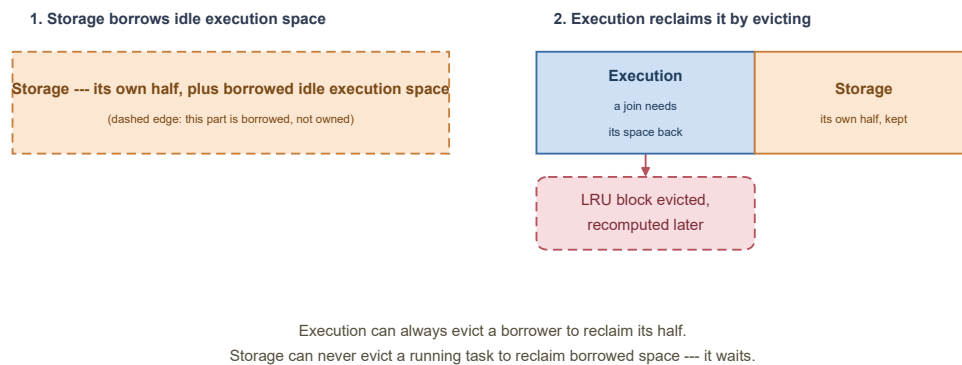


Figure 8.1: Borrowing is symmetric; reclaiming is not. Storage can spread into idle execution space, but the moment a join or sort needs that space back, execution evicts the borrower’s cached blocks to reclaim it — storage has no equivalent power over a running task.

8.5 Exchange and subquery reuse

Caching is a decision you make; Spark also reuses identical work on its own when the physical plan contains the same subtree twice. If two operators need data shuffled the same way — same partitioning, same partition count — the planner reuses one exchange for both instead of running it twice (`ReuseExchange`, Section C.4); an identical scalar subquery appearing more than once in a query is deduplicated the same way (`ReuseSubquery`). Both are visible directly in the physical plan and require no caching from you — and both are fragile in exactly the way Section 4.7 warned about: a partition count or column order that differs by even a little between the two occurrences is enough to defeat the match.

8.6 Unpersist and cache thrashing

A cached `DataFrame` you no longer need should be explicitly released with `unpersist()`. Left alone, storage memory fills with `DataFrames` nobody reads anymore, and Spark's LRU eviction starts discarding whichever cached *block* was used least recently to make room for new ones — eviction happens at the level of individual blocks (one per partition), not whole `DataFrames`, so a large cached `DataFrame` is evicted piecemeal as its least-recently-touched partitions come due, not all at once.

Concretely: cache A (an early pipeline step) was last read 10 minutes ago; cache B (a later, still-active step) was read 1 minute ago. A new cache C needs room storage does not have. LRU picks purely on recency of read, with no notion of which `DataFrame` you still "need" — it evicts blocks from A first, since A's blocks are the least recently used, regardless of whether A or C is actually more important to the job. Under enough pressure this can reach further than intended: if C is read once and then also goes stale, its own blocks become the next LRU candidates, and eventually even B's blocks are at risk if B is not read again soon enough — evicting the cache you actually still need before its next read, forcing a recompute you cached specifically to avoid. This *cache thrashing* is a caching bug that looks like a memory bug: the fix is fewer, more deliberate caches (and prompt `unpersist()` on ones you are done with), not more memory.

8.7 Summary

Cache what is read more than once and fits the storage budget; a cache read once is waste. `cache()` only marks a `DataFrame` lazily, so materialize it on your own schedule with a cheap action; `CACHE TABLE` in SQL is eager by contrast. The deserialized default trades memory for GC pressure; `_SER` and `OFF_HEAP` trade back at the cost of CPU or off-heap capacity. Execution and storage share unified memory and may borrow each other's idle space, but only execution can evict a borrower to reclaim it — storage waits. Spark reuses an identical exchange or subquery on its own, fragile to any difference between the two occurrences; and a cache you forget to release does not just waste memory, it can evict the cache you still need.

Exercises

1. Build a query that reads one CTE twice. Show the plan reusing the exchange, then change the query so the reuse breaks and measure the difference.
2. Cache a `DataFrame` with `MEMORY_AND_DISK`, then with `MEMORY_AND_DISK_SER`. Compare GC time in the Executors tab for a job that scans it repeatedly.
3. Cache three `DataFrames` larger than storage memory can hold at once, read each once, then re-read the first. Confirm from the Storage tab that it was evicted and recomputed.

Part III

The Iceberg Lakehouse

Chapter 9

Table Formats and the Iceberg Spec

An Iceberg table is not a directory of Parquet files with a manifest bolted on. It is a tree: a catalog pointer to a metadata file, to a manifest list, to manifests, to data and delete files — every level immutable, every commit a single atomic pointer swap. Reads consult this tree before touching any data. This chapter walks the spec (format version 2) against a real table’s metadata tables so you can draw the tree from memory.

9.1 From Hive tables to table formats

A classic Hive table’s truth is the filesystem: the Hive Metastore records a table’s schema and its partition-to-directory mapping, but knowing which *files* exist inside a partition means listing the directory at query time. That listing is slow at scale (Section 1.4’s driver work before any task runs), and it is not atomic: a writer that has written three of five new files and a reader that lists the directory in between sees a table that never existed, with no snapshot to fall back to and no way to tell that a write is in progress.

A *table format* — Iceberg, Delta Lake, Hudi are the three in wide use — moves that truth out of the filesystem and into an explicit metadata layer the engine reads instead of listing directories. Every file that belongs to the table is named individually, in a structure written once and never mutated in place; a commit is complete the instant one pointer is swapped to reference a new version of that structure, and a reader either sees the whole new version or the whole old one, never a partial write. Everything else in this chapter

is what that metadata layer is actually made of.

9.2 The Iceberg spec, format version 2

Iceberg’s metadata is a tree of four kinds of file, each immutable once written, each read by the layer above it, top to bottom (Figure 9.1):

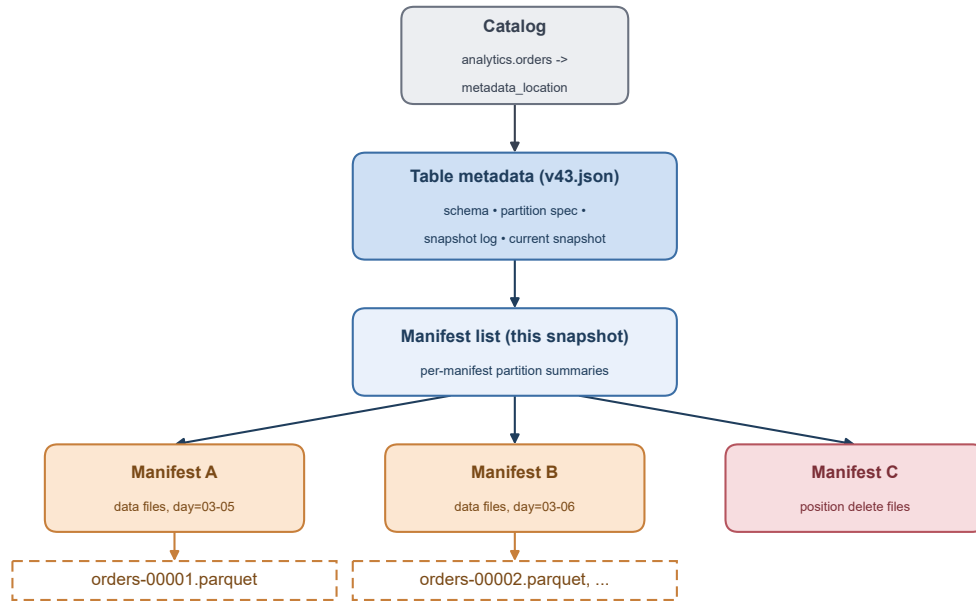


Figure 9.1: The metadata tree for one snapshot of orders. The catalog resolves to one table metadata file, which points to one manifest list, which points to the manifests that make up this snapshot, which list the actual data and delete files. Every level is immutable; a new commit writes new files at whichever levels changed and leaves the rest untouched.

Table metadata file (JSON) is the object a catalog entry points to. It holds the current schema, the partition spec, the full list of snapshots and a *snapshot log*, and the current snapshot’s pointer to its manifest list. This is the one file a reader must locate through the catalog; everything below it is found by following pointers already inside it.

Manifest list (Avro), one per snapshot, is the list of manifest files that make up that snapshot, each entry carrying the manifest’s path, its partition-spec ID, and summary statistics (row counts, per-column value ranges) for the partitions it covers —

enough for a reader to skip an entire manifest without opening it, the first pruning layer Chapter 12 builds on.

Manifests (Avro) each list a set of data files *or* delete files (never both) as `manifest_entry` records, one row per file, carrying the file’s path, format, partition values, per-column min/max and null counts, and the sequence number it was added at. Manifests are grouped by partition spec and reused across snapshots when their files have not changed, which is why an append does not rewrite every manifest in the table — only the ones touched by that commit. Concretely: appending one new day’s worth of orders writes exactly one new manifest, listing only that new day’s data files; the commit’s new manifest list then references that one new manifest plus every manifest from the previous snapshot, unchanged and unrewritten, since none of those older files were touched by this append.

Data and delete files (Parquet on this stack) are the files themselves; delete files are the subject of Section 9.4.

Note

Nothing in this tree is a directory listing. A reader that wants to know which files exist reads exactly the metadata file, manifest list, and manifests that the current snapshot’s pruning has not already excluded — never the filesystem. Chapter 12 is entirely about how much of that tree a well-formed query gets to skip.

9.3 Snapshots and the metadata tree

A *snapshot* is one committed version of the table: a `snapshot-id`, a pointer to its manifest list, an optional `parent-snapshot-id` linking it to the version it replaced, a `timestamp-ms`, and — the detail that makes format v2 usable for row-level deletes — a monotonically increasing `sequence-number`. Every file recorded anywhere in the tree inherits the sequence number of the snapshot that added it, and that number is what lets a reader decide, cheaply, whether a given delete file is even relevant to a given data file: a delete only applies to a data file with an *older* sequence number, so a data file written after a delete was committed is never touched by it. Concretely: data file A is written at sequence number 5; a later commit deletes some of its rows, writing a delete file at

sequence number 8 — that delete applies to A (5 is older than 8) and any reader must apply it when reading A's rows. A still-later commit appends new rows in data file B at sequence number 9. Because 9 is *not older* than the delete file's 8, the sequence number alone tells a reader that this delete cannot possibly apply to B — no need to even open the delete file and check its contents against B's rows.

The table metadata file's *snapshot log* is the ordered history of every snapshot the table has ever had, and it is what powers time travel — `VERSION AS OF / TIMESTAMP AS OF` in Spark SQL resolve to a past `snapshot-id` by walking this log, then read that snapshot's manifest list exactly as if it were current. A commit does not delete an old snapshot's entry from this log; that is a separate, deliberate operation (`expire_snapshots`, Chapter 13) — until it runs, the old files a superseded snapshot referenced are still reachable, which is also why an append does not immediately reclaim the disk space of files it replaced.

9.3.1 The commit as an atomic pointer swap

Writing new data and delete files, and new manifests and a new manifest list to describe them, does not change the table at all — none of it is visible until the table metadata file's pointer is swapped to reference the new snapshot, and that swap is the one atomic operation the catalog performs (Chapter 10 covers exactly how). A reader holding an open scan against the old snapshot is unaffected by a commit happening concurrently: it keeps reading the manifest list it already resolved, which still points at files that still exist, because nothing in this tree is ever mutated in place.

9.4 Position deletes versus equality deletes

Format v1 could only append or replace whole files; format v2's core addition is deleting or replacing individual *rows* inside an existing, untouched data file by recording the deletion in a separate delete file instead of rewriting the data file itself. Two encodings exist, and they trade off *write* cost against *read* cost in opposite directions.

Position deletes identify a deleted row by `(file_path, pos)` — the exact data file and its 0-indexed row ordinal. Applying one at read time is cheap: merge the

sorted list of deleted positions against the data file's rows as they stream past, an operation with no lookup and no hashing. Writing one requires knowing the row's exact position, which means the writer has usually already read the data file — the natural output of a MERGE or DELETE that identifies matching rows during a scan (Section 13.2).

Equality deletes identify a deleted row by column value — `id = 5` — recorded once, independent of which data files happen to contain a matching row. Writing one is cheap, because the writer needs to know only the key, not any row's position; it is the only option available to a writer that never reads the data it is deleting, which is exactly the position a streaming or CDC job is in (Chapter 16). Applying one at read time costs a value comparison against every row of every data file it could apply to, which is why a table accumulating equality deletes without compaction gets measurably slower to read from over time.

Tip

A batch MERGE INTO on this stack writes position deletes, because the join already located the exact rows; a table only accumulates equality deletes if something writes them deliberately (typically a streaming upsert). If read performance is degrading on a table you know only runs batch merges, the delete files are not why — look at file count and partition pruning instead (Chapter 12).

9.5 Iceberg, Delta Lake, and Hudi

All three formats solve the same problem this chapter opened with — metadata as truth instead of directory listing — with different defaults. Table 9.1 is not a ranking; it is what to expect walking into a table built on one of the others.

The mechanism-level questions this book answers — what makes a commit atomic, why a delete file changes read cost, why hidden partitioning survives a scheme change — transfer directly to Delta and Hudi tables even though the concrete file layout differs; only the exact metadata format is Iceberg-specific.

Table 9.1: Where the three table formats put their defaults

	Iceberg	Delta Lake	Hudi
Metadata	Manifest list + manifests (Avro)	JSON + Parquet transaction log	Timeline + per-partition metadata
Default write mode	Copy-on-write; merge-on-read opt-in per table	Copy-on-write; deletion vectors opt-in	Merge-on-read is the common case
Partitioning	Hidden, engine-computed (Chapter 11)	Explicit partition columns	Explicit partition columns
Primary engine origin	Netflix / multi-engine from the start	Databricks / Spark-first	Uber / Spark-first, CDC-oriented

9.6 Summary

A table format replaces filesystem listing with an explicit, immutable metadata tree: a catalog-resolved metadata file points to a manifest list per snapshot, which points to manifests, which list data and delete files. A commit is one atomic swap of that top pointer, which is why a reader never sees a partial write. Format v2’s sequence numbers let a reader tell which delete files apply to which data files without comparing every pair; position deletes are cheap to read and require the writer to have already scanned the row, while equality deletes are cheap to write and progressively more expensive to read as they accumulate. The next chapter covers exactly how that top-level pointer swap becomes atomic when two writers race for it.

Exercises

1. Commit three appends to a table, then inspect `metadata_log_entries`, `snapshots`, and `manifests`. Explain how many metadata files and manifests exist and why.
2. Run one `MERGE INTO` and, separately, simulate an equality-delete write. Compare the

resulting delete files' schema in the `files` metadata table.

3. Query a table with `VERSION AS OF` an old snapshot after a subsequent commit. Confirm from `metadata_log_entries` which manifest list that snapshot still points to.

Chapter 10

Catalogs and Commits

The catalog is the one component that turns a table name into the location of its current metadata — and the point at which a commit becomes atomic. Writing new metadata is not a commit until the catalog’s pointer is swapped to it. When two writers race, one wins the swap and the other retries. This chapter covers what the Hive Metastore and REST catalogs do, how the atomic swap works, and how to tune retries so a busy table does not fail commits under contention.

10.1 What a catalog does

Everything in Chapter 9’s metadata tree is reachable once you have the path to the current table metadata file — the catalog’s only job is to be the durable, atomically-updatable record of that one path per table name. Concretely, a catalog answers two questions: given `db.table`, what metadata file is current, and, on a commit, can I swap that pointer to a new file *only if* no one else has moved it since I last read it. Nothing about the manifest tree, delete files, or partitioning from Chapter 9 lives in the catalog at all — it is small and simple by design, because it is the one piece of the whole system every reader and writer has to agree on synchronously.

10.2 The Hive Metastore catalog

On this stack, `spark.sql.catalog.<name>.type=hive` points Iceberg at the same Hive Metastore (HMS) already serving classic Hive tables. Iceberg stores the current metadata file's location in the Metastore's own table properties (`metadata_location`), and the swap is one HMS `alter_table` call with an expected-old-value check — HMS itself functions as the lock, rejecting the call if `metadata_location` already differs from what the writer last read. This is why an HMS-catalog table can be queried by any engine that already speaks Hive Metastore (Kyuubi included, on this stack) without extra configuration: to that engine, an Iceberg table's catalog entry looks like any other table, only the properties differ.

10.3 The REST catalog

A REST catalog (`spark.sql.catalog.<name>.type=rest`, with a `uri`) puts a dedicated service between every engine and the metadata store, instead of each engine's client implementing Metastore's Thrift protocol directly. A commit is one HTTP request — typically a `POST` to that table's endpoint carrying the base metadata file's identity (its UUID or version number) alongside the new one — and the service rejects it with a conflict response if its own record of the current metadata no longer matches that base, the same expected-old-value check HMS does, just enforced by application code and its own backing store (a database row, most commonly) instead of by a Thrift call into HMS. Every client, regardless of language or engine, calls this same small HTTP API to read or commit a table. The practical difference from HMS shows up at the edges: a REST catalog can support features HMS's schema was never built for (multi-table transactions, credential vending scoped per table — the catalog hands each client short-lived, table-specific storage credentials for exactly the table it just asked to read or write, instead of every client sharing one long-lived, all-tables-accessible static key) without every client needing new Thrift-parsing logic, at the cost of running and operating one more service.

Note

Both catalog types answer the same two questions from Section 10.1; nothing in Chapter 9 or the rest of this book depends on which one a table uses. Pick based on what already exists in the platform (HMS is already running on this stack) and what the ecosystem around the table needs, not on a claimed metadata-format difference — there is none.

10.4 The atomic pointer swap

A commit’s actual sequence, regardless of catalog:

1. read the current metadata file to get the base snapshot;
2. build new manifests, a new manifest list, and a new snapshot on top of that base, writing all of it to storage — none of this is visible to any reader yet;
3. write a new table metadata file referencing the new snapshot;
4. ask the catalog to swap its pointer from the base metadata file to the new one, *conditioned on* the pointer still being the base file.

Step 4 is the only step that touches shared, contended state; steps 1–3 are pure writes to storage that never conflict with another writer’s steps 1–3, because each writer is writing its own new files. This is exactly why Iceberg calls its concurrency model *optimistic*: every writer does the expensive part assuming it will win, and only the cheap, final compare-and-swap can fail.

10.5 Optimistic concurrency and retries

When step 4 fails because another commit already moved the pointer, Iceberg does not discard the writer’s work — it re-reads the new current metadata file, re-applies the same logical change on top of it (a new manifest referencing the same underlying data/delete files, at most a cheap manifest rewrite rather than a data rewrite), and retries the swap. That rewrite is cheap specifically because a manifest is a small listing of file paths and statistics (Section 9.2), not the data itself — its size tracks the number of files touched

by *this* commit, entirely decoupled from how large the table as a whole has grown. Table 10.1 controls how persistent that retry is.

Watching it happen. Say a nightly compaction job (`rewrite_data_files` on `orders`, Chapter 13) and an ad-hoc `INSERT` from a Kyuubi user session both reach step 4 within the same second. Whichever request the catalog processes first — say the compaction job — succeeds: `metadata_location` moves from `v42.metadata.json` to `v43.metadata.json`. The `INSERT`’s swap arrives a moment later still expecting `v42` as the base, HMS rejects it (the expected-old-value check fails), and Iceberg’s client re-reads `v43` as the new base, rebuilds a manifest referencing its two new files on top of it, and retries — succeeding as `v44`. Querying the table’s snapshot history right after both commits shows exactly this, with no failure visible to either job:

```

1 SELECT snapshot_id, parent_id, committed_at, operation
2 FROM lakehouse.analytics.orders.snapshots
3 ORDER BY committed_at DESC LIMIT 2;
4
5 -- snapshot_id | parent_id | committed_at |
6 --            | operation
7 -- 5821...044  | 5821...043 | 2026-03-08 02:00:01.812 | append
8 -- 5821...043  | 5821...042 | 2026-03-08 02:00:01.406 | replace

```

Listing 10.1: Confirming a retried commit in the snapshot log

Two snapshots land within half a second of each other, each with a different `parent_id` — `v43` (the `replace` from compaction) and `v44` (the `append` from the retried `INSERT`, now parented on `v43` rather than the `v42` it originally read). Neither job’s caller ever saw an error; the retry happened entirely inside the client library, as Figure 10.1 lays out step by step.

A table with many concurrent writers hitting the same partitions — several streaming jobs upserting into one table, say — can exhaust the default 4 retries under sustained contention; raising `commit.retry.num-retries` (a table property, set via `ALTER TABLE ... SET TBLPROPERTIES`) trades a slower worst-case commit for a much lower failure rate. It does not help a table where every writer touches the *same* partition on every com-

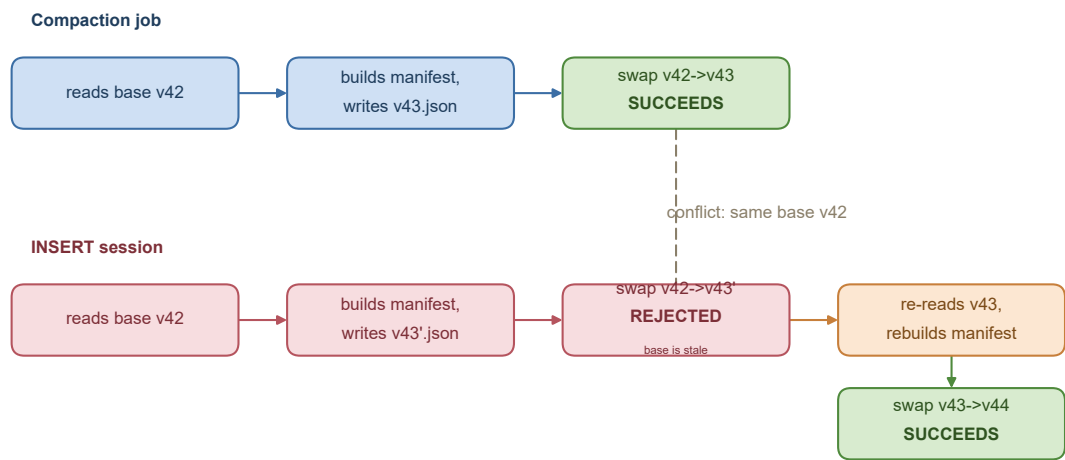


Figure 10.1: Two writers racing for the same base version. The compaction job’s swap lands first and succeeds; the INSERT session’s swap against the same stale base is rejected, so it re-reads the new current version, rebuilds its manifest on top of it, and retries — succeeding one version later, with no error surfaced to either caller.

Table 10.1: Iceberg commit-retry table properties

Property	Default	Meaning
<code>commit.retry.num-retries</code>	4	Max retry attempts before failing the commit
<code>commit.retry.min-wait-ms</code>	100	Initial backoff between attempts
<code>commit.retry.max-wait-ms</code>	60,000	Backoff ceiling
<code>commit.retry.total-timeout-ms</code>	1,800,000	Overall time budget across all retries

mit — retries only help when the underlying data actually merges cleanly with the new base, which Section 11.1’s partition design controls more than any retry setting does.

Warning

Retrying a failed swap is safe because retries re-read the real current state and reapply on top of it — it is not blind resubmission of the same metadata file. A writer that instead cached the base snapshot and kept resubmitting it would silently lose every commit that happened in between.

10.6 Catalog configuration in Spark

A catalog is registered per Spark session under a name of your choosing:

```
1 spark.sql.catalog.lakehouse=org.apache.iceberg.spark.  
    SparkCatalog  
2 spark.sql.catalog.lakehouse.type=hive  
3 spark.sql.catalog.lakehouse.uri=thrift://hive-metastore:9083  
4 spark.sql.catalog.lakehouse.warehouse=gs://my-bucket/warehouse
```

Listing 10.2: Registering an Iceberg catalog against the Hive Metastore

Every table reference is then three-part, `lakehouse.analytics.orders`, resolved through that specific catalog — which matters the moment more than one catalog is registered (a Hive-backed one for the existing warehouse, a REST-backed one for a newer set of tables): the catalog name in the identifier, not any global default, decides which metadata store and which commit protocol from this chapter actually apply.

Note

`org.apache.iceberg.spark.SparkCatalog` has to actually be on the JVM’s classpath for this registration to resolve to anything — on this stack, that means the Iceberg Spark runtime jar is baked into the engine image Kyuubi launches (or added via an init container), not something set by this config block alone. A catalog name that fails to resolve with a `ClassNotFoundException` rather than a clear catalog-not-found error is almost always this: the config is correct, the jar is missing from the engine that tried to honor it.

10.7 Summary

The catalog's whole job is a name-to-current-metadata-file mapping that can be swapped atomically; the Hive Metastore catalog does this with a conditional `alter_table` against the same Metastore serving classic Hive tables, a REST catalog does it behind an HTTP API that can support more than HMS's schema allows, and neither changes anything about the metadata tree itself. A commit writes all its new files optimistically, then attempts one conditional swap; on conflict it re-reads the real current state, reapplies the same change on top of it, and retries within the bounds of `commit.retry.*`, which is the lever to raise for a table under genuine concurrent-writer contention.

Exercises

1. Run two writers into the same table concurrently. Observe the retry, then find the winning and losing commit attempts in the snapshot log.
2. Register the same physical warehouse location under two differently named catalogs in one Spark session. Confirm both resolve the same table to the same current snapshot.
3. Lower `commit.retry.num-retries` to 1 on a table and reproduce a commit failure under two concurrent writers; raise it back and confirm both succeed.

Chapter 11

Iceberg Partitioning and Layout

Iceberg partitioning is hidden: you filter on `order_ts` and Iceberg derives the predicate on `day(order_ts)` for you, so a query never references a partition column directly and pruning does not break when the scheme changes. This chapter covers the partition transforms, evolving a spec without rewriting history, sort orders and Z-ordering, target file size, and the write-distribution mode — the choices that decide how much a reader and a writer have to move.

11.1 Hidden partitioning and partition transforms

A classic Hive table's partition column is a real column duplicated into the directory path — a query has to filter on that exact derived value (`WHERE order_date = '2026-09-20'`) for pruning to fire, and a filter on the underlying timestamp (`WHERE order_ts >= ...`) does not automatically resolve to it. Iceberg's partition spec instead names a *transform* of a source column — `day(order_ts)`, `bucket(16, customer_id)`, `truncate(4, category)` — and stores the computed value in the manifest, not as a queryable column at all. The planner derives the partition predicate from a filter on the source column itself, so the specimen query's plain `order_ts >= TIMESTAMP '...'` filter from Section 7.1 prunes correctly with no partition column ever appearing in the query text.

Table 11.1 covers the transforms this stack uses.

Table 11.1: Common Iceberg partition transforms

Transform	Use case	Notes
day / hour	Time-series data filtered by a timestamp range	Coarser (day) means larger partitions; however, high-volume streams (Chapter 16)
bucket(n, col)	High-cardinality key with equality or join predicates	Even file distribution; hash; enables skip-partitioned joins (join skip the shuffle when sides share the same spec, Section 5.7)
truncate(n, col)	String or numeric prefix filters	Groups by a value without one partition per distinct value
identity	Low-cardinality column filtered directly	Same as classic partitioning; just recorded the hidden

Tip

Prefer day over identity on a raw timestamp: an identity partition on `order_ts` would create one partition *per distinct timestamp value*, which is not partitioning at all — it is one file per row’s worth of metadata. Always partition on a transform coarse enough to group many rows per partition.

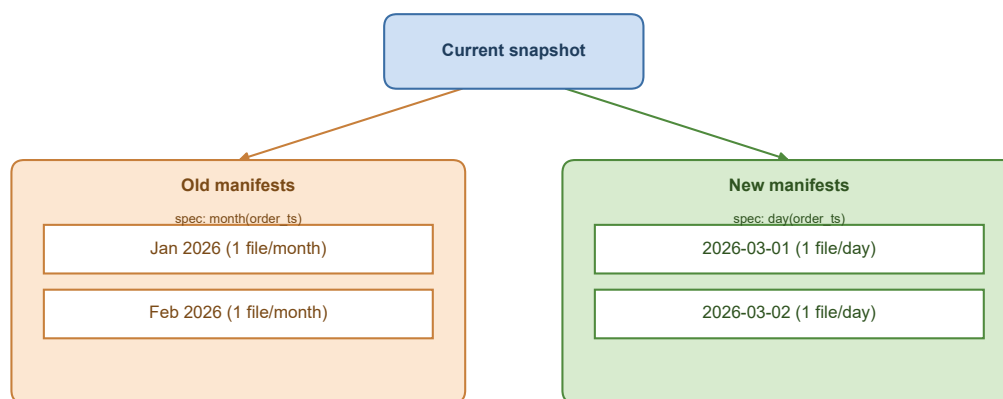
Choosing the specimen’s actual spec. `orders` holds a few hundred million rows a month (Appendix A); at roughly 70 bytes a row once Parquet-encoded, that is on the order of 20 GB a month, or ~700 MB a day. `day(order_ts)` lands squarely inside the 128 MB–1 GB per-partition target this chapter’s exercises ask you to hit, with room to spare before an hour transform would be justified — hour only earns its keep once daily volume grows past roughly 15–20 GB, the point past which a single day’s partition starts exceeding a few gigabytes and a query filtering a narrow time window would otherwise scan a whole day’s files to find a few hours’ worth of rows. This is the concrete version of the abstract choice Section 11.1 describes: pick the coarsest transform that still keeps a typical partition in the target band, and re-derive it as volume grows rather than guessing once and never revisiting it.

11.2 Partition evolution

A classic Hive table’s partitioning is fixed at creation; changing it means rewriting every file under the new scheme. Iceberg’s partition spec can change (`ALTER TABLE ...ADD PARTITION FIELD`) without touching a single existing file, because the transform that produced a file’s partition value is recorded per-manifest, not assumed globally — old manifests keep describing old files under the old spec, new manifests describe new files under the new spec, and a query spans both by evaluating each manifest against its own recorded spec.

This is why moving from `month(order_ts)` to `day(order_ts)` as a table grows is a metadata-only operation, not a maintenance project: new data lands under the finer scheme immediately, old data keeps the coarser scheme it was written with, and a query filtering by date prunes correctly against both — coarser pruning on the old partitions, finer pruning on the new ones, with no explicit migration step and no downtime (Fig-

ure 11.1).



A query filtering by date evaluates each manifest against its OWN recorded spec --- coarser skipping on the left, finer on the right, in the same scan.

Figure 11.1: After evolving orders from `month(order_ts)` to `day(order_ts)`, both specs coexist under the same current snapshot. Old manifests keep describing month-partitioned files; new manifests describe day-partitioned files. One scan reads both, pruning each manifest against its own recorded spec.

11.3 Sort orders and Z-ordering

A partition spec controls which *files* a filter can skip; a *sort order* controls how much of the *content* within a kept file can still be skipped, via the column min/max statistics Chapter 9’s manifests already carry per file. Files written sorted by a column have tight, non-overlapping min/max ranges across files — a filter on that column can then skip whole files even within one partition, the same statistics-based pruning Section C.11 described for row groups, just at the file level.

A single sort order only helps predicates on the columns it is sorted by, front to back — a table sorted by `(category, customer_id)` gives `category` clean file-skipping and gives `customer_id` almost none, because rows are grouped by `category` first and only ordered by `customer` *within* that grouping.

Z-ordering (`rewrite_data_files` with `sort_order` type `zorder`) fixes exactly that asymmetry by building one sort key out of *interleaved bits* from several columns, so no single column gets front-of-key priority. Interleaving means taking each column’s

bits one at a time in round-robin order: with two columns A and B whose binary values are $a_2a_1a_0$ and $b_2b_1b_0$, the Z-order key is $a_2b_2a_1b_1a_0b_0$. Sorting by that combined key groups rows that are close in *either* column, because a small change in A or B alone only flips a few bits near the end of the interleaved key — rows near each other in the two-dimensional (A, B) space end up near each other in the one-dimensional sort order, in both directions at once, at the cost of neither column getting the clean front-of-key locality Section C.11 described for a plain sort. This is the right trade when queries filter on `category` sometimes and `customer_id` other times, rather than one column consistently.

11.4 Target file size and the small-files problem

Every file a reader opens costs a fixed overhead — opening the file, reading its footer, resolving its schema — independent of how many rows it holds (Section 1.4’s input-partition sizing runs into the same fixed cost from the other direction). A table accumulating many small files, typically from frequent low-volume commits (a streaming job flushing every micro-batch, Chapter 16), pays that overhead over and over for little data each time. `write.target-file-size-bytes` (512 MB by default) sets what a write aims for; Chapter 13’s compaction procedures are the after-the-fact fix for files that already landed too small.

11.5 Write distribution modes

Before a write’s rows are grouped into per-partition files, Spark may need to shuffle them so that all rows for a given partition value land together rather than scattered across every task’s output — exactly the shuffle mechanics from Chapter 4, applied to a write instead of a read. `write.distribution-mode` chooses how:

none writes rows in whatever order the upstream plan already produced them, with no shuffle. Cheapest, but every task can touch every partition, so a table with many partitions ends up with many small files per commit — one per (task, partition) pair touched.

hash (the default for unpartitioned or partitioned tables with a defined spec) shuffles by partition value first, so each task after the shuffle owns one partition's rows and writes one file for it. This is the direct fix for the none mode's small-files problem, at the cost of the shuffle itself.

range shuffles by a sampled range of the sort order instead of a hash, producing files with tighter, non-overlapping value ranges — the write-time complement to Section 11.3's pruning benefit. A hash sends a given key to the same partition every time, but two different keys can land in the same partition or two adjacent keys can land in different ones — a hash has no notion of order. Range partitioning needs to know *where* the value boundaries between partitions should fall to keep them evenly sized, so a lightweight sampling pass reads a small fraction of the rows first, estimates the value distribution, and picks boundaries from that sample before the real, boundary-aware shuffle runs — the extra planning cost this mode carries.

For the specimen table, this is the mode that determines what a write does with the viral product from Section 1.6's skew: under hash distribution, every row for that one `product_id` lands in the same write task regardless of how many there are, so a write task for that key can end up far larger than the others — the write-side version of the same skew, covered in full in Section 13.3.

11.6 Summary

Iceberg partitioning is hidden behind a transform of a source column, so pruning works from a plain filter on that column with no partition column in the query; day/hour suit time-series filters, bucket suits high-cardinality keys and storage-partitioned joins, and an `identity` transform on a high-cardinality column is a modeling mistake, not a partitioning choice. Evolving the spec is metadata-only because old files keep the spec they were written under. A sort order or Z-order tightens the per-file statistics that let a reader skip whole files inside a kept partition; target file size and the write distribution mode are what decide whether a write produces a few well-sized files or many small ones in the first place.

Exercises

1. Given a daily data volume and a query pattern, choose a partition spec. Justify it against the target of 128 MB–1 GB per partition per write.
2. Evolve a table's partition spec from `month(order_ts)` to `day(order_ts)`. Confirm from the `partitions` metadata table that old and new data report different specs.
3. Write the same `DataFrame` with `write.distribution-mode` set to `none` and then `hash`. Compare the file count per partition in each case.

Chapter 12

Reading Iceberg Efficiently

Before the Iceberg reader opens a single data file, it has already skipped most of them — by partition, by manifest, by column statistics, and optionally by bloom filter. Each layer needs the query’s filter in a form it understands, and the scan report tells you how well each one worked. This chapter covers the pruning layers, the metrics modes that feed them, and how to read a scan report as a diagnostic.

12.1 Manifest pruning and metadata filtering

A scan resolves in layers, each cheaper than the last and each capable of excluding the whole layer below it (Table 12.1 lists them top to bottom): the manifest list’s per-manifest partition summaries (Section 9.2) let the planner skip an entire manifest without opening it; the surviving manifests’ per-file partition values and column statistics let it skip individual files without opening them; only files that survive both layers are opened at all, and even then column pruning (Section 7.5) and row-group statistics (Section C.11) keep working exactly as they do for any Parquet scan. Static and dynamic partition pruning (Chapter 7) are what feed the literal or derived values these layers filter against — this chapter is about what Iceberg does with a value once Spark has produced one, not about producing the value itself.

Figure 12.1 draws the same four layers as what they are: a funnel, narrowing from every file in the table down to the rows a query actually reads, each layer paying only

Table 12.1: The pruning layers a scan passes through, in order

Layer	Skips
Manifest list	Whole manifests, via per-manifest partition summaries
Manifest entries	Whole data/delete files, via partition value and column min/max/null statistics
Bloom filter (optional)	Row groups, for an equality lookup on a high-cardinality column
Row-group statistics	Row groups within an opened file, via Parquet’s own min/max (Section C.11)

for what survived the one above it.

12.2 Column statistics and predicate pushdown into Iceberg

The manifest-level statistics that power the second layer above — per-file min, max, null count, and value count per column — are exactly the numbers the *metrics modes* in the next section control the granularity of. Iceberg pushes a Spark filter down to this layer the same way it pushes one into Parquet row groups (Section C.11): a plain comparison against a literal resolves; a CAST, a UDF, or a non-deterministic expression does not, and defeats file-level pruning for exactly the reasons already catalogued there. A filter that fails to push down still runs correctly — Spark applies it after reading — it is only pruning, not correctness, that is lost.

12.3 Metrics modes: full, truncate, counts, none

`write.metadata.metrics.default` (default `truncate(16)`) controls how much of each column’s statistics get written into the manifest in the first place, and it is a write-time property with a direct read-time consequence:

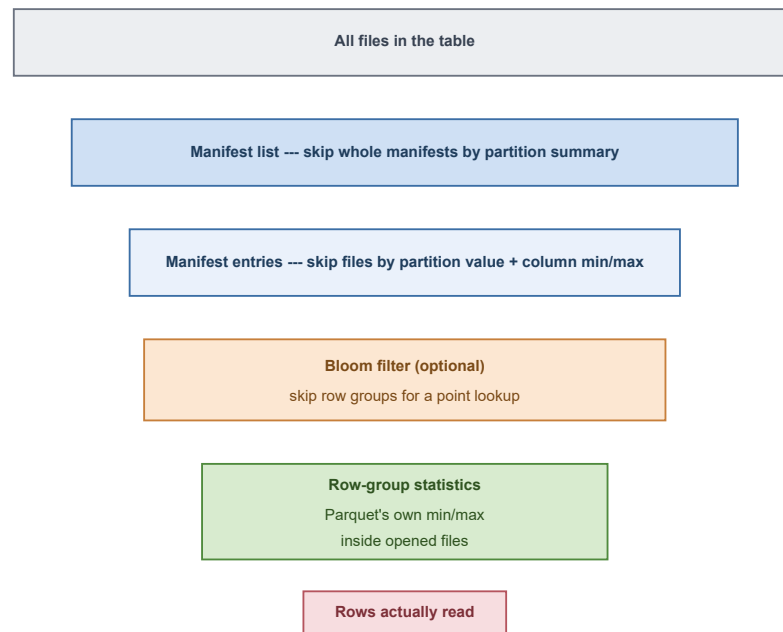


Figure 12.1: The four pruning layers, narrowing top to bottom. Each layer only ever has to consider what the layer above it did not already exclude — a file skipped by the manifest list is never even checked against a bloom filter or row-group statistics.

full stores the exact min and max value, uncapped. Best pruning, largest manifest entries — expensive for a wide string column with long values.

truncate(16) (the default) stores only the first 16 bytes of a string’s min/max. Pruning still works for any predicate that differs within the first 16 bytes; two values agreeing on their first 16 bytes look identical to the pruning layer even if they differ further in. Concretely: `order-2026-03-05-00000001` and `order-2026-03-05-00099999` share their first 16 characters (`order-2026-03-0`) — so a manifest holding only a file’s *first* of those two values as its recorded max still reports a max of `order-2026-03-0...` for the file, and a filter for the *second*, higher value is not confidently excluded even if the file in fact never contains it: the truncated max simply cannot distinguish the two, so the file is kept as a candidate and opened to check, costing exactly the read the metadata layer exists to avoid.

counts keeps null and value counts only, no min/max at all — rules out “is this file all-null” checks but nothing value-based.

none keeps nothing; the file is never skipped by this layer regardless of the filter.

`write.metadata.metrics.column.<col>` overrides the default per column, which is the practical lever: set `full` on the column your biggest queries filter by most (the specimen's `order_ts`, say) and leave wide free-text columns at `counts` so their manifests do not balloon for no pruning benefit.

12.4 Bloom filters

Min/max statistics prune well for a range predicate but not for a point lookup on a high-cardinality column with no ordering to exploit — a `WHERE order_id = '...'` on an unsorted column has a min/max range covering nearly every file, so almost nothing gets skipped. A Parquet bloom filter, enabled per column with `write.parquet.bloom-filter-enabled`, answers a narrower question directly — “can this row group possibly contain this exact value” — with no false negatives and a small, tunable false-positive rate (`write.parquet.bloom-filter-fpp`, 1 MB default, per row group). It costs write-time CPU and file size to build and is only worth enabling on columns that are actually queried by equality, not by range.

12.5 Vectorized reads

`read.parquet.vectorization.enabled` (default `true`) reads a *batch* of rows into a columnar, Tungsten-friendly in-memory format (Chapter 17) per call instead of materializing one Java object per row — the same columnar-processing benefit Parquet's own layout is built for, carried through into Spark's execution rather than undone by row-at-a-time decoding. `read.parquet.vectorization.batch-size` (default 5,000) sets how many rows make up one batch; larger batches amortize per-batch overhead further but hold more decoded data in memory at once.

12.6 Split planning

Section 1.4 described input partitions as roughly `read.split.target-size` (128 MB default) worth of data each; here is where that number actually comes from for an Iceberg scan. The planner bins manifest entries toward that target, using `read.split.open-file-cost` (4 MB default) as a floor weight per file so that many small files are not treated as free

to combine — each one still carries roughly 4 MB of assumed per-file overhead in the packing decision, whether or not it is actually that large.

Put numbers on why that floor matters: ten small compacted-away files at 1 MB of real data each add up to only 10 MB, which naive size-based packing would combine with over a hundred such files into a single 128 MB split — except each open file costs something at read time (opening a handle, reading its footer, initializing a reader) regardless of how little data it holds, and a split with 128 tiny files pays that overhead 128 times over. The 4 MB floor corrects for this: each of those ten 1 MB files is costed at 4 MB for packing purposes, so their true 10 MB counts as 40 MB of “effective” size, and only about three such 4 MB-floored files (12 MB effective) fit into a target-size bin alongside a couple of normal 40 MB files — far fewer tiny files crammed into one split than their real bytes would suggest, which keeps the file-open overhead from dominating the split’s actual read time. `read.split.planning-lookback (10)` bounds how many candidate bins the packer considers at once, trading perfect packing for bounded planning time on a manifest with very many files.

12.7 The metadata tables as a diagnostic

Every layer above is inspectable directly, without guessing from query latency: `table.files` lists every live data and delete file with its size and record count; `table.manifests` shows manifest-level summaries; `table.partitions` aggregates file counts and sizes per partition, the fastest way to spot a partition accumulating too many small files (Section 11.4); `table.snapshots` and `table.history` show the commit sequence a scan report’s `snapshot-id` resolves against. Reading these tables directly, rather than inferring layout from query behavior, is usually the faster way to confirm a suspicion from the scan report below.

Tip

Spark’s `EXPLAIN` on an Iceberg scan reports `skippedDataManifests`, `scannedDataManifests`, `resultDataFiles`, and `resultDataFileSize` (Section C.3) — the pruning ratio at each of the first two layers in Table 12.1 is just `skipped / (skipped + scanned)` at the manifest level, and `resultDataFiles`

against `table.files`'s total count at the file level.

12.8 Summary

A scan prunes in layers of increasing cost — manifest list, manifest entries, optional bloom filter, row-group statistics — and each depends on the query's filter surviving pushdown the same way it must for plain Parquet. Metrics modes decide how much statistics-based pruning is even possible per column, trading manifest size against pruning power; bloom filters answer point lookups that min/max cannot; vectorized reads carry Parquet's columnar layout through execution instead of decoding row by row; and split planning is where the 128 MB input-partition target from Chapter 1 is actually computed for an Iceberg table. The metadata tables and the scan report's own counters are how you confirm any of this actually fired, rather than assuming it from configuration alone.

Exercises

1. Run a selective query and, from the scan report, compute the fraction of data pruned at each layer (partition, manifest, file).
2. Enable a bloom filter on a high-cardinality column, run an equality lookup before and after, and compare `resultDataFiles`.
3. Query `table.partitions` on a table you suspect has small-file pressure and identify which partitions need compaction first.

Chapter 13

Writing and Maintaining Iceberg

Copy-on-write rewrites whole files on every update: slow writes, fast reads. Merge-on-read writes small delete files instead: fast writes, slower reads until compaction catches up. The choice is a function of your update rate and your read latency budget, and it commits you to a maintenance schedule — compaction, snapshot expiration, orphan-file cleanup — in a specific order. This chapter covers the write modes, optimizing MERGE, and running maintenance without deleting data a reader still needs.

13.1 Copy-on-write versus merge-on-read

Every UPDATE, DELETE, or MERGE against a matched row is, underneath, one of two things: rewrite the data file that row lives in with that row changed or removed (*copy-on-write*, CoW), or leave the data file untouched and record the change as a delete file plus, for an update, a new data file holding only the changed rows (*merge-on-read*, MoR, built on Section 9.4’s delete files). `write.delete.mode`, `write.update.mode`, and `write.merge.mode` set this independently per operation type; check a table’s current properties rather than assuming a default, since Iceberg has changed which mode ships as the default across releases.

The trade is exactly where each mode’s name says it is: CoW pays the full cost of rewriting a file at write time so every subsequent read sees plain data files with no delete files to merge; MoR pays almost nothing at write time — a small delete file,

no data rewrite — and pushes the cost onto every read until compaction (Section 13.4) catches up. Table 13.1 is the concrete rule: a table with a low update rate and latency-sensitive reads wants CoW; a table absorbing frequent small updates (a streaming upsert, Chapter 16) wants MoR, because CoW there would mean rewriting the same large file on nearly every micro-batch.

Table 13.1: Choosing copy-on-write versus merge-on-read

	Copy-on-write		Merge-on-read	
Write cost	Rewrites	whole matched files	Small delete file, no rewrite	
Read cost	Plain data files, no merge		Merges delete files at read time	
Best fit	Infrequent updates, latency-sensitive reads		Frequent updates, latency-sensitive	small write-
Needs	—		Regular compaction to bound read cost	

Figure 13.1 follows the same UPDATE down both paths to make the trade concrete: same three matched rows, opposite placement of the cost.

13.2 MERGE, UPDATE, and DELETE

A row-level operation’s ON or WHERE predicate does two different jobs, and conflating them is the single most common performance mistake against these tables. If the predicate matches whole partitions — resolvable the same way Section 7.1 resolves a scan filter — Iceberg performs a *metadata-only delete*: it drops the file references from the manifest without reading or rewriting a single row, because no row inside an untouched partition needed to change. If the predicate only narrows which *rows* inside a file match, Iceberg has to open those files, find the matching rows, and rewrite (CoW) or emit deletes

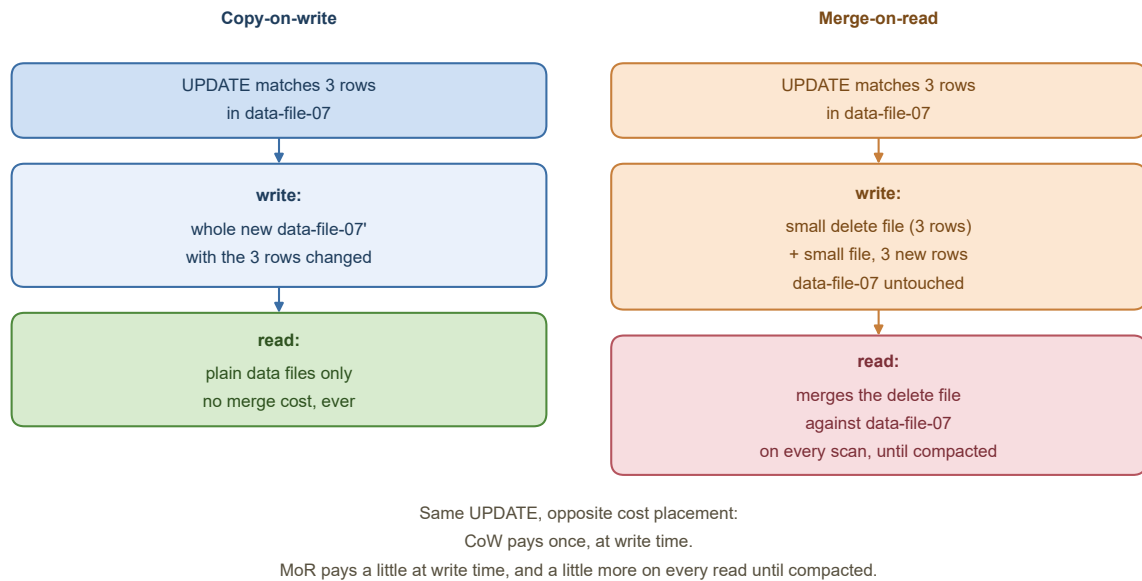


Figure 13.1: The same UPDATE under copy-on-write and merge-on-read. CoW pays once, at write time, and every later read is plain data with no merge; MoR pays a small write-time cost and pushes the rest onto every read until the next compaction.

for (MoR) exactly the files that contained a match — every other file in the table is untouched either way, provided the predicate lets partition pruning rule them out first.

```

1 MERGE INTO orders o
2 USING staged_orders s
3 ON o.order_date = s.order_date AND o.order_id = s.order_id
4 WHEN MATCHED THEN UPDATE SET *
5 WHEN NOT MATCHED THEN INSERT *;

```

Listing 13.1: Partition predicate in the ON clause keeps the scan narrow

Dropping `o.order_date = s.order_date` from the ON clause does not change which rows match — `order_id` alone is still sufficient — but it removes the one predicate the planner could resolve against partition values before scanning, so every partition in `orders` is opened to search for an `order_id` match instead of only the partitions `staged_orders` actually touches. This is Chapter 7’s pruning story again, specific to the write path: a MERGE without a partition predicate is not wrong, it is just a full scan wearing a row-level operation’s syntax.

13.3 Write distribution and skew on write

Section 11.5 covered `write.distribution-mode` for a plain append; a row-level operation inherits the same choice, with one extra failure mode. A MERGE whose matched rows are concentrated in a handful of partitions — a backfill touching one day’s data, say — shuffles unevenly under hash mode exactly the way a skewed join does (Section 6.5), except AQE’s skew-split machinery is a read-side mechanism and does not rebalance a write.

The specimen’s own viral, heavily-discounted product (Section A.3) makes this concrete. Say a MERGE runs against one week of orders to correct a pricing error, matching on `product_id`. Under hash distribution, matched rows shuffle by `product_id` before the rewrite, so every row for the viral product — 40 million of the week’s 200 million rows, all sharing one key — lands in a single shuffle partition, while each of the week’s few thousand other products spreads its handful of rows thinly across the rest. The task that draws the viral product’s partition rewrites roughly 40 million rows; every other task in the same stage rewrites a few hundred thousand at most. That one task runs as long as it takes to process 40 million rows while the rest of the stage finishes early and sits idle — the exact straggler shape Section 6.5’s figure showed for a skewed join, except here it is a CoW rewrite task, and AQE’s join-side skew-split machinery does not apply to it: nothing currently detects or splits an oversized write task the way AQE does for a read.

13.4 Compacting data files

`rewrite_data_files` is the direct fix for both small files (Section 11.4) and MoR read cost: it rewrites a set of files into fewer, better-organized ones, in one of three strategies. `binpack` (the default) merges small files toward the target size with no reordering — cheapest, and sufficient when the only problem is file count. `sort` rewrites according to a defined sort order (Section 11.3), restoring tight min/max ranges that ordinary appends erode over time. `zorder` does the same for several columns interleaved together, at a higher compute cost, when queries filter on different columns unpredictably. All three also apply outstanding delete files to the data they rewrite, which is why running

compaction is the concrete answer to a MoR table whose reads have been getting slower.

13.5 Rewriting manifests

Manifests accumulate their own version of the small-files problem: many small commits each add a small manifest, and a scan's manifest-list pruning (Section 12.1) works less well the more, smaller manifests it has to consult individually. `rewrite_manifests` consolidates them into fewer, larger manifests with the same entries, improving planning time without touching a single data or delete file — a metadata-only compaction, distinct from and cheaper than `rewrite_data_files`.

13.6 Snapshot expiration and orphan files

Section 9.3 noted that superseded files stay reachable until something removes their snapshot from the log — that something is `expire_snapshots`: past a retention window, it drops old snapshot entries and, for any file no longer referenced by *any* remaining snapshot, deletes the physical file. This is the mechanism that actually reclaims the disk space compaction's rewrites and a row-level operation's rewritten files leave behind; skipping it does not corrupt anything, it just means old files accumulate indefinitely.

`remove_orphan_files` is a different, independent safety net: it scans the table's storage location directly for files that are not referenced by *any* manifest at all — typically the debris of a failed write that never committed, since a successful commit's files are always referenced. Because it does its own storage listing rather than reasoning from snapshots, it needs a generous `older_than` window (the default is three days) to avoid racing an in-flight write that has written files but not yet committed the metadata referencing them.

Warning

Guides disagree on a strict global order for these two procedures, and the official documentation does not mandate one. The mechanism explains why: `expire_snapshots` is what makes old files eligible for deletion by un-referencing them, and it is reasonable to run it before or interleaved with

`remove_orphan_files`, which only ever touches files no snapshot ever referenced in the first place — the two operate on disjoint sets of files by construction, as long as `remove_orphan_files`'s retention window is wide enough to never touch a write still in flight. Treat the "run compaction, then expire, then remove orphans" sequencing some operational guides recommend as a safe default cadence, not a correctness requirement.

13.7 Metadata management for streaming tables

A streaming job committing every micro-batch (Chapter 16) is the worst case for every mechanism in this chapter at once, and each consequence compounds a different cost already covered above. Many small commits mean many small files, which is Section 13.4's small-files problem, but arriving continuously rather than from one bad batch job. They also mean many small manifests, which slows manifest-list pruning (Section 12.1) on every subsequent read, since more, smaller manifests must be opened and checked individually to plan the same scan. And every commit adds one more entry to the table metadata file's snapshot log (Section 9.3), so that log — and the metadata JSON a client must parse just to resolve the current snapshot, and the search space time-travel queries walk — keeps growing for as long as the stream runs uncompact, all from normal operation rather than a mistake. The practical answer is scheduling all three maintenance operations to run continuously and automatically against a streaming table — compaction and manifest rewriting on a short cadence matched to the commit interval, snapshot expiration on a looser one — rather than treating maintenance as an occasional manual task the way it can be on a table that only receives periodic batch writes.

13.8 Summary

Copy-on-write and merge-on-read trade write cost against read cost in opposite directions, set independently per operation type; pick from the update rate and the read-latency budget, not by default. A row-level operation's predicate does double duty — matching rows and, if it also resolves against partition values, keeping the scan narrow the same

way any other filter does. Compaction and manifest rewriting are metadata- and data-level cleanups that keep reads fast; snapshot expiration and orphan-file removal are what actually reclaim disk space, operating on disjoint sets of files with no single mandated order between them. A streaming table needs all of this running continuously rather than occasionally.

Exercises

1. Run a MERGE with and without the partition column in the ON clause. Explain the difference in files scanned from the plan.
2. Set a table to merge-on-read, run ten small updates, then run `rewrite_data_files`. Compare scan latency before and after.
3. Delete an entire partition's worth of rows in one statement and confirm from the plan that it resolved as a metadata-only delete.

Part IV

Operating Spark

Chapter 14

Tuning Resources and Configuration

This chapter does not hand you a table of recommended settings. It derives an executor shape from the memory map in Chapter 3 and the shuffle in Chapter 4, so that every number — cores, heap, overhead, partition count — has a reason you can defend. It then covers dynamic allocation, the external shuffle service on Kubernetes, configuration precedence, and running Spark behind Kyuubi for many users. For the standalone list of tuning knobs with values, see `spark_memory_tuning.md` in the repository root.

14.1 Executor sizing from the mechanism

Sizing an executor is choosing three numbers — `spark.executor.cores`, `spark.executor.memory`, `spark.executor.memoryOverhead` — and each has a mechanism from an earlier chapter that bounds it, not a rule of thumb.

Cores per executor sets how many tasks run concurrently inside one JVM, which sets how many task-sized shares the unified memory pool from Section 3.3 gets divided into — more cores per executor means each concurrent task’s share of execution memory is smaller, which means a join or aggregation that was comfortable at four cores can start spilling at eight with no other change. Five cores is the commonly cited ceiling

on this stack's Kubernetes deployment, not because of memory but because the GCS connector's own per-JVM connection pool degrades under too many concurrent requests from one process: past roughly five to eight simultaneous readers sharing that one pool, per-request latency climbs faster than the added parallelism gains you back, so an eight-core executor can finish a GCS-heavy scan stage *slower* than a five-core one running the same stage, purely from connection contention that never shows up as a memory or CPU metric. This is the same throughput ceiling originally observed against HDFS clients, and one that applies here for the identical reason: one executor JVM, one shared client, one pool, regardless of which object store sits behind it. There is no memory-model reason to stop earlier than five if the workload reads little from GCS directly (all its input already cached, say); the ceiling is specifically about concurrent storage-client requests, not about anything this section's memory arithmetic bounds.

Executor memory is sized backward from the largest single operation you expect a task to run — a sort-merge join's sort buffer, a hash aggregation's hash table — divided by the fraction of unified memory one task actually gets (execution memory divided evenly among concurrently running tasks, Section 3.3), then inflated to account for `spark.memory.fraction`'s default 60% split between unified and reserved-plus-user memory. Guessing low here is what Section 15.3's spill metrics exist to catch.

Memory overhead (`spark.executor.memoryOverhead`, default the greater of 10% of executor memory or 384 MB) covers everything outside the JVM heap that the container still needs: off-heap Tungsten memory if enabled, JVM metaspace and thread stacks, and — specific to this stack — PySpark's separate Python process when UDFs are in use. The formula is a floor, not a measurement: 384 MB covers the JVM's own fixed costs (metaspace, thread stacks) on a small executor where 10% would be too little, and 10% scales that floor up for a large executor where metaspace stays roughly fixed but off-heap Tungsten buffers and any PySpark worker grow with it. Neither number is derived from anything about *your* workload, which is exactly why a container OOM kill with no corresponding Java heap error (Section 15.4) is almost always this default proving too small for what is actually running outside the heap — raise it directly rather than trusting the formula once PySpark UDFs or off-heap execution memory are in use.

A worked shape for the specimen. Section 1.4’s stage 2 runs the specimen’s sort-merge join and hash aggregate across 200 shuffle partitions; at the reference cluster’s 4 cores per executor (Appendix A), each core is one task slot, so up to 4 of those 200 partitions execute concurrently per executor. If the join’s build-side partition (post-shuffle, decoded) runs 300 MB at its largest — the viral product’s oversized partition from Section 1.6 — each concurrent task needs that much execution memory, and Section 3.3’s unified pool splits evenly across the 4 concurrent tasks: $4 \times 300\text{MB} = 1.2\text{GB}$ of execution memory alone. At `spark.memory.fraction`’s default 0.6 unified share, that implies roughly $1.2\text{GB}/0.6 \approx 2\text{GB}$ of heap just to keep that one partition from spilling, before adding headroom for the rest of a normal working set — a defensible starting point of `spark.executor.memory=4g`, with `spark.executor.memoryOverhead` left at its 10% default (409.6 MB) since no PySpark UDF or off-heap pool is in play here.

Tip

On Kubernetes, the pod’s memory limit is executor memory plus overhead plus (if set) off-heap size — exceeding it kills the pod outright with no JVM exception to read, which is why Section 3.3’s ”container limit” bracket exists as a distinct concept from the heap it contains.

14.2 Dynamic allocation

A fixed executor count is either wasted (idle executors between jobs in a long-lived session) or insufficient (a burst of work with no room to grow). `spark.dynamicAllocation.enabled` lets Spark request and release executors against the cluster manager as work arrives and drains: it adds executors when the scheduler’s pending-task backlog persists past `spark.dynamicAllocation.schedulerBacklogTimeout` (1s default), and removes an executor once it has been idle past `spark.dynamicAllocation.executorIdleTimeout` (60s default), bounded by `spark.dynamicAllocation.minExecutors` and `.maxExecutors`.

The one condition that makes this safe is exactly the failure mode Section 15.4 described: an executor holding shuffle files that a later stage still needs cannot simply be released. Without an external shuffle service, Spark tracks this via `spark.dynamicAll`

`ocation.shuffleTracking.enabled` (on by default since Spark 3.0), which keeps an executor from being removed while its shuffle output is still referenced; with an external service (Section 14.4), the files outlive the executor entirely and removal is unconstrained by this concern.

14.3 Parallelism defaults

`spark.default.parallelism` governs RDD operations with no other hint (defaulting to total core count across executors); for DataFrame/SQL workloads, `spark.sql.shuffle.partitions` (Section 4.4, Section 6.2) is the number that actually matters on this stack. The one addition worth making here rather than in either earlier chapter: this default and AQE’s coalescing are a *resource-sizing* lever as much as a shuffle one — the same 200-versus-2000-partitions choice that trades task-scheduling overhead for spill risk also decides how many of the executor shape’s task slots a given stage can actually fill, so size partitions and executors together, not as two independent decisions.

14.4 The external and push-based shuffle service

Without an external shuffle service, a reduce task fetches shuffle blocks directly from the executor that wrote them (Section 4.2) — which is exactly the dependency dynamic allocation has to work around above, and exactly what turns a voluntary executor removal into a `FetchFailedException` if the removal races a still-pending fetch. `spark.shuffle.service.enabled` moves block serving out of the executor JVM and into a separate long-lived process, so shuffle data remains servable after the executor that wrote it is gone — dynamic allocation can then scale down aggressively with no data-loss risk, because the data was never tied to the executor’s own lifecycle. Push-based shuffle (Section 4.6) builds on the same external service to also cut fetch-side amplification.

Warning

On YARN, the external shuffle service is a NodeManager auxiliary service available on every node by default. On Kubernetes it is not: it has to be deployed and run explicitly, as a DaemonSet with one pod per node, before `spark.shuffle.se`

`rvice.enabled` does anything — setting the config alone with no `DaemonSet` running leaves executors unable to reach a shuffle service at all, which surfaces as fetch failures indistinguishable from the problem the config was meant to prevent. Confirm the `DaemonSet` is actually running on the cluster before relying on this setting.

14.5 Configuration precedence and profiles

A given key can be set in more than one place, and Spark resolves conflicts in a fixed order, highest priority first:

1. values set programmatically on a `SparkConf` in application code;
2. `--conf` flags (or `--executor-memory-style` shortcuts) passed to `spark-submit`;
3. values in `spark-defaults.conf` on the client;
4. each setting's built-in default.

A value that looks ignored is almost always being overridden from a higher-priority layer than the one you edited — checking `spark.conf.get(key)` inside a running session, which reports the value actually in effect, settles this faster than re-reading config files.

14.6 Kyuubi and multi-tenancy

Kyuubi (this stack's gateway) runs one Spark application per Kyuubi *engine* rather than one per user's individual query, and its *share level* setting decides how those engines are pooled and how a new connection finds its engine. Mechanically, Kyuubi's server process keeps a registry of live engines keyed by a scope derived from the share level, backed by Zookeeper (or etcd) so the registry survives a Kyuubi server restart and multiple Kyuubi server instances see the same engines. A new JDBC connection computes its key, looks it up in that registry, and either attaches to the engine already running under that key (a registry *hit*) or, on a miss, launches a fresh Spark application and registers it

under that key for the next connection to find. Table 14.1 lays out the three share levels this stack chooses between.

Share level	Registry key	Isolation	Cold-start cost
CONNECTION	connection id	one engine per JDBC connection	every connection is a registry miss — maximum startup latency, maximum isolation
USER	username	one warm engine per user, reused across their connections	only that user’s first connection is a miss; every later one is a hit
SERVER	a constant	one engine shared by every user on the instance	almost always a hit, but every user’s jobs compete for the same executors

Table 14.1: Kyuubi’s share levels trade isolation for cold-start cost by changing what the registry key is derived from.

Trace what this means for one user, *alice*, under USER sharing. Her first JDBC connection of the day computes key `alice`, finds nothing registered under it, and pays the full cost of launching a fresh Spark application before her first query can even start planning — a registry miss. She disconnects and reconnects an hour later for a second query; this connection computes the same key `alice`, finds her still-running engine from before registered under it, and attaches to it directly with no new application launched — a registry hit, and her query starts planning immediately. Had the deployment used CONNECTION sharing instead, that second connection would have computed a fresh connection id, found nothing under it, and paid the same cold start *alice*’s very first connection did — the isolation CONNECTION buys costs exactly this warm-reuse benefit USER sharing gives for free.

An *engine pool* pre-registers several engines under a share level before any connection asks for one, so even a first connection under that scope is a registry hit rather than a cold start. All of this sits above the resource-sizing questions from this chapter’s earlier sections, not in place of them — each pooled engine is still an ordinary Spark application whose executors are sized exactly as described above.

14.7 Summary

Executor cores, memory, and overhead each trace back to a mechanism from earlier chapters — concurrent tasks dividing the unified memory pool, the largest operation’s actual memory need, and everything living outside the JVM heap, respectively — rather than a rule of thumb. Dynamic allocation is safe to scale aggressively once shuffle data no longer lives and dies with the executor that produced it, which the external (and, further, push-based) shuffle service guarantees. Configuration resolves in a fixed precedence order regardless of how many places a key is set, and Kyuubi’s engine share level is a pooling decision layered on top of, not instead of, ordinary executor sizing.

Exercises

1. Take a job that spills. Propose two different fixes — more memory versus fewer cores — and predict which the mechanism favours, and why.
2. Given a 32 GB, 8-core node, propose an executor shape (cores, memory, overhead) and justify the core count from the connection-pool consideration in Section 14.1.
3. Enable dynamic allocation without the external shuffle service on a job with a long-running final stage. Predict, then observe, what happens when an idle executor holding needed shuffle data is considered for removal.

Chapter 15

Debugging and Profiling

A slow job leaves evidence: in the SQL tab, the stages tab, the executor metrics, the event log, and — when you need it — a flame graph. This chapter teaches you to read that evidence in order, so that "the job is slow" becomes "stage 4 is skewed" or "the executors are GC-bound" in a few minutes. It also catalogs the failure signatures — OOM and container kills, fetch failures, stragglers — and what each looks like before you reach for a fix.

15.1 The Spark UI as a diagnostic

The UI is organized as a funnel (Figure 15.1), and reading it in the wrong order is the most common way to waste time on a slow job. The **SQL** tab is the right starting point for a SQL/DataFrame job: it shows the physical plan with live metrics on every node — rows produced, time spent, bytes shuffled — so you can see *which operator* is expensive before worrying about which stage or task. The **Stages** tab is next: each stage's task-duration distribution (min/median/max, visualized as a histogram) is where skew becomes visible as a wide gap between median and max, and Section 6.5's straggler pattern is exactly what a stage page with one outlier bar looks like. The **Executors** tab aggregates across the whole application — GC time, shuffle read/write totals, task counts per executor — and is where an imbalance across executors (rather than across tasks within one stage) shows up. Only once the SQL tab has named the operator and the

Stages tab has named the stage does drilling into individual task metrics (Section 15.3) become the right next step, rather than the first one.

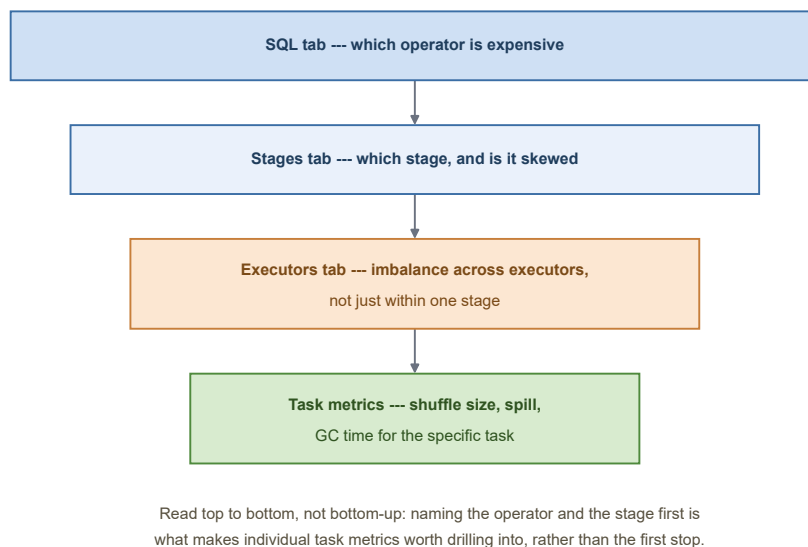


Figure 15.1: The UI read top to bottom: each tab narrows the search before the next one is worth opening. Starting at task metrics without first naming the operator and the stage is the most common way to waste time reading a plan that turns out not to be the bottleneck.

15.2 Event logs and the history server

Everything the live UI renders is first written to an *event log* — a JSON-lines file of every scheduler event (job/stage/task start and end, executor added/removed, SQL execution metrics), one line per event, in the order they occurred. The Spark History Server does nothing more exotic than replay that file from the top: it feeds each line into the same in-memory listener objects that build the live UI’s job, stage, and task tables while an application is actually running, just fed from a file instead of from live scheduler events. By the time it reaches the last line, it holds the identical in-memory picture the live UI would have shown at the moment the application exited — and renders that as a normal, browsable UI. This matters for exactly the jobs you most need to debug: a job that failed or was killed no longer has a live UI to inspect, but its event log (if `spark.eventLog.enabled` was on, which it should be by default on this stack) survives it. On

Kubernetes specifically, the event log is the only diagnostic that outlives the executor pods themselves, which are usually garbage-collected by the cluster shortly after the application ends.

15.3 Reading task, shuffle, spill, and GC metrics

A single task’s detail page is the finest-grained evidence the UI offers, and four numbers do most of the diagnostic work. Take the specimen’s viral, heavily-discounted product from Section A.3: the join stage over orders has 200 tasks, and one of them — the task holding that product’s rows — is the one worth opening.

Task duration vs. scheduler delay separates genuine compute time from a task that merely waited for a slot. The skewed task shows a duration of 4 minutes against a scheduler delay under 100ms: almost all of that time is real work, not queueing — which rules out “not enough executors were free yet” and points straight at “this task has too much to do.” A different task showing a short duration but a long scheduler delay would mean the opposite: a parallelism problem, not a per-task one, and no amount of per-task tuning would fix it.

Shuffle read/write size compared across tasks *in the same stage* is the most direct skew signal: the median task in this stage reads about 40MB shuffled in; the skewed task reads 3.2GB — eighty times the median, because every row for that one product landed in its partition. This is the same imbalance Section 6.5 detects and fixes automatically for a join, made visible here for any stage regardless of whether AQE’s skew handling applies to it.

Spill (memory and disk) appears when a task’s execution memory share (Section 3.3) was not enough for its sort or hash operation, forcing a spill to disk mid-task. The skewed task shows 1.8GB of disk spill where every other task in the stage shows zero: its 3.2GB of input simply did not fit in the same per-task memory slice that comfortably held every other task’s 40MB. Non-zero spill appearing on a stage that previously had none, after a data-volume increase, is the direct evidence a memory-based fix (Section 14.1) targets.

GC time as a fraction of task time above roughly 10% signals a heap doing more collection than the work is worth (Section 3.4). The skewed task’s GC time is 45 seconds out of its 4-minute duration — about 19% — because holding and spilling 3.2GB of rows keeps the heap under sustained pressure the other, lightly-loaded tasks never experience; this is the number Section 8.3’s `_SER`-level advice and any heap-vs-off-heap decision should be checked against.

Read together, these four numbers tell one consistent story rather than four separate ones: a task with far more shuffle input than its neighbors, which therefore spills, which therefore drives up GC time, which therefore dominates the task’s own duration while barely touching its scheduler delay. That is what data skew looks like from inside a single task’s metrics, and it is the same underlying cause Section 6.5 names from the stage-wide view.

15.4 Common failure signatures

A job rarely fails as one incomprehensible error. It fails as one of a small number of recognizable signatures, each with its own evidence in the UI and its own fix.

OOM and container kills. Three distinct failures share the word “OOM,” and the fix depends entirely on which one occurred. A **driver OOM** throws `java.lang.OutOfMemoryError` in the driver’s own log, and its usual cause on this stack is an action that pulls too much data back to one JVM — `collect()` on a large `DataFrame`, or a broadcast join whose broadcast side turned out larger than estimated (Section 5.5). An **executor OOM** throws the same exception but on an executor, from a task’s own operation outrunning its execution-memory share (Section 3.3) with no room left to spill further — visible as the exception in that executor’s log, attached to a specific task. A **container OOM kill** shows neither exception: the cluster manager (Kubernetes) kills the whole pod because its total memory usage — heap plus everything Section 14.1 put under “overhead” — exceeded the pod’s limit, and the pod simply disappears with an `OOMKilled` status visible in `kubectl describe pod`, not in any Spark log at all, because the JVM never got the chance to log anything.

Tip

No exception in any Spark log is itself the diagnostic signal for a container kill. If a task's executor simply vanishes with no `OutOfMemoryError` anywhere, check the pod's exit reason before assuming a fetch failure or a node eviction — raising `spark.executor.memoryOverhead` is the fix for this signature specifically, not for a heap-level executor OOM.

Stragglers. A straggler is one task in a stage running far longer than the rest with no corresponding data-size difference to explain it — distinct from the skew signature in Section 15.3, where the slow task really is processing more bytes. A straggler with ordinary-sized input usually traces to the node it landed on: contended CPU from a noisy neighbor, a slow disk, or a transient network issue with GCS specific to that node, none of which show up in the shuffle-size or spill metrics because the task's own data volume was never the problem. *Speculative execution* (`spark.speculation`, off by default) is the direct mitigation: past a threshold multiple of the median task duration in a stage (`spark.speculation.multiplier`, 1.5 by default), Spark launches a duplicate copy of the slow task on a different executor and keeps whichever copy finishes first, discarding the other. It trades some duplicated compute for tail latency, and is worth enabling exactly when stragglers are confirmed to be node-related rather than data-related — turning it on to paper over genuine skew just runs the same oversized task twice.

Fetch failures and stage retries. Shuffle files are not deleted as soon as they are read — Spark keeps them in case a later stage needs the same shuffle again, and cleans them up in the background as the driver's JVM garbage-collects the objects that reference them (the *ContextCleaner*). In practice the deciding moment is simpler: that storage is local to the executor that wrote it (Section 4.2), and on the reference stack it is an `emptyDir` tied to the executor's pod, so losing the pod — an OOM kill, a node drain, dynamic allocation scaling down — deletes its shuffle files immediately, with no way to recover them from that executor again.

A reduce task that tries to fetch from such an executor gets a `FetchFailedException`, visible in the driver log and the stage's tasks tab. This is not a generic task failure: the DAG scheduler recognizes it specifically, marks that executor's map output as gone in

the map output tracker, and resubmits only the map tasks that ran on it — not the whole stage, and not the job — before retrying the reduce stage. You will see this in the UI as a stage that appears to run twice, with a smaller second attempt.

Tip

Two different layers retry, and tuning the wrong one does nothing. A fetch that fails from a flaky network but a still-live executor is retried by the shuffle client itself: `spark.shuffle.io.maxRetries` (3 attempts) and `spark.shuffle.io.retryWait` (5s between them). A fetch that fails because the executor is genuinely gone is handled by the stage resubmission above, bounded by `spark.stage.maxConsecutiveAttempts` (4) and, per task, `spark.task.maxFailures` (4) — raise both on a cluster where executors are routinely reclaimed (spot instances, aggressive dynamic allocation) rather than let the job fail after only a handful of retries. If the lost executors are *voluntary* removals under dynamic allocation (Section 14.2) rather than crashes, `spark.shuffle.service.enabled` avoids the failure entirely by keeping shuffle data servable after its executor is gone (Section 4.6). Enabling `spark.excludeOnFailure.enabled` keeps subsequent retries from landing back on a node that is already causing them.

15.5 async-profiler and flame graphs

The UI's metrics answer *which stage and which task*; they do not answer *which line of code, or which JIT-compiled method, is actually burning CPU* inside a task that is neither spilling nor GC-bound but is simply slow. Older Java profilers worked by *instrumentation*: rewriting every method's bytecode to insert a timing call at entry and exit, so the overhead scales with how often a method is called — exactly the methods you most want to profile are the ones this slows down the most. `async-profiler` instead uses *sampling*: it interrupts the JVM at a fixed interval (say, every 10ms) and simply reads whatever call stack every thread happens to be running at that instant, using the OS's own `perf_events` rather than any JVM-level instrumentation. The cost is fixed per interrupt, independent of how hot the code is, which is why it can be left attached to a production executor with negligible overhead. Sampled over enough intervals, the

fraction of samples landing in a given method converges on the fraction of wall-clock time it actually consumes — and that is the whole flame graph: one horizontal bar per stack frame, width proportional to time spent, stacked by call depth. A wide bar low in the stack under a Spark-internal frame usually means the bottleneck is in a UDF or an expensive expression the generated code calls into, not in Spark’s own execution machinery (Chapter 17) — the distinction that decides whether the fix is rewriting a UDF or tuning Spark configuration.

15.6 Debugging query plans

A plan that does not do what you expect is diagnosed the same way Chapter 2 taught you to read one, just applied adversarially: compare the *analyzed* plan against what you intended to express (a wrong join condition or an unexpected implicit cast usually shows up here, before any optimization has had a chance to help or hurt); check the *optimized* plan for whether pushdown and pruning rules actually fired (Section C.11’s killer catalogue is the first thing to check when they did not); and compare the *physical* plan’s join and exchange nodes against what Chapter 5 and Chapter 4 predict from the table sizes and partitioning involved. EXPLAIN FORMATTED’s numbered node list plus per-node detail block is almost always more useful for this than the tree-only default mode, because it lets you cross-reference a specific node’s detail (its exact PartitionFilters, its exact join build side) rather than inferring it from the tree shape alone.

15.7 Summary

Diagnose top-down: the SQL tab names the expensive operator, the Stages tab names the stage and shows skew as a task-duration outlier, and only then do individual task metrics — shuffle size, spill, GC time — confirm the specific mechanism. Event logs and the history server are what make this possible after a job has already exited, which is exactly when you need it most. The three OOM signatures — driver, executor, container — point at different fixes and are distinguished by where (or whether) an exception appears at all; a straggler with no data-size explanation is a node problem that speculative execution mitigates, not a skew problem that repartitioning fixes. When metrics alone do not

localize a CPU-bound slowdown, a flame graph does; when a plan does not match what you expected, re-read it stage by stage — analyzed, optimized, physical — the same way Chapter 2 taught, just with suspicion.

Exercises

1. Given three anonymized stage summaries, diagnose each as skew, under-partitioned, or GC-bound, and name the evidence.
2. Trigger each of the three OOM signatures deliberately (a driver `collect()`, an undersized executor heap, an undersized container overhead) and confirm each leaves a different trace.
3. Take a job with a confirmed straggler and enable `spark.speculation`. Confirm from the Stages tab that a duplicate task attempt was launched and which copy won.

Chapter 16

Streaming into the Lakehouse

Structured Streaming is the same query engine over an unbounded table, processing one micro-batch at a time and ending each batch with a single Iceberg commit. The whole model from the earlier parts still applies — the shuffle, the executor budget, the commit — but with two new pressures: state that must be kept between batches, and a commit cadence that produces small files faster than a batch job. This chapter covers the streaming model and what it does to an Iceberg table.

16.1 The micro-batch model and the unbounded table

Structured Streaming treats a stream as a table that keeps growing, and a streaming query as an ordinary DataFrame query re-run incrementally against the new rows since the last run. Each run is a *micro-batch*: the driver's `StreamExecution` asks the source for the range of new offsets available, plans and executes an ordinary batch job over exactly that range — the same Catalyst pipeline, the same job/stage/task machinery from Chapter 1 — and, on success, records the offsets it just processed to a *checkpoint* location before asking for the next range. Nothing about execution is special to streaming; what is special is that this loop runs forever and that recovery after a failure means replaying from the last checkpointed offset rather than restarting the whole computation from the beginning (Figure 16.1).

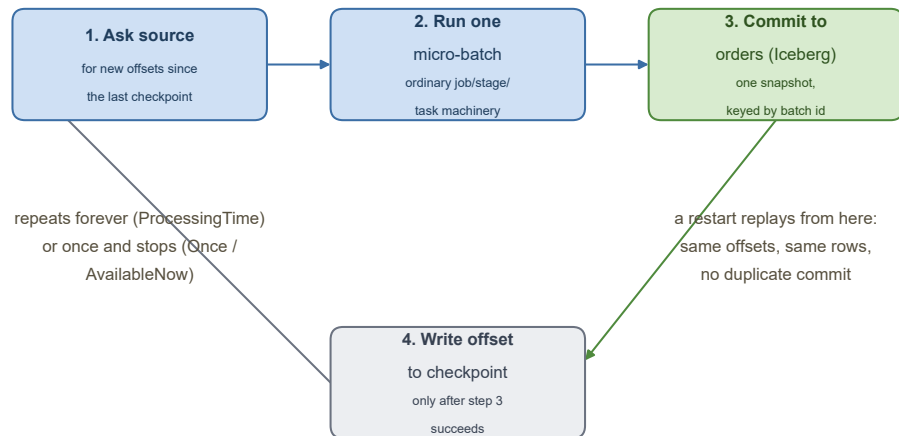


Figure 16.1: One iteration of the micro-batch loop. The checkpoint offset is only written after the Iceberg commit succeeds, which is what makes a restart safe: replaying from the last checkpointed offset re-runs exactly the range that either never committed or already did, never anything in between.

16.2 Sources and sinks

A source only needs to support one contract: given a starting offset, return the new records since it, and report the latest offset available. Two sources make that abstract contract concrete. Kafka is the most common source in the wild, with a Kafka partition's offset serving directly as the streaming offset, and is used here as the running example for exactly that reason — nothing in this chapter is Kafka-specific, though, and the same offset contract is what lets Spark treat a directory of arriving files (the `file` source, polling GCS for new objects under a prefix, using each file's name as its offset) exactly the same way. A sink is symmetric, needing to either support exactly-once writes idempotently keyed by batch ID, or tolerate the batch being reprocessed after a failure. Writing to an Iceberg table as a sink (Section 16.7) satisfies this because a commit is already atomic (Section 9.3.1) — a batch that failed after writing files but before committing simply never became visible, and retrying it from the same checkpointed offset produces the same rows again with no duplicate ever committed. A stream ingesting order events into the specimen's orders table, for instance, would run a Kafka source

(one partition per, say, a coarse hash of `customer_id`) into a `writeStream` targeting `lakehouse.analytics.orders`, with every micro-batch's rows carrying the same columns Chapter 1's batch queries already read.

16.3 Event time and watermarking

Grouping by *event time* (when something happened, carried in the data — the specimen's `order_ts`) rather than *processing time* (when Spark saw it) needs a rule for when a time-based group is finally complete, since late-arriving data can always show up for a window Spark already emitted a result for. A *watermark* (`withWatermark("order_ts", "10 minutes")`) declares how late data is tolerated to be: Spark tracks the maximum `order_ts` seen so far and treats any window ending more than the watermark delay before that maximum as closed, discarding (or routing separately) anything arriving later for it.

```
Max order_ts seen so far: 10:47    ->  watermark = 10:37
Window [10:30, 10:40) already closed (ends before 10:37)    ->
    emitted, forgotten
Window [10:40, 10:50) still open    (ends after 10:37)        ->
    still accepting rows
An order_ts=10:35 row arrives late -> falls in the closed
    [10:30,10:40)
                                     window -> dropped, not
                                     counted
An order_ts=10:41 row arrives late -> falls in the still-open
                                     [10:40,10:50) window ->
                                     counted normally
```

Listing 16.1: Which orders close a 10-minute window, and which arrive too late

The watermark is also the mechanism that bounds state size (Section 16.4) — without one, a windowed aggregation would have to keep every window's partial state forever, since nothing would ever tell it a window was safe to forget.

16.4 Stateful operations and the state store

A windowed aggregation or a streaming join needs to remember partial results across micro-batches — this is the *state store*, checkpointed alongside the offsets so it survives a restart. The default in-memory provider keeps state as JVM objects, off-heap’s usual trade from Section 17.2 in reverse: fast access, but subject to the same GC pressure as any other heap data once state grows large. A one-shot batch aggregate never has this problem: it reads its input once, aggregates, and the state disappears when the job ends. A streaming aggregation is different precisely because the job never ends — its partial state (one entry per open window, per distinct key) keeps accumulating for as long as the query runs, which is why state size, not input size, is what eventually threatens the heap in a way no batch operator ever has to plan around. `RocksDBStateStoreProvider` (`spark.sql.streaming.stateStore.providerClass`) moves state into an embedded RocksDB instance per task instead — not a plain off-heap byte buffer the way Tungsten’s pages are, but a full *log-structured merge tree* (LSM tree): a storage structure that never updates a file in place, only ever appending. Writes land in a small in-memory buffer first; once that buffer fills, it flushes as one new, immutable, sorted file on disk; older files are periodically compacted (merged and deduplicated) in the background to bound how many separate files a single read has to check across. That structure is what lets RocksDB hold far more state than fits comfortably in the JVM heap, at the cost of an occasional compaction pause and a disk read for state that is not in its block cache — the practical choice once state size (a wide window, a large number of distinct keys) makes the default provider’s GC pressure the bottleneck rather than the actual streaming computation.

16.5 Output modes and triggers

Output mode controls what a micro-batch emits: `append` emits only new rows that are already final (rows in a closed watermark window, or a stateless transform); `update` emits any row whose aggregate result changed this batch, including one already emitted before; `complete` re-emits the entire result table every batch, viable only for a small aggregate.

Take a streaming count of orders per category, windowed hourly: under update, a batch that saw three new `electronics` orders emits one row, (`electronics`, `count`=+3...), for the category that changed — categories with no new orders this batch emit nothing; under complete, the same batch emits one row for *every* category, changed or not, which is only affordable because the number of categories is small and fixed, unlike the number of orders. `append` would reject this query outright, because a running count has no notion of “final” until its window closes.

Trigger controls when a micro-batch runs: `ProcessingTime` fires on a fixed interval regardless of how much data is waiting, running the same `StreamExecution` loop from Section 16.1 forever; `Once` (and its successor, `AvailableNow`) reuses that identical loop for exactly one iteration — it asks the source for everything available, runs one micro-batch over all of it, checkpoints, and then simply does not ask for a next range — rather than following a separate code path, which is exactly why the claim that streaming is “the same query engine, incrementally” holds even in this mode. This is the trigger that turns Structured Streaming into a scheduled incremental batch job rather than an always-on process: point a nightly Kubernetes cron job at a streaming query with `AvailableNow`, and it processes everything that landed since the last run and exits, without paying for a permanently running cluster.

16.6 Exactly-once and the commit

“Exactly-once” describes the combination of two guarantees, neither sufficient alone: the checkpoint’s offset log records exactly which input range a batch covers before that batch’s output is committed, so a restart after a mid-batch failure replays the same range rather than skipping or duplicating input; and the sink must make committing that batch’s output idempotent, so replaying the same range twice does not produce the output twice. Writing to Iceberg gets the second half for free, structurally: Spark’s Iceberg sink writes each micro-batch’s incrementing batch id into that snapshot’s own summary metadata (`spark.batch-id`) as part of the ordinary atomic commit from Section 9.3.1 — no extra write, no separate bookkeeping table. On a retry, before committing, the sink checks the table’s existing snapshots for one already carrying that batch id: if the previous at-

tempt's commit already succeeded (crashed only after the swap, before acknowledging the driver), the sink finds its own batch id already there and skips committing again; if the previous attempt never reached the swap, no snapshot carries that id yet and the commit proceeds normally. Either way there is no in-between state where a duplicate partial write is visible to a reader — the batch id living inside the same atomic commit it protects is what makes the check race-free.

Trace it with numbers. Batch 42 processes its assigned offset range, writes its data files, and the executor JVM is killed mid-commit — the driver never receives an acknowledgment. Structured Streaming restarts the query, re-reads the checkpoint, sees batch 42 as the next batch to run, and replays the exact same offset range. Before writing anything, the sink queries `lakehouse.analytics.orders.snapshots` for one whose summary carries `spark.batch-id = 42`. Two outcomes are possible, and only the metadata decides which: if the killed attempt's commit had actually completed its atomic pointer swap (Section 9.3.1) a moment before the JVM died, snapshot v88 already carries `spark.batch-id = 42` in its summary, the sink recognizes its own work, and skips writing batch 42's rows a second time; if the swap never happened, no snapshot anywhere carries that id, and the sink commits batch 42's data files normally as snapshot v88. A reader querying the table mid-retry sees either the pre-batch-42 state or the fully committed batch-42 state — never a partial or duplicated one.

16.7 Writing to Iceberg: cadence, small files, maintenance

Every micro-batch that writes to Iceberg ends in exactly one commit — one new snapshot, one new manifest list, at least one new manifest — regardless of how little data that batch contained. A short trigger interval processing a modest input rate is therefore the single fastest way to produce the small-files and rapidly-growing-manifest problems Chapter 13 described in the abstract: a 10-second trigger against a source producing a few megabytes per batch commits a new, small-file-laden snapshot six times a minute, all day, whether or not the table's total data volume ever justifies that many files.

Sizing it for the specimen. Say order events arrive steadily enough to add 5 MB of new rows to orders per micro-batch, and the write uses `write.distribution-mode=hash` (Section 11.5), so each batch produces roughly one file per touched partition rather than scattering across all of them — close to one 5 MB file per commit at low traffic. At a 10-second `ProcessingTime` trigger that is $6 \times 60 = 360$ commits, and so also roughly 360 new small files, every hour: $360 \times 24 \approx 8,600$ files a day, almost all far below the 512 MB target from Section 11.4. Widening the trigger to 2 minutes drops that to 30 commits an hour — 720 files a day, each nearer 60 MB — at the cost of up to 2 minutes’ extra latency before a row is queryable instead of up to 10 seconds.

Tip

The lever is the trigger interval traded against latency, not the Iceberg side of the write. Widening `ProcessingTime` multiplies the average file size roughly proportionally for the same throughput, at the cost of that much more end-to-end latency before a row becomes queryable — decide the interval from the latency budget first, then treat Section 13.7’s continuous-maintenance schedule as the backstop for whatever file size the chosen interval implies, not as a substitute for choosing a sane interval.

16.8 Summary

A streaming query is an ordinary batch query re-run incrementally over new offsets each micro-batch, recovering by replaying from a checkpoint rather than restarting; nothing about execution itself is special. Watermarking is what lets a windowed aggregation ever consider a window closed, and is also what bounds state size; RocksDB moves that state off the JVM heap once it gets large enough to matter. Exactly-once needs both an offset log that knows exactly what a batch covers and a sink that can commit that batch idempotently — Iceberg’s atomic, batch-keyed commit supplies the second half structurally. And every commit’s fixed overhead, multiplied by a short trigger interval, is the direct cause of a streaming table’s small-files and manifest-growth pressure, which is a cadence decision to make deliberately, not a defect to discover later.

Exercises

1. Given a trigger interval and an input rate, estimate files committed per hour and the compaction cadence needed to keep the table healthy.
2. Run a windowed aggregation with and without a watermark; observe state store size growth over time in the UI's Streaming tab.
3. Kill a streaming query mid-batch and restart it. Confirm from the table's snapshot log that no duplicate commit resulted.

Part V

Internals

Chapter 17

Tungsten and Code Generation

Below the plan, a stage does not interpret operators row by row. Tungsten stores rows as packed off-heap bytes, and whole-stage code generation collapses an operator subtree into a single generated Java method that the JIT compiles to tight machine code. This chapter opens that layer: the binary row format, off-heap memory management, cache-aware computation, and the generated code you can actually read with `EXPLAIN CODEGEN`.

17.1 The Tungsten binary row format

A plain JVM object holding a row — one field per Java object, boxed primitives, object headers — is exactly the per-object overhead Section 3.2 described, multiplied by every row in every partition. `UnsafeRow` avoids it by laying a row out as one contiguous byte buffer instead of a graph of objects: a fixed-size null-tracking bitset first, then one 8-byte slot per field (a primitive stored inline, or an offset-and-length pair pointing into a variable-length region at the end of the same buffer for a string or binary value). Figure 17.1 shows this for one row of the specimen’s orders table: three fixed 8-byte fields sit inline, while `product_id` — a string, so its length varies row to row — carries only an (offset, length) pair in its fixed slot, pointing further into the same buffer where its actual bytes live.

Reading a field is a direct byte-offset access into this buffer — for a fixed field, read

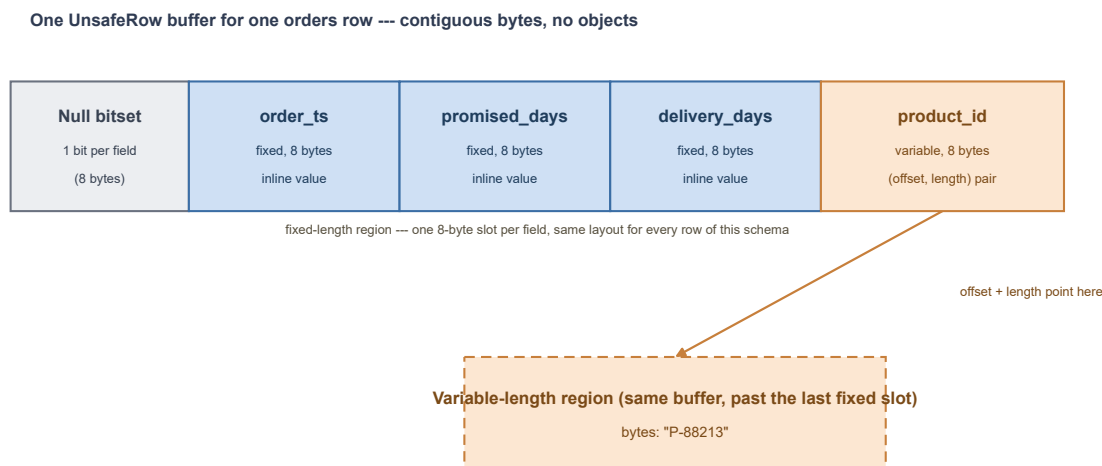


Figure 17.1: One UnsafeRow: a null bitset, one 8-byte slot per field, and a variable-length region past the last fixed slot. A fixed field’s slot holds its value directly; a variable field’s slot holds an (offset, length) pointer into the variable region of the same buffer.

8 bytes at a statically known offset; for a variable field, read the (offset, length) pair, then read that many bytes starting at that offset — with no pointer chasing through a graph of objects and no per-field object to allocate or garbage-collect. The trade is that every value must fit this fixed encoding, which is exactly why Catalyst carries a strict, fully resolved schema (Chapter 2) all the way to execution — the row format has no room for a value whose type or size was not known in advance.

17.2 Off-heap memory management

UnsafeRow buffers do not have to live on the JVM heap at all. When `spark.memory.offHeap.enabled` is set, Tungsten’s `TaskMemoryManager` allocates fixed-size pages directly from off-heap memory (the same pool Section 3.3 carved out of the executor’s total footprint) using `sun.misc.Unsafe`-style raw memory access rather than the JVM’s normal object allocator. A row living in an off-heap page is invisible to the garbage collector entirely — not “cheap to collect,” but never considered by a GC cycle at all — which is the mechanism behind Section 3.4’s claim that off-heap execution memory trades GC pressure for manual lifecycle management: Spark itself must now free a page explicitly when a task no longer needs it, work the JVM would otherwise have done

automatically at some GC pause's cost.

Trace one page through its lifecycle to see what "manual" actually means here. A task building a hash table for the specimen's join over `product_id` asks the `TaskMemoryManager` for a page — say 64 KB, Tungsten's usual default page size — and gets back a raw pointer into off-heap memory, not a Java object. As matching orders rows arrive, the task writes each one's `UnsafeRow` bytes directly into that page at increasing offsets, tracking how much of the page is used itself, since nothing else is tracking it. When the page fills, the task asks for another one; when the task finishes, either successfully or via a failure that unwinds it, it calls `freePage()` on every page it still holds, explicitly returning that memory to the pool. Skip that last step — a bug in an operator's cleanup path, not a normal case — and the memory is simply gone until the executor JVM itself exits, since no collector is ever going to notice it is unreachable and reclaim it the way it would an abandoned Java object.

17.3 Cache-aware computation

A CPU cache miss costs orders of magnitude more than a cache hit, and a sort or hash operation over an array of object pointers — the classic JVM collection layout — follows a different, effectively random pointer on every comparison, defeating the cache almost completely. Tungsten's sort implementation instead packs a compact (*prefix*, *pointer*) pair per row into one flat array: the prefix is a fixed-size encoding of enough of the sort key to resolve most comparisons without ever dereferencing the pointer, so the hot comparison loop reads sequentially through one small, cache-resident array, only chasing the pointer to the full row on the rare comparison the prefix cannot resolve alone. Concretely, sorting the specimen's orders by `order_total` (a double): the prefix is that value's own sortable bit encoding, packed directly into the fixed-size prefix slot — so two rows with `order_total` of \$42.50 and \$187.00 are fully ordered by comparing their prefixes alone, with the pointer to each row's full data never touched at all during that comparison; the pointer is only followed on the rare case of two rows sharing the exact same `order_total`, where the prefix cannot distinguish them and a real tie-break against the full row is unavoidable. For fixed-width numeric keys, Tungsten goes further

with a radix sort, which sorts by key bytes directly with no comparisons at all.

17.4 Whole-stage code generation internals

Interpreting a plan operator by operator means a virtual call and a full row materialization at every boundary between them — a filter producing a row, handed to a project, handed to an aggregate, each step paying function-call overhead the JIT cannot see through because the call target varies with the plan. *Whole-stage code generation* collapses a chain of adjacent operators that support it (marked `CodegenSupport`, a Scala trait an operator implements to opt into this fusion, internally) into one generated Java method, using a *produce/consume* protocol: each operator’s `produce()` emits the code that generates its rows, and calls into the next operator’s `consume()` inline rather than through a virtual dispatch — the whole fused chain becomes one straight-line loop the JIT compiler can then optimize as a single unit. The `*(n)` markers this produces in a plan were already introduced in Section 2.5 and catalogued in Section C.9; what those two places don’t show is what the generated method actually looks like, which is the point of running `EXPLAIN CODEGEN` yourself rather than taking the marker on faith.

Listing 1.1’s stage 1 scan-and-filter of orders (Section 2.6’s plan, node `*(1)`) compiles to something close to this, trimmed to the *produce/consume* skeleton:

```
== Subtree 1 / 2 (maxMethodCodeSize:389) ==
*(1) Filter (order_ts#12 >= 2026-03-01 AND order_ts#12 <
    2026-03-08)
+- *(1) ColumnarToRow
    +- BatchScan orders

Generated code:
final class GeneratedIteratorForCodegenStage1 extends
    BufferedRowIterator {
    protected void processNext() throws java.io.IOException {
        while (scan_input.hasNext()) {
            InternalRow scan_row = (InternalRow) scan_input.next();
            long scan_value = scan_row.getLong(1);           //
                order_ts, inline field
```

```
// Filter.consume(): the range check, straight-line, no
    virtual call
if (scan_value >= 1772323200000000L
    && scan_value < 1772928000000000L) {
    append(scan_row);                                // ->
    next_operator's consume()
}
}
}
}
```

Listing 17.1: EXPLAIN CODEGEN, trimmed to the *(1) scan-filter stage

Three unfamiliar names carry the whole mechanism, so before reading the payoff, here is what each one is. `BufferedRowIterator` is the base class every generated whole-stage method extends — it is Spark’s own plumbing, not something you write, and it is what lets the generated class plug into the rest of the engine as an ordinary iterator despite being freshly compiled at runtime. `InternalRow` is Catalyst’s internal row representation — the `UnsafeRow` byte layout from Section 17.1 accessed through this interface, which is why `scan_row.getLong(1)` above is a direct, typed field read rather than a generic get-and-cast. `append(scan_row)` is how one fused operator hands a row to the next one *inside* the same generated method: it is not a function return, it is the literal inlined code of the next operator’s `consume()` step, spliced in at compile time — the mechanism the produce/consume protocol described in the abstract a moment ago, spelled out here in the concrete generated source.

One loop, one method: the scan’s `next()`, the columnar-to-row conversion, and the filter’s comparison all inline into `processNext()` with no call back out to a generic `Filter` object per row — this is the whole payoff whole-stage codegen exists for, made concrete instead of asserted.

Note

Not everything fuses. An Exchange always breaks the chain — you cannot generate one straight-line loop across a network boundary, so codegen restarts on the far side of every shuffle, exactly the stage boundary Section 2.4 already described from the

other direction. A sort-merge join itself, despite materializing state per key, does implement `CodegenSupport` and fuses like any other operator — this is why the specimen’s own plan (Listing 2.1) shows `*(2) SortMergeJoin Inner (9)` carrying a star rather than breaking the chain. What genuinely does not fuse is an aggregate whose buffer needs arbitrary objects (`ObjectHashAggregate` falling back past its distinct-key threshold, Section C.7) or a row-at-a-time UDF Catalyst cannot generate code for at all — both visible as a plan node with no asterisk, or the codegen number resetting around it.

17.5 Expression code generation

Beneath whole-stage codegen, an individual Catalyst Expression (`a + b * 2`, a CASE WHEN) generates its own Java source fragment rather than being evaluated through a generic interpreter tree — the same “generate code instead of interpreting a tree” idea applied one level down. A single JVM method has a 64 KB bytecode size limit; a query with a very wide SELECT or a very deep expression tree can generate code that would exceed it, and Spark’s codegen falls back to splitting the generated code across multiple smaller methods first; if the schema is wide enough to cross `spark.sql.codegen.maxFields` (100 columns by default, the same limit Section 2.5 named as a whole-stage fusion boundary), Spark instead gives up on generating code for that expression at all and falls back to plain, interpreted Expression evaluation — a real ceiling worth knowing about, since a query built by tooling that emits hundreds of generated columns can hit it in practice.

17.6 Columnar processing and vectorization

`UnsafeRow` is still fundamentally row-oriented: one contiguous buffer per row. `ColumnarBatch` (backed by Arrow-compatible `ColumnVectors`) stores a batch of rows the other way, one contiguous buffer per *column* across many rows — exactly the layout Section 12.5’s vectorized Parquet reader produces directly from the file’s own columnar encoding, with no row-oriented conversion step in between. This is also the representation a Pandas

UDF or Arrow-based Python UDF communicates through, and the reason it is fast is more specific than "same layout": ColumnarBatch and Arrow's own in-memory format are close enough in byte-level structure that handing a batch to the Python process can reuse the same underlying buffers rather than walking the data and re-encoding it — the JVM and Python sides read the identical bytes over shared memory, with no per-value (de)serialization loop in between. A row-at-a-time Python UDF has no such shortcut: every row crosses the JVM/Python boundary as its own converted Python object, one round trip per row instead of one per batch.

17.7 Summary

UnsafeRow replaces one-object-per-row with a packed byte buffer, optionally living entirely off-heap and thus entirely outside GC's view; sorting and hashing operate on compact prefix-plus-pointer arrays instead of following object references, for the sake of the CPU cache. Whole-stage codegen fuses a chain of operators into one generated method via produce/consume, visible in EXPLAIN's `*(N)` markers and readable directly via EXPLAIN CODEGEN; expression codegen does the same one level down, bounded by the JVM's 64 KB method size limit. Columnar batches are the same cache- and vectorization-friendly idea applied to a whole batch of rows at once, and are exactly what a vectorized Parquet read or an Arrow-based UDF already speaks natively.

Exercises

1. Run EXPLAIN CODEGEN on a filter-and-project query. Identify which operators were fused into one generated method and which were not.
2. Force a sort-merge join and inspect the plan's codegen markers. Identify the exact node where the `*(N)` numbering resets and explain why that operator cannot fuse.
3. Build a query with an unusually wide SELECT list (hundreds of derived columns) and observe whether codegen falls back or splits its generated method.

Chapter 18

The Shuffle Framework and the Scheduler

Chapter 4 treated the shuffle as a wall. This chapter builds the wall: the shuffle manager and its writers, the map output tracker that tells reduce tasks where to fetch from, the DAG scheduler that cuts a plan into stages, the task scheduler that places tasks on executors, and the block transfer service that moves the bytes. It is the machinery behind every stage boundary and every straggler.

18.1 The shuffle manager and shuffle writers

Every shuffle on a modern Spark version goes through `SortShuffleManager` — the separate hash-based shuffle manager from early Spark versions was removed once sort-based shuffle’s one-file-plus-index-per-task layout (Section 4.2) proved strictly better at scale. What varies is which *writer* a given shuffle uses underneath that one manager, chosen automatically — not configured — from the shuffle’s own properties, in this order. One of those properties, referenced repeatedly below, is whether the shuffle needs *map-side aggregation*: combining values that share a key (a running SUM, say) on the map side, before the shuffle write, so only one partially-combined value per key crosses the network instead of every raw value — the same partial-then-final idea Section 2.4

traced for HashAggregate. A plain shuffle-then-join has nothing to combine, so it never needs this; a GROUP BY always does.

Table 18.1: Which shuffle writer a given shuffle gets, in the order Spark checks

Writer	Fires when	Why it's cheaper
BypassMergeSort-ShuffleWriter	no map-side aggregation, and partition count $\leq \text{spark.shuffle.sort.bypassMergeThreshold}$ (200)	writes one file per partition directly and concatenates them — skips sorting entirely, cheap only because so few files need merging
UnsafeShuffleWriter	serializer supports relocating serialized values without deserializing (Kryo, under most configs), and no map-side aggregation	sorts compact (<i>partition-id, pointer</i>) arrays (Section 17.3) over already-serialized bytes, never reconstructing an object mid-sort
SortShuffleWriter	neither precondition above holds (bypass threshold exceeded, or the shuffle aggregates)	general fallback: a real record-by-record sort by partition (and key, if aggregating)

The specimen's stage-2 shuffle read (Section 1.4) is a SortMergeJoin over 200 partitions — at exactly the bypass threshold rather than under it, and sort-merge join needs its input sorted by key regardless, so both map-side stages (0 and 1) get UnsafeShuffleWriter: no map-side aggregation, Kryo in play, and the serialized-sort path is strictly cheaper than a full deserialized sort for a plain shuffle-then-join. A GROUP BY feeding a HashAggregate instead would disqualify both the bypass and serialized writers (aggregation needs the writer to combine values by key as it sorts) and fall to SortShuffleWriter regardless of partition count. The upshot: a shuffle you did not particularly optimize is not nec-

essarily running the expensive general path — Spark already picks the cheapest writer whose preconditions hold for that specific shuffle.

18.2 The map output tracker

A reduce task needs to know which executors hold which map task’s output before it can fetch anything — that registry is the *map output tracker*. The driver’s copy, `MapOutputTrackerMaster`, is the single authoritative record, updated every time a map task finishes (with its output location) and every time an executor is lost (with that executor’s outputs invalidated, which is precisely the trigger behind Section 15.4’s stage-resubmission behavior). Executors hold a read-only, periodically-refreshed cache of the same information, `MapOutputTrackerWorker`, so that a reduce task can resolve fetch locations locally most of the time without an RPC to the driver for every single task.

18.3 The DAG scheduler and the task scheduler

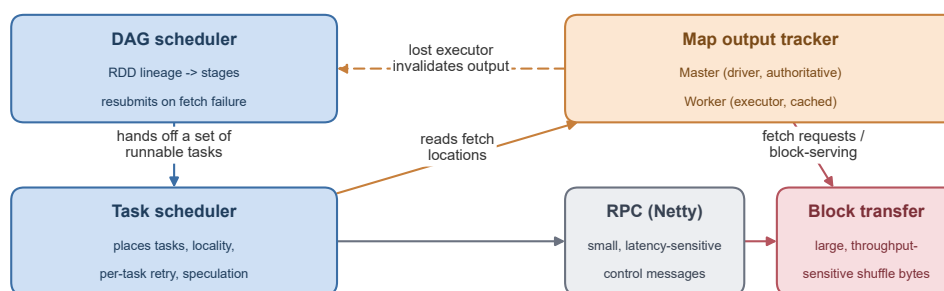
Two schedulers do two different jobs, one layered on top of the other, and the split matters because they operate on different objects: the DAG scheduler never sees an individual task’s placement, and the task scheduler never sees the dependency graph. The **DAG scheduler** builds the job/stage structure Chapter 1 and Section 1.5 already described (walking RDD lineage backward from an action, cutting a stage at every wide dependency) and owns the stage-resubmission decision behind Section 15.4’s fetch-failure recovery — what is new here is what it hands off once a stage is ready: a *set of runnable tasks*, with no further opinion on execution order or placement. The **task scheduler** (`TaskSchedulerImpl`) owns everything past that handoff: given that set and the current executors, it decides *which* task runs on *which* executor, honoring the locality preferences below and retrying a task, up to `spark.task.maxFailures`, on a different executor before giving up on the task entirely. A stage resubmission from the DAG scheduler and a task retry from the task scheduler look similar from the UI but are two different components acting at two different granularities — one re-running a whole stage’s missing map output, the other re-running one task that failed for a reason narrower than a lost executor.

18.4 RPC and block transfer

Coordination messages (task assignment, status updates, the map output tracker’s queries) travel over Spark’s internal RPC layer, built on Netty; the actual shuffle bytes travel over a separate path, the `NettyBlockTransferService`. The two are kept apart because they have opposite traffic shapes and opposite failure costs: an RPC message is small and latency-sensitive — a slow task-assignment reply stalls scheduling for every waiting task — while a shuffle fetch is large and throughput-sensitive, and can be retried or resumed without threatening the scheduler’s own responsiveness. Running both over one channel would let a handful of multi-hundred-megabyte block transfers queue up behind (or starve out) the small control messages the scheduler depends on to keep moving; separating them means a busy shuffle fetch never blocks the RPC call that would otherwise reassign a freed task slot. This split is also what makes the external shuffle service possible at all: it only has to reimplement the `NettyBlockTransferService` half as a standalone process (Section 14.4), with no need to also speak the scheduler’s RPC protocol, because block serving never depended on it. Figure 18.1 puts all five pieces from this chapter in one picture, with the RPC/block-transfer split drawn as the two separate paths it actually is.

18.5 Locality and speculation

The RPC and block-transfer split above is about *how* data moves once the scheduler has decided a task needs to fetch it; locality is the scheduler’s other lever, deciding *where* to place a task in the first place so that as little of that movement is needed at all. Placing a task on the executor that already holds its input data avoids a network read entirely, and the task scheduler ranks candidate executors by four locality levels, each one hop further from the data than the last: `PROCESS_LOCAL` means the data is already in the same JVM, so no network or even inter-process copy is needed at all; `NODE_LOCAL` means a different executor process on the same physical machine, so any transfer stays on the machine’s own loopback network rather than the cluster fabric; `RACK_LOCAL` means a different machine but one sharing the same network rack, typically a cheaper hop than a cross-rack



Two separate paths on purpose: a busy shuffle fetch on the Block transfer path never blocks the RPC call the Task scheduler needs to reassign a freed slot.

Figure 18.1: The scheduler’s five components. The DAG scheduler hands the task scheduler a set of runnable tasks and is only reinvoked when the map output tracker reports a lost executor’s output as gone; the task scheduler reads fetch locations from the tracker and issues its own small control messages over RPC, kept separate from the large shuffle bytes on the block-transfer path.

one; and ANY means no locality preference at all, the level every task can always fall back to. The scheduler will wait briefly (`spark.locality.wait`, 3s by default) for a better-locality slot to free up before falling back to a worse one. The specimen’s stage-0 tasks, reading orders directly from GCS, never see anything better than ANY: GCS is remote object storage with no notion of “the node this byte range lives on,” so there is no better level for the scheduler to wait for, and it falls straight through. Locality still does its job for stage 2’s shuffle read, though — a stage-2 task scheduled on the same node as one of its map-side executors reads that executor’s block over a local network hop rather than a cross-node one, still cheaper even though it is not `PROCESS_LOCAL`.

Speculation (already introduced as a symptom-level fix in Section 15.4) is that same task scheduler’s mechanism underneath: once a task has run past `spark.speculation.multiplier` times the median duration for its stage, it launches a second, speculative copy on a different executor and accepts whichever attempt’s result lands first. What that earlier discussion left implicit is the layer it operates at: this is a decision the task scheduler makes about one task, with no visibility into or coordination from the DAG

scheduler's stage-level view — consistent with the split above, since a straggler is a single task problem the task scheduler already owns, not a stage problem that would need the DAG scheduler involved at all.

18.6 Summary

One shuffle manager, three writers chosen by preconditions rather than configuration you set directly: bypass for few partitions, serialized-sort when the serializer allows it, plain sort otherwise. The map output tracker is the registry that makes fetching possible and that a lost executor invalidates, triggering the DAG scheduler's stage resubmission. The DAG scheduler owns stage-level lineage and boundaries; the task scheduler owns task placement within a stage, locality, and per-task retry, with speculation as its answer to a straggler that locality alone cannot explain. Both RPC coordination and the shuffle bytes themselves travel over the same underlying Netty-based transport, just on logically separate paths.

Exercises

1. Trace one wide query from `collect()` to task completion, naming each scheduler component it passes through.
2. Given a shuffle with 50 partitions and no aggregation, predict which shuffle writer handles it, then confirm from `spark.shuffle.sort.bypassMergeThreshold`.
3. Kill an executor mid-stage and, from the event log, find the map output tracker invalidation and the DAG scheduler's resubmitted stage attempt.

Chapter 19

DataSource V2 and Extensions

Iceberg is not built into Spark. It plugs in through the DataSource V2 API, and the planner negotiates with it: which filters can you apply, which columns do you need, can you do this aggregate yourself? Understanding that negotiation explains why some predicates prune and others do not. This chapter covers the DSv2 read and write API, the pushdown negotiation, how Iceberg reports splits and statistics, and how to add your own Catalyst rules and SQL extensions.

19.1 The DataSource V2 read and write API

DataSource V2 (DSv2) is the interface contract between Catalyst and an external table source, with a symmetric shape on the read and write sides. For a read, a `TableProvider` resolves a table name to a `Table`, which reports its schema and capabilities and hands back a `ScanBuilder` the planner negotiates with (Section 19.2) before it ever produces a physical scan. For a write, the same `Table` hands back a `WriteBuilder`, and from there the write path is a fixed sequence of hops from the driver out to the executors and back:

1. the `WriteBuilder` produces a `BatchWrite` (or, for a streaming sink, a `StreamingWrite`) — a driver-side object representing the write as a whole;
2. the driver asks that `BatchWrite` for one `DataWriterFactory` and ships it to every executor as part of the task’s serialized closure;

3. each task, running that factory, gets back its own `DataWriter` and independently writes its partition's rows — no coordination between tasks at this step, each one just writes;
4. each `DataWriter` finishes by returning a small `WriterCommitMessage` — a per-task summary (for Iceberg, the list of files that task wrote) — back to the driver;
5. only once *every* task's message has arrived does the driver call `BatchWrite.commit()` with the whole collected set.

That last step is the DSv2-level hook Iceberg's atomic pointer swap (Section 10.4) runs behind: nothing becomes visible until this one call, so a job that fails after some tasks finish writing simply never reaches `commit()`, and no partial write is ever visible to a reader.

This is the same interface every Iceberg catalog type from Chapter 10 implements underneath, and it is also what lets a single Spark session read an Iceberg table, a JDBC table, and a plain Parquet directory through one consistent planner path — DSv2 does not know or care that Iceberg has a metadata tree at all; from the planner's point of view, Iceberg is simply an implementation of this contract that happens to be unusually good at answering its questions cheaply.

19.2 Pushdown negotiation

Pushdown under DSv2 is a negotiation, not a guarantee: the planner offers a source a set of filters, columns, and (since Spark 3.2, for a narrow set of cases) aggregates, and the source reports back which of them it can actually apply itself. `SupportsPushDownFilters` lets Iceberg claim a filter and mark it as either fully handled (Spark drops it from the plan entirely) or only partially handled (Spark still applies it after the scan, as a safety net) — exactly the distinction behind Section C.11's pushdown-killer catalogue: a filter that defeats Iceberg's manifest-and-statistics pruning is not rejected outright, it is simply reported back as unhandled, and Spark's own `Filter` node downstream is what actually enforces it, correctly but without any pruning benefit.

For the specimen's own `o.order_ts >= TIMESTAMP '2026-03-01 ...'` filter, this

negotiation is why the scan node shows it in *both* `PartitionFilters` and `PushedFilters` (Section C.3) rather than as a separate `Filter` operator above the scan: Iceberg’s `ScanBuilder` claims the predicate as fully handled — it resolves against `day(order_ts)` for partition skipping *and* against row-group statistics for file-level skipping, so nothing is left over for Spark to re-check. Changing it to `WHERE CAST(o.order_ts AS STRING) >= '2026-03-01'` breaks the negotiation, not the filter’s correctness: the `CAST` makes the predicate one `SupportsPushDownFilters` cannot resolve against either partition values or column statistics, so it reports the filter back as unhandled, and a standalone `Filter` node reappears above the scan, evaluating the same logical condition row-by-row after every row has already been read. Figure 19.1 lines the two cases up side by side.

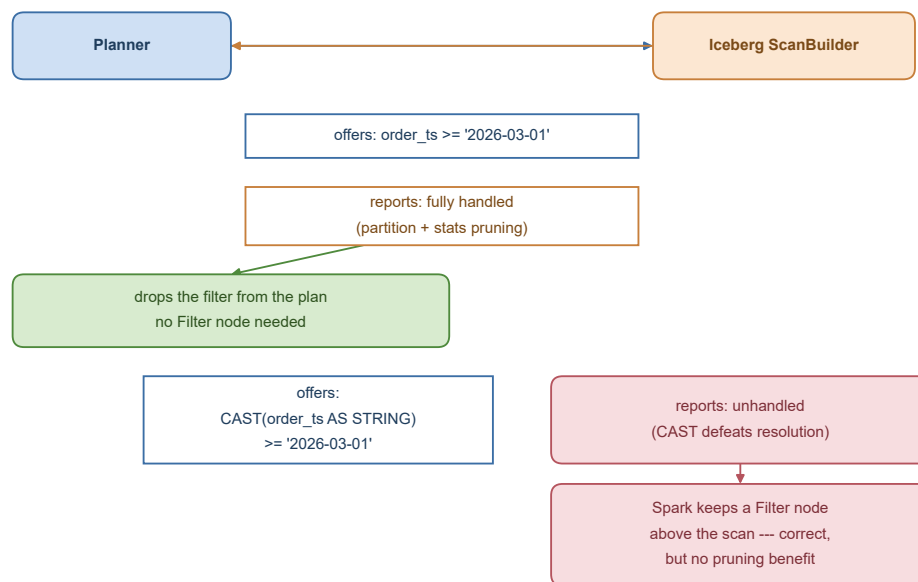


Figure 19.1: The same negotiation, two outcomes. A plain comparison resolves against partition values and column statistics, so Iceberg reports it fully handled and Spark drops the filter from the plan. A `CAST` on the same column cannot be resolved the same way, so Iceberg reports it unhandled and Spark keeps a `Filter` node to enforce it — still correct, just with no pruning benefit.

`SupportsPushDownRequiredColumns` is the same negotiation for column pruning (Section 7.5), and `SupportsPushDownAggregates` lets a narrow class of aggregates (`COUNT`, `MIN`, `MAX` on a column with the needed statistics already in the manifest) be answered from metadata alone, with no data file ever opened.

19.3 How Iceberg reports splits and statistics

A DSv2 source does not hand the planner data directly — it hands back a set of `InputPartitions` (Iceberg’s version of the split planning from Section 12.6) and a `PartitionReaderFactory` that knows how to read each one, letting the planner parallelize across them exactly like any other scan. Separately, `SupportsReportStatistics` lets Iceberg tell the planner an estimated row count and byte size for a scan *before* it runs, computed from the manifest-list summaries (Section 9.2) rather than an actual read — this is the estimate Section 5.5’s planning-time broadcast decision is built on, and it is also why an Iceberg table with well-maintained metadata tends to get better join-strategy decisions at planning time than a Hive table relying on a stale, manually triggered `ANALYZE TABLE`.

`EXPLAIN COST` on the specimen’s join surfaces exactly this number: against `dim_product` filtered by nothing, it might report `sizeInBytes=8.4MB`, `rowCount=1.2E4` — comfortably under `spark.sql.autoBroadcastJoinThreshold`’s 10 MB, which is what actually lets Chapter 5’s broadcast decision fire once statistics exist at all. Against orders filtered by the specimen’s own `order_ts` range, the reported `sizeInBytes` reflects only the seven partitions `PartitionFilters` already resolved to, not the whole table — `SupportsReportStatistics` answers *after* partition pruning has narrowed the manifest list, not before, so a tightly-pruned scan of a huge table can legitimately report itself as small.

19.4 Custom Catalyst rules and SQL extensions

Nothing about this negotiation requires modifying Spark itself. A `SparkSessionExtensions` entry point lets an external jar register additional Catalyst rules — injected into the analyzer, the optimizer, or physical planning, alongside Spark’s own rules from Section 2.3 — as well as entirely new SQL syntax the parser would not otherwise recognize. Iceberg’s own `IcebergSparkSessionExtensions` is exactly this mechanism: it is how `CALL` statements and Iceberg-specific SQL like `ALTER TABLE ... ADD PARTITION FIELD` become valid syntax at all — none of it is native Spark SQL; it is grafted on through the same public extension point available to any table source. The compaction

call this book has referenced without showing, for the small-files pressure both Section 11.4 and Section 16.7 describe, is one such statement:

```
1 CALL lakehouse.system.rewrite_data_files(  
2   table => 'analytics.orders',  
3   strategy => 'binpack'  
4 );
```

Listing 19.1: Compacting the small files a streaming write left behind

With no session extension registered, Spark’s parser has never heard of CALL at all and rejects this before it reaches any table — the concrete, checkable version of “none of it is native Spark SQL.”

19.5 Plugins

Registering a catalog (`spark.sql.catalog.<name>`, Section 10.6) and a session extension (`spark.sql.extensions`) are both just configuration keys naming a class on the classpath — concretely, this stack’s entire Iceberg installation is these two lines, plus putting the jar itself on the classpath:

```
spark.sql.catalog.lakehouse=org.apache.iceberg.spark.  
    SparkCatalog  
spark.sql.extensions=org.apache.iceberg.spark.extensions.  
    IcebergSparkSessionExtensions
```

Listing 19.2: The whole mechanism by which Iceberg is “installed” into a Spark session

The first key is what turns `lakehouse.analytics.orders` into a resolvable table name at all, by naming the class that implements the `TableCatalog` interface Section 10.1 described; the second is what teaches the SQL parser the CALL syntax the previous section used, by naming the class that registers Iceberg’s extra parser rules and Catalyst extensions into the session. No Spark source change, no rebuild — just these two keys and a jar on the classpath. Any table format or connector built to the same DSV2 and extensions contracts plugs in exactly the same way, which is the practical reason a table format can evolve on its own release cadence independent of Spark’s.

19.6 Summary

Iceberg is an ordinary DSv2 implementation from Spark's point of view: it resolves through a catalog, negotiates filter/column/aggregate pushdown by reporting back what it can handle rather than being told what to do, reports splits and statistics computed from its own manifest metadata rather than a real scan, and adds its CALL syntax and maintenance procedures through the public session-extensions mechanism. None of this required a Spark source change, and every mechanism in this section — catalog registration, pushdown negotiation, statistics reporting, SQL extension — is available to any other table format or connector built to the same contract.

Exercises

1. Write a query whose filter is pushed into Iceberg and one whose filter is not. Explain the difference from the DSv2 pushdown rules.
2. Run EXPLAIN on a `SELECT COUNT(*)` against an Iceberg table with no filter. Confirm from the plan whether it was answered from statistics alone.
3. List the classes named by `spark.sql.catalog.lakehouse` and `spark.sql.extensions` in a running session and identify which DSv2 interface each implements.

Appendix A

The Reference Environment and the Running Example

Every version-specific claim in this book is written against one stack, one cluster shape, and one running example. This appendix fixes all three, plus the columnar-format background Part III assumes.

A.1 The stack

Table A.1 is the reference environment. A claim elsewhere in the book that depends on version-specific behavior — an AQE default, a config key introduced or renamed in a particular release — is written against exactly this stack; where behavior differs meaningfully on an older or newer version, the text says so explicitly rather than leaving it implicit.

A.2 The cluster shape

Worked numbers throughout the book — executor counts, task-slot arithmetic, memory sizing — assume one concrete shape unless a section says otherwise: executors with 4 cores and 16 GB of executor memory each (plus the memory overhead Section 14.1 derives separately), running as pods on GKE with no co-located storage, since GCS is remote object storage rather than node-local disk (Section 18.5’s locality discussion

Table A.1: The reference stack

Component	Version / detail
Apache Spark	3.5.2
Table format	Apache Iceberg, format version 2
Catalog	Hive Metastore (Section 10.2)
SQL gateway	Apache Kyuubi (Section 14.6)
Cluster manager	Google Kubernetes Engine (GKE)
Object storage	Google Cloud Storage (GCS)
JVM	OpenJDK 17
File format	Apache Parquet

depends on exactly this separation). A cluster of 10 such executors gives 40 task slots — the number Section 4.4’s shuffle-partition-count reasoning starts from.

A.3 The running example: orders

The specimen dataset is an online marketplace’s order history. The fact table, `orders`, is an Iceberg table partitioned by `day(order_ts)` (Section 11.1), holding a few hundred million rows per month, with the columns in Table A.2.

Table A.2: orders schema

Column	Type	Meaning
<code>order_id</code>	<code>string</code>	Unique identifier for one order
<code>order_ts</code>	<code>timestamp</code>	When the order was placed
<code>product_id</code>	<code>string</code>	Foreign key to <code>dim_product</code>
<code>customer_id</code>	<code>string</code>	Foreign key to <code>dim_customer</code>
<code>promised_days</code>	<code>int</code>	Delivery days promised at checkout
<code>delivery_days</code>	<code>int</code>	Actual days to delivery
<code>order_total</code>	<code>double</code>	Order value, in dollars

Three dimensions complete the star schema: `dim_product` (`product_id`, `category`, `list_price`, and other catalog attributes); `dim_customer` (`customer_id`, `signup date`, `loyalty segment`); and `dim_date` (`date`, `year`, `month`, `day_of_week`), used throughout Chapter 7’s dynamic-partition-pruning examples to filter orders indirectly by calendar attributes that are not columns on the fact table itself. `dim_product` is small enough to broadcast under default settings (Section 5.5); `dim_date` is smaller still. `dim_customer` is larger — large enough that it is *not* broadcast by default, the deliberate contrast Section 5.5 uses to show a sort-merge join actually chosen rather than assumed.

Note

One product, a heavily-discounted, briefly viral item, accounts for a disproportionate share of a given week’s orders. This single hot key is the running skew example Section 1.6 first spots and Section 6.5 later fixes.

A.4 A columnar-format primer

Part III assumes familiarity with two ideas Parquet is built on, summarized here for a reader coming from row-oriented formats.

A **row-oriented** format (CSV, JSON, Avro used row-wise) stores a complete row contiguously — reading one column out of a wide table still means reading every other column’s bytes for every row along the way. A **columnar** format stores one column’s values contiguously across many rows instead, so reading a subset of columns reads only their bytes (Section 7.5’s `ReadSchema` pruning depends on exactly this layout to be worth anything) and values of the same type sit next to each other, which compresses far better than mixed-type row bytes and is what a vectorized reader (Section 12.5) processes efficiently.

Parquet organizes a file as a set of **row groups** (a horizontal slice of rows, typically sized to match the input-partition target from Section 1.4), each holding one **column chunk** per column, itself split into **pages** that are individually encoded and compressed. Every row group’s footer carries per-column **statistics** — min, max, null count — which is exactly what Section C.11’s row-group-level pushdown filters against, and what Section 12.3’s Iceberg-level metrics modes summarize one level up, at the file level, in the

manifest.

Note

"Compressed" and "encoded" are distinct steps applied in sequence: encoding (dictionary encoding, run-length encoding) exploits repeated or low-cardinality values before general-purpose compression (Snappy, Zstd, Gzip) runs on the result. This is why Section 3.2's decompressed-size warning matters: a compression ratio of 3–5× on typical analytic data means an on-disk size is not a reliable proxy for the deserialized, in-memory size Spark actually has to hold.

Appendix B

Configuration Reference

Every configuration key the book relies on to explain a mechanism, grouped by subsystem. This is a "which knob touches what, and what it costs" reference, not a recommended-values table — read the chapter cited before changing a value on a table or job you cannot afford to be wrong about.

B.1 Memory (Chapter 3)

Key	Default	Controls
<code>spark.executor.memory</code>	—	JVM heap size; the base the unified memory pool is computed from (Section 3.3).
<code>spark.executor.memoryOverhead</code>	<code>max(10%, 384 MB)</code>	Off-heap JVM needs, container-limit headroom (Section 14.1).
<code>spark.memory.fraction</code>	0.6	Share of (heap — reserved) given to the unified execution/storage pool.
<code>spark.memory.storageFraction</code>	0.5	Storage's guaranteed share within the unified pool (Section 8.4).
<code>spark.memory.offHeap.enabled</code>	false	Route unified memory off-heap, outside GC's view (Section 17.2).

Key	Default	Controls
<code>spark.serializer</code>	Java- Serial- izer	Set to Spark’s Kryo serializer class as the practical choice on this stack (Section 3.5).

B.2 Shuffle (Chapter 4, Chapter 18)

Key	Default	Controls
<code>spark.sql.shuffle.partitions</code>	200	Shuffle partition count before AQE coalescing (Section 4.4).
<code>spark.shuffle.sort.bypassMergeThreshold</code>	200	Partition-count ceiling for the cheap bypass shuffle writer (Section 18.1).
<code>spark.shuffle.io.maxRetries</code>	3	Shuffle-client retries for a transient fetch failure against a still-live executor (Section 15.4).
<code>spark.shuffle.io.retryWait</code>	5s	Delay between those retries.
<code>spark.stage.maxConsecutiveAttempts</code>	4	Stage resubmission attempts after a lost-executor fetch failure.
<code>spark.task.maxFailures</code>	4	Per-task retry ceiling before the job fails.
<code>spark.shuffle.service.enabled</code>	false	External shuffle service — decouples shuffle data lifetime from the executor’s own (Section 14.4).
<code>spark.shuffle.push.enabled</code>	false	Push-based shuffle merging on top of the external service.
<code>spark.excludeOnFailure.enabled</code>	false	Stop retries from landing back on a node already causing failures.

B.3 Adaptive Query Execution (Chapter 6)

Key	Default	Controls
<code>spark.sql.adaptive.enabled</code>	true (since 3.2)	Master switch for every mechanism in this section.
<code>spark.sql.adaptive.advisoryPartitionSizeInBytes</code>	128 MB	Coalescing target (Section 6.2).
<code>spark.sql.adaptive.coalescePartitions.minPartitionSize</code>	1 MB	Floor that stops over-merging.
<code>spark.sql.adaptive.coalescePartitions.parallelismFirst</code>	true	Protects core-count parallelism ahead of the size target (Section 6.6).
<code>spark.sql.adaptive.autoBroadcastJoinThreshold</code>	same as the planning-time key	Runtime broadcast-conversion threshold, measured against real post-shuffle bytes (Section 6.4).
<code>spark.sql.adaptive.skewJoin.skewedPartitionFactor</code>	5.0	Multiple of the median partition size that counts as skewed (Section 6.5).
<code>spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes</code>	256 MB	Absolute-size floor, required alongside the factor above.

B.4 Joins (Chapter 5)

Key	Default	Controls
<code>spark.sql.autoBroadcastJoinThreshold</code>	10 MB	Planning-time broadcast decision, against an on-disk size estimate (Section 6.4).
<code>spark.sql.join.preferSortMergeJoin</code>	true	Prefer sort-merge over shuffle-hash when both are viable (Section 5.5).

B.5 Partitioning and pruning (Chapter 7)

Key	Default	Controls
<code>spark.sql.files.maxPartitionBytes</code>	128 MB	Input-partition target for Hive-catalogued file scans (Section 1.4).
<code>spark.sql.optimizer.dynamicPartitionPruning.enabled</code>	true	DPP for a star-schema join filtered from the dimension side (Section 7.2).

B.6 Iceberg catalog and commits (Chapter 10)

Key	Default	Controls
<code>commit.retry.num-retries</code>	4	Retry attempts before a commit fails under contention (Section 10.5).
<code>commit.retry.min-wait-ms</code>	100	Initial backoff.
<code>commit.retry.max-wait-ms</code>	60,000	Backoff ceiling.
<code>commit.retry.total-timeout-ms</code>	800,000	Overall retry time budget.

B.7 Iceberg table properties (Chapter 11, Chapter 12, Chapter 13)

Key	Default	Controls
<code>write.target-file-size-byte</code>	512 MB	Write-time file-size target (Section 11.4).
<code>write.distribution-mode</code>	hash	Shuffle-before-write strategy (Section 11.5).
<code>write.delete.mode</code>	copy-on-write	Sets copy-on-write vs. merge-on-read independently per operation type. Iceberg has changed this default across releases, so confirm with <code>SHOW TBLPROPERTIES</code> rather than assuming (Section 13.1).
<code>write.update.mode</code>		
<code>write.merge.mode</code>		
<code>write.metadata.metrics.default-truncate</code>	<code>truncate(16)</code>	Manifest statistics granularity (Section 12.3).
<code>write.parquet.bloom-filter-enabled.col</code>	<code>enabled.col</code>	Enable col bloom filter for point lookups (Section 12.4).
<code>read.split.target-size</code>	128 MB	Iceberg-side input-split target (Section 12.6).
<code>read.split.open-file-cost</code>	4 MB	Assumed per-file overhead when packing splits.
<code>read.parquet.vectorization.batch-size</code>	5000	Rows per vectorized-read batch (Section 12.5).

B.8 Streaming (Chapter 16)

Key	Default	Controls
<code>spark.sql.streaming.stateStore.providerClass</code>	HDFSBackedStateStoreProvider	Switch to RocksDBStateStoreProvider to move state off-heap (Section 16.4).

B.9 Dynamic allocation and scheduling (Chapter 14, Chapter 18)

Key	Default	Controls
<code>spark.dynamicAllocation.enabled</code>	false	Master switch (Section 14.2).
<code>spark.dynamicAllocation.schedulerBacklogTimeout</code>	1s	Delay before requesting more executors under a pending-task backlog.
<code>spark.dynamicAllocation.executorIdleTimeout</code>	60s	Idle time before an executor is released.
<code>spark.dynamicAllocation.shuffleTracking.enabled</code>	true (since 3.0)	Keeps an executor alive while its shuffle output is still referenced, without needing the external shuffle service.
<code>spark.speculation</code>	false	Launch a duplicate copy of a straggling task (Section 18.5).
<code>spark.speculation.multiplier</code>	1.5	Multiple of median task duration that triggers a speculative copy.
<code>spark.locality.wait</code>	3s	Time to wait for a better-locality slot before falling back.

Appendix C

Reading Query Plans

A reference for the physical-plan operators the book refers to. Chapter 2 explains the compilation pipeline that produces these plans and how to read one quickly; this appendix is the catalogue you come back to. Everything here is as of the reference environment (Appendix A).

C.1 The six EXPLAIN modes

Form	Output
EXPLAIN	physical plan only
EXPLAIN EXTENDED	parsed, analyzed, optimized logical, and physical plans
EXPLAIN FORMATTED	physical plan as a tree + a numbered list expanding every node
EXPLAIN COST	optimized logical plan with the planner's row-count / size estimates
EXPLAIN CODEGEN	physical plan + the generated Java for each whole-stage stage
df.explain(mode=)	same five, as "simple", "extended", "formatted", "cost", "codegen"

C.2 Reading a physical plan

Read **bottom-up**: the leaves are scans, the root is the last operator to run. The tree connectors are `+-` for the child in line and `:-` plus a `:` spine for the first child of a binary operator (a join). A `*` prefix, written `*(n)`, marks an operator fused into whole-stage-generated Java stage `n` (Section C.9).

Here is that process actually applied, to the specimen’s own orders $\bowtie$ `dim_product` plan from Listing 2.1:

```
*(3) Sort (13)
+- *(3) HashAggregate (12)
  +- *(2) HashAggregate (10)
    +- *(2) SortMergeJoin Inner (9)
      :- *(1) Sort (5)
      : +- Exchange hashpartitioning(product_id, 200) (4)
      :   +- *(1) Filter (3)
      :     +- *(1) ColumnarToRow (2)
      :       +- BatchScan orders (1)
    +- *(1) Sort (8)
      +- Exchange hashpartitioning(product_id, 200) (7)
      +- BatchScan dim_product (6)
```

Reading it bottom-up, one operator at a time: node (1) is the first thing that runs — `BatchScan orders` reads the raw files. Node (2) converts the columnar batches it produces into row form. Node (3) applies the `order_ts` range filter, discarding rows outside the query’s week. That is the entire left-hand side up to node (4), an `Exchange`: the filtered rows are shuffled by `hash(product_id)` into 200 partitions, so that every row with a given `product_id` ends up in the same partition as every other row sharing it. Node (5) locally sorts each of those partitions by `product_id`. The right-hand branch, nodes (6)–(8), does the exact same three things — scan, shuffle, sort — to `dim_product`, independently and in parallel with the left branch. Only once both branches have reached a sorted, co-partitioned state does node (9), `SortMergeJoin`, run: it walks both sorted streams together, matching rows with equal `product_id`. From there, node (10) runs a partial aggregation per partition (no new exchange needed, since the join already grouped matching keys together), node (12) finishes that aggregation, and node (13) sorts the

small, already-aggregated result for final output. Every step in this reading is exactly “what does this node need before it can run, and has that already happened lower in the tree” — which is the whole discipline bottom-up reading is.

C.3 Scan operators

Where data enters the plan.

Operator	Source
BatchScan	a DataSource V2 table — Iceberg on the reference stack
FileScan parquet / orc / csv	a file-based table through the Hive catalog (DataSource V1)
InMemoryTableScan	a cached DataFrame or CACHE TABLE
DataSourceV2ScanRelation	the logical-plan form of a V2 scan

```
BatchScan orders
  PartitionFilters: [order_ts >= 2026-03-01, order_ts <
    2026-03-08]
  PushedFilters:    [IsNotNull(product_id)]
  DataFilters:      [IsNotNull(product_id)]
  ReadSchema:       struct<product_id,promised_days,
    delivery_days,order_ts>
  Batched:          true
```

Listing C.1: A scan node’s fields

PartitionFilters filters resolved against partition values, so whole partitions (directories on object storage) are skipped without being opened. Empty on a partitioned table means the filter did not match the partition transform (Chapter 7).

PushedFilters filters handed to the file reader, which skips row groups using column min/max statistics in the footer and any bloom filters, without decompressing them.

DataFilters every filter on the scan, including the residue evaluated row-by-row after reading. DataFilters minus PushedFilters is what could not be pushed.

ReadSchema the columns actually read. Wider than the query needs means a `SELECT *` defeated column pruning.

Batched: true vectorized columnar reads are enabled (`spark.sql.parquet.enableVectorizedReader`); `false` falls back to row-at-a-time decoding.

C.4 Exchange operators

Every Exchange is a shuffle and a stage boundary: serialize, write to local disk, transfer over the network, re-read on the destination. Counting them is the fastest estimate of a query’s cost.

Partitioning	In the plan	Created by
Hash	<code>hashpartitioning(col, n)</code>	<code>joins, GROUP BY, repartition(n, col)</code>
Range	<code>rangepartitioning(col ASC, n)</code>	<code>global ORDER BY, repartitionByRange</code>
Round-robin	<code>roundrobinpartitioning(n)</code>	<code>repartition(n)</code> with no column
Single	<code>SinglePartition</code>	<code>global aggregate</code> with no group key, <code>coalesce(1)</code>

Exchange `hashpartitioning(product_id, 200), ENSURE_REQUIREMENTS, [plan_id=42]`

Listing C.2: A shuffle exchange

`ENSURE_REQUIREMENTS` means the planner’s `EnsureRequirements` rule inserted this exchange because the operator above it needs its input distributed that way. `ReuseExchange` in a plan means two identical exchanges were deduplicated — one shuffle feeds both consumers.

`BroadcastExchange` is not a shuffle. It collects one small side to the driver and ships it whole to every executor; it appears only below the build side of a `BroadcastHashJoin` or `BroadcastNestedLoopJoin`.

C.5 Sort operators

```
Sort [product_id ASC NULLS FIRST], false, 0
```

The boolean is true for a global sort (a single ordered output across all partitions, preceded by a rangepartitioning exchange) and false for a local per-partition sort. A `SortMergeJoin` always shows two local sorts, one per input, between the exchanges and the join. Sorts need a sort buffer in execution memory and spill when it is exhausted (Chapter 3).

C.6 Join operators

Chapter 5 covers how the planner chooses; this is how to recognize the choice in a plan.

BroadcastHashJoin. `BroadcastHashJoin [...], BuildRight` (or `BuildLeft`) with a `BroadcastExchange` below the build side and *no* exchange on the probe (large) side. The cheapest join: no shuffle of the big table.

SortMergeJoin. `SortMergeJoin [...], Inner` with two child branches, each a `Sort` over an `Exchange` hashpartitioning. The default for two large inputs, and the most expensive pattern — two full shuffles plus two sorts.

ShuffledHashJoin. `ShuffledHashJoin [...], BuildRight` with two `Exchange` nodes but *no* sorts: one side is shuffled and built into a hash table in memory. Cheaper than sort-merge when the build side fits and the sort is the bottleneck; enabled by `spark.sql.join.preferSortMergeJoin=false` or a `SHUFFLE_HASH` hint.

BroadcastNestedLoopJoin. A non-equi join condition (`order_total > credit_limit`). $O(N \times M)$. Fine when one side is tiny; a warning sign on two large tables — usually a missing or non-equi join key.

```
BroadcastNestedLoopJoin BuildRight, Inner, (order_total#22 >
    credit_limit#41)
:- *(1) BatchScan orders
```

```
+-- BroadcastExchange IdentityBroadcastMode
    +- *(2) Filter (segment#39 = VIP)
        +- *(2) BatchScan dim_customer
```

Listing C.3: BroadcastNestedLoopJoin on a non-equi condition

There is no `Exchange` `hashpartitioning` anywhere in this plan — the giveaway that the join key is not an equality. `dim_customer` itself is too large to broadcast by default (Section A.3), but here it is filtered down to its small VIP segment first, and it is *that* filtered result, small enough to broadcast, that the `BroadcastExchange` ships whole; every task on the orders side then compares each of its rows against every row of that broadcast copy, checking `order_total > credit_limit` directly. This is only affordable because the broadcast side is small; the same plan shape with neither side small enough to broadcast falls to `CartesianProduct` instead.

CartesianProduct. Every row of one side with every row of the other. Unless it is a deliberate cross join, the `ON` clause is missing.

C.7 Aggregation operators

Aggregation is two-phase: a partial aggregate on each partition (map-side combine), a shuffle, then a final aggregate.

```
*(3) HashAggregate(keys=[product_id], functions=[avg(on_time)])
+- Exchange hashpartitioning(product_id, 200),
    ENSURE_REQUIREMENTS
    +- *(2) HashAggregate(keys=[product_id], functions=[
        partial_avg(on_time)])
    +- ...
```

Listing C.4: Two-phase aggregation

Reading bottom-up: the lower `partial_avg(on_time)` runs once per input partition, producing one partial running-sum-and-count per `product_id` it saw locally; the `Exchange` then redistributes those small partial results by `product_id`, so every partial

result for the same product lands together; and the top `avg(on_time)` combines the partials that arrive for each key into the query’s final per-product average. Nothing about the specimen’s real values is shuffled directly — only the already-shrunk partial aggregates are.

Operator	Chosen when	Memory
HashAggregate	all aggregate buffers are fixed-width (numeric, date, timestamp)	off-heap hash map; falls back to sort-based under memory pressure
ObjectHashAggregate	a buffer holds a variable-width or complex type (collect_list, collect_set, some UDAFs)	on-heap objects; falls back to sort-based past <code>spark.sql.objectHashAggregate.sortBased fallbackThreshold</code> (128) distinct keys
SortAggregate	hash aggregation is not feasible	needs input pre-sorted by the group keys; handles any size, slower

C.8 Filter and Project

Filter applies a predicate; Project selects columns and evaluates expressions. Both are usually fused into a neighbouring scan or join inside a `*(n)` block. A standalone Filter directly above a scan is worth a look: if its predicate is not also in the scan’s `PushedFilters`, the pushdown failed and the predicate is being evaluated row-by-row after reading (Section C.11).

C.9 Whole-stage codegen markers

```

*(2) HashAggregate(keys=[city], functions=[sum(revenue)])
+- Exchange hashpartitioning(city, 200)
   +- *(1) HashAggregate(keys=[city], functions=[partial_sum(
       revenue)])
      +- *(1) Filter isnotnull(city)
         +- *(1) ColumnarToRow

```

```
+ - BatchScan sales [city, revenue]
```

`*(1)` is one generated Java method: scan, columnar-to-row, filter, and partial aggregate run without leaving it. The Exchange breaks the chain — you cannot fuse across a network boundary — so the final aggregate is `*(2)`, a second generated method on the post-shuffle data. The number is a sequential id, not an order or a priority. Operators with no star ran through the interpreter; Section 2.5 lists why a stage loses codegen.

C.10 AQE nodes in a final plan

When Adaptive Query Execution is on (Chapter 6), the plan the SQL tab shows after the query finishes is rooted at `AdaptiveSparkPlan isFinalPlan=true`. `isFinalPlan=false` is the compile-time plan from `explain()` before an action — it may change substantially at runtime.

`AQEShuffleRead coalesced` AQE merged small post-shuffle partitions after seeing their real sizes.

`AQEShuffleRead (skew=true)` AQE split one or more oversized partitions.

`BroadcastQueryStage` a `SortMergeJoin` was converted to a broadcast at runtime because the shuffled side turned out small.

`SortMergeJoin [...], isSkew=true` AQE detected and handled skew in this join.

Here is one final plan showing several of these together, for a join between orders and the genuinely large `dim_customer` (Section A.3) during a week hit by the viral product's skew — both sides too large to broadcast, so this is a real sort-merge join, not one AQE later converts:

```
AdaptiveSparkPlan isFinalPlan=true
+ - *(3) HashAggregate(keys=[category], functions=[avg(on_time)
  ])
  + - AQEShuffleRead coalesced
    + - ShuffleQueryStage 2
```

```

+- *(2) SortMergeJoin [product_id], [product_id],
    Inner, isSkew=true
:- AQEShuffleRead (skew=true)
:  +- ShuffleQueryStage 0
+- AQEShuffleRead (skew=true)
    +- ShuffleQueryStage 1

```

Listing C.5: A final AQE plan: a skew-split join feeding a coalesced aggregation

The root confirms this is the plan as it actually ran, not the compile-time guess. Both join inputs (ShuffleQueryStage 0 and 1) carry AQEShuffleRead (skew=true): AQE found the viral product’s oversized partition after the shuffle and split it into several sub-partitions on the orders side, and replicated the corresponding dim_customer partition to match, exactly Section 6.5 described. The join node’s own isSkew=true flag confirms this handling applied specifically to this join, not just somewhere upstream. Above the join, AQEShuffleRead coalesced shows a separate, independent AQE decision: the aggregation’s own shuffle output, originally many small partitions, merged back down toward the 128 MB target before the final HashAggregate runs over it — one query, two unrelated runtime-measured corrections, each visible as its own marker in the same final plan.

A BroadcastQueryStage node appears in a different scenario: recall from Section 6.3 that the specimen’s orders $\bowtie$ dim_product join is planned as sort-merge (Iceberg has no column-level statistics to broadcast it up front), but dim_product turns out small once its own shuffle is actually measured. In *that* final plan, the join operator itself is no longer SortMergeJoin at all — AQE rewrites it to BroadcastHashJoin, with BroadcastQueryStage standing in for the exchange-and-sort pair that would have run on the now-broadcast side, and orders reads through with no shuffle on that side either. A join cannot show both a skew-split shuffle read *and* a broadcast promotion on the same side: promoting to broadcast removes that side’s shuffle entirely, and skew-split handling has nothing left to split once there is no post-shuffle partition to measure.

C.11 Predicate-pushdown killers

Predicates that cannot be pushed to Parquet, ORC, or Iceberg, so they run row-by-row after the read (Chapter 7):

- a UDF: `WHERE my_udf(col) = v`
- a function or complex expression on the column: `WHERE length(city) > 5`,
`WHERE a + b > 100`
- a CAST on the column side: `WHERE CAST(s AS INT) > 100`
- a non-deterministic function: `WHERE rand() > 0.5`
- a leading-wildcard LIKE: `WHERE name LIKE '%smith'` (a trailing wildcard, `'smith%'`, does push)
- an OR with any non-pushable branch — the whole OR fails

Verification. Run `EXPLAIN FORMATTED`, find the scan, and check that each partition-column filter is in `PartitionFilters` and each data-column filter is in `PushedFilters`. If one is missing, it hit a killer above.

Take `WHERE CAST(product_id AS INT) > 1000` against the specimen's orders, where `product_id` is a string (Section A.3). The scan node's detail block reads:

```
(1) BatchScan orders
    PushedFilters:      []
    DataFilters:        [(cast(product_id as int) > 1000)]
    ReadSchema:         struct<product_id,order_ts,...>
```

Listing C.6: A CAST silently killing pushdown

`PushedFilters` is empty — nothing made it to the storage layer at all — while the exact same condition sits in `DataFilters`, meaning every row of every file is read and decoded before this predicate is checked row-by-row in Spark itself. Compare that to the clean version of the same intent, `WHERE product_id > '1000'` (a plain string comparison, no CAST): the scan's `PushedFilters` would show `[NotNull(product_id),`

`GreaterThan(product_id,1000)]` instead, and `DataFilters` would be empty — the same logical filter, but now resolved by Iceberg’s own statistics before a row is ever decoded. The `CAST` is what moved it from one list to the other.

Appendix D

Glossary

Terms as this book uses them, in the order they are first introduced.

Driver The process holding the `SparkSession`, running the application’s own code, and coordinating everything else (Chapter 1).

Executor A JVM process that runs tasks and holds data in memory or on disk for a running application (Chapter 1).

Job The unit of work triggered by one action; made of one or more stages (Chapter 1).

Stage A set of tasks that can run without a shuffle between them, bounded by wide-dependency shuffle boundaries (Chapter 1).

Task The smallest unit of scheduled work: one stage’s operators applied to one partition (Chapter 1).

Narrow dependency A child partition depends on a known, fixed set of parent partitions — no shuffle needed (Chapter 1).

Wide dependency A child partition may depend on data from any parent partition — requires a shuffle (Chapter 1).

Catalyst Spark SQL’s query optimizer: parse, analyze, optimize, plan, generate code (Chapter 2).

Whole-stage code generation Fusing a chain of operators into one generated Java method via produce/consume (Section 17.4).

Unified memory The single JVM memory pool execution and storage share and borrow from, with a boundary either side may cross into the other's currently-idle space, but which only execution can enforce by evicting the borrower back out (Section 3.3).

Spill Writing intermediate data (a sort buffer, a hash table) to disk because it does not fit in its memory budget (Chapter 3).

Shuffle Redistributing data across partitions between two stages, via a map-side write and a reduce-side fetch (Chapter 4).

Broadcast hash join A join where one side is small enough to be sent whole to every executor, avoiding a shuffle of the large side (Chapter 5).

Sort-merge join A join where both sides are shuffled and sorted by join key, then merged — the robust default for two large sides (Chapter 5).

Adaptive Query Execution (AQE) Re-planning after each shuffle using measured, rather than estimated, partition sizes (Chapter 6).

Skew One partition or key holding disproportionately more data than others, creating a straggler task (Section 6.5).

Partition pruning Skipping whole partitions (or files) based on a filter resolved before reading data (Chapter 7).

Dynamic partition pruning (DPP) Partition pruning using a value discovered at runtime from the other side of a join (Section 7.2).

Bucketing Dividing data within a partition into a fixed number of files by hashing a column, to avoid a shuffle on a repeated join (Section 7.3).

Storage-partitioned join A join with no shuffle because both sides are already co-located by the same bucket/partition transform (a partition transform — see Hidden partitioning, below) (Section 5.7).

Manifest An immutable file listing a set of data or delete files with their partition values and column statistics (Chapter 9).

Manifest list A per-snapshot file listing the manifests that make up that snapshot, with per-manifest summary statistics (Chapter 9).

Snapshot One committed version of a table's state, with its own sequence number and manifest list (Section 9.3).

Sequence number A monotonically increasing number recording a snapshot's relative age, used to decide which delete files apply to which data files (Section 9.3).

Position delete A delete recorded as an exact (file, row-position) pair — cheap to apply, requires the writer to know the row's position (Section 9.4).

Equality delete A delete recorded as a column-value match — cheap to write, progressively more expensive to apply as they accumulate, because every read must check each one against every candidate row (Section 9.4).

Copy-on-write (CoW) A row-level update strategy that rewrites whole affected data files (Section 13.1).

Merge-on-read (MoR) A row-level update strategy that records a delete file instead of rewriting data (Section 13.1).

Hidden partitioning Iceberg partitioning by a transform of a source column, resolved from a plain filter with no partition column in the query text (Section 11.1).

Catalog The component mapping a table name to its current metadata file location, atomically swappable on commit (Section 10.1).

Optimistic concurrency A commit protocol where writers do the expensive work assuming they will win, retrying only the cheap final compare-and-swap on conflict (Section 10.4).

UnsafeRow Tungsten's packed, off-heap-capable binary row format (Section 17.1).

DataSource V2 (DSv2) The Spark API contract an external table source (Iceberg included) implements to plug into the planner (Chapter 19).

Abbreviations

Abbreviation	Full form
AQE	Adaptive Query Execution
CBO	Cost-Based Optimization
CoW / MoR	Copy-on-Write / Merge-on-Read
DAG	Directed Acyclic Graph
DPP	Dynamic Partition Pruning
DSv2	DataSource V2 API
GC	Garbage Collection
GCS	Google Cloud Storage
GKE	Google Kubernetes Engine
HMS	Hive Metastore
JIT	Just-In-Time (compilation)
JVM	Java Virtual Machine
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing
RDD	Resilient Distributed Dataset
RPC	Remote Procedure Call
SMJ	Sort-Merge Join
SQL	Structured Query Language
UDF	User-Defined Function

Technologies

Technology	What it is
Apache Spark	a distributed compute engine
Catalyst	Spark SQL's extensible query optimizer
Tungsten	Spark's binary in-memory execution back end
Apache Iceberg	an open table format for analytic datasets on object storage
Apache Parquet	a columnar file format
Hive Metastore	a catalog service mapping table names to metadata
Apache Kyuubi	a multi-tenant SQL gateway in front of Spark
Google Cloud Storage	an S3-compatible object storage system
GKE	Google's managed Kubernetes; the cluster manager here
async-profiler	a low-overhead JVM sampling profiler

Appendix E

Further Reading

Full citations are in the bibliography. This appendix groups them, plus a few further pointers, by the part of the book they extend.

Foundations (Part I) and the engine’s own papers

The original research papers behind Spark’s execution model are short, readable, and still the clearest statement of *why* the engine is shaped the way Chapter 1 and Chapter 2 describe it: *Resilient Distributed Datasets* [15] (the RDD abstraction and lineage-based fault tolerance underlying every job/stage/task concept in this book), *Spark SQL: Relational Data Processing in Spark* [7] (Catalyst itself), and the retrospective *Apache Spark: A Unified Engine for Big Data Processing* [14]. *Spark: The Definitive Guide* [9] and *Learning Spark, 2nd ed.* [10] cover the same ground at tutorial pace, useful where this book moves quickly from mechanism to mechanism.

Making queries fast (Part II)

High Performance Spark, 2nd ed. [11] is the closest comp to this book’s own emphasis — shuffle, joins, and partitioning as performance levers rather than API surface — and is the natural next stop after Chapter 4–Chapter 8. The official *Tuning Spark* guide [5] and *Spark SQL, DataFrames and Datasets Guide* [4], both pinned to this book’s reference version (Appendix A), are worth checking directly whenever a config default is suspected

to have changed since.

The Iceberg lakehouse (Part III)

The *Lakehouse* paper [6] is the argument for why a table format over object storage exists at all, ahead of Iceberg specifically. For Iceberg itself: the official *Apache Iceberg Table Spec* [2] is the primary source for everything in Chapter 9 on the metadata tree and delete files, and the *Apache Iceberg Documentation* [1] covers catalogs, maintenance procedures, and Spark-side configuration in more day-to-day detail than this book's mechanism-first treatment. *Apache Iceberg: The Definitive Guide* [13] is the book-length comp, and the Cazpian engineering blog (<https://cazpian.ai/blog>) was a useful secondary source while drafting this part — cross-checked against the official spec and docs above wherever the two disagreed, most notably on maintenance-operation ordering (Section 13.6).

Operating Spark (Part IV)

The *Structured Streaming* paper [8] is the primary source behind Chapter 16's micro-batch and exactly-once model. *Apache Kyuubi Documentation* [3] covers engine share levels and pooling (Section 14.6) in more depth than a single chapter section can. *Designing Data-Intensive Applications* [12] is not Spark-specific but is the best available treatment of the distributed-systems ideas this book leans on throughout — partitioning, replication, consistency, fault tolerance — for a reader who wants the general theory behind a specific mechanism.

Internals (Part V)

Most of Chapter 17–Chapter 19 has no single canonical paper; the most reliable way to go deeper is the Spark source itself, read alongside *Spark: The Definitive Guide* [9]'s higher-level description of the same components and the official *Spark SQL, DataFrames and Datasets Guide* [4] for whichever configuration surface a given internal exposes:

Whole-stage codegen `sql.execution` package (Chapter 17).

The shuffle framework `shuffle` package (Chapter 18).

DataSource V2 `sql.connector` package (Chapter 19).

References

- [1] Apache Iceberg. *Apache Iceberg Documentation*. <https://iceberg.apache.org/docs/latest/>. 2024.
- [2] Apache Iceberg. *Apache Iceberg Table Spec*. <https://iceberg.apache.org/spec/>. Format version 2. 2024.
- [3] Apache Kyuubi. *Apache Kyuubi Documentation*. <https://kyuubi.apache.org/docs/latest/>. 2024.
- [4] Apache Spark. *Spark SQL, DataFrames and Datasets Guide (Spark 3.5.2)*. <https://spark.apache.org/docs/3.5.2/sql-programming-guide.html>. 2024.
- [5] Apache Spark. *Tuning Spark (Spark 3.5.2)*. <https://spark.apache.org/docs/3.5.2/tuning.html>. 2024.
- [6] Michael Armbrust et al. “Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics”. In: *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*. 2021.
- [7] Michael Armbrust et al. “Spark SQL: Relational Data Processing in Spark”. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. ACM, 2015, pp. 1383–1394.
- [8] Michael Armbrust et al. “Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark”. In: *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*. ACM, 2018, pp. 601–613.
- [9] Bill Chambers and Matei Zaharia. *Spark: The Definitive Guide*. O’Reilly Media, 2018.

- [10] Jules S. Damji et al. *Learning Spark: Lightning-Fast Data Analytics*. 2nd ed. O'Reilly Media, 2020.
- [11] Holden Karau, Adi Polak, and Rachel Warren. *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark*. 2nd ed. O'Reilly Media, 2026.
- [12] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [13] Tomer Shiran, Jason Hughes, and Alex Merced. *Apache Iceberg: The Definitive Guide*. O'Reilly Media, 2024.
- [14] Matei Zaharia et al. "Apache Spark: A Unified Engine for Big Data Processing". In: *Communications of the ACM* 59.11 (2016), pp. 56–65.
- [15] Matei Zaharia et al. "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing". In: *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2012, pp. 15–28.